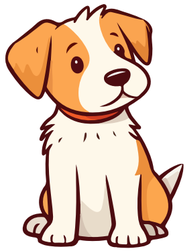
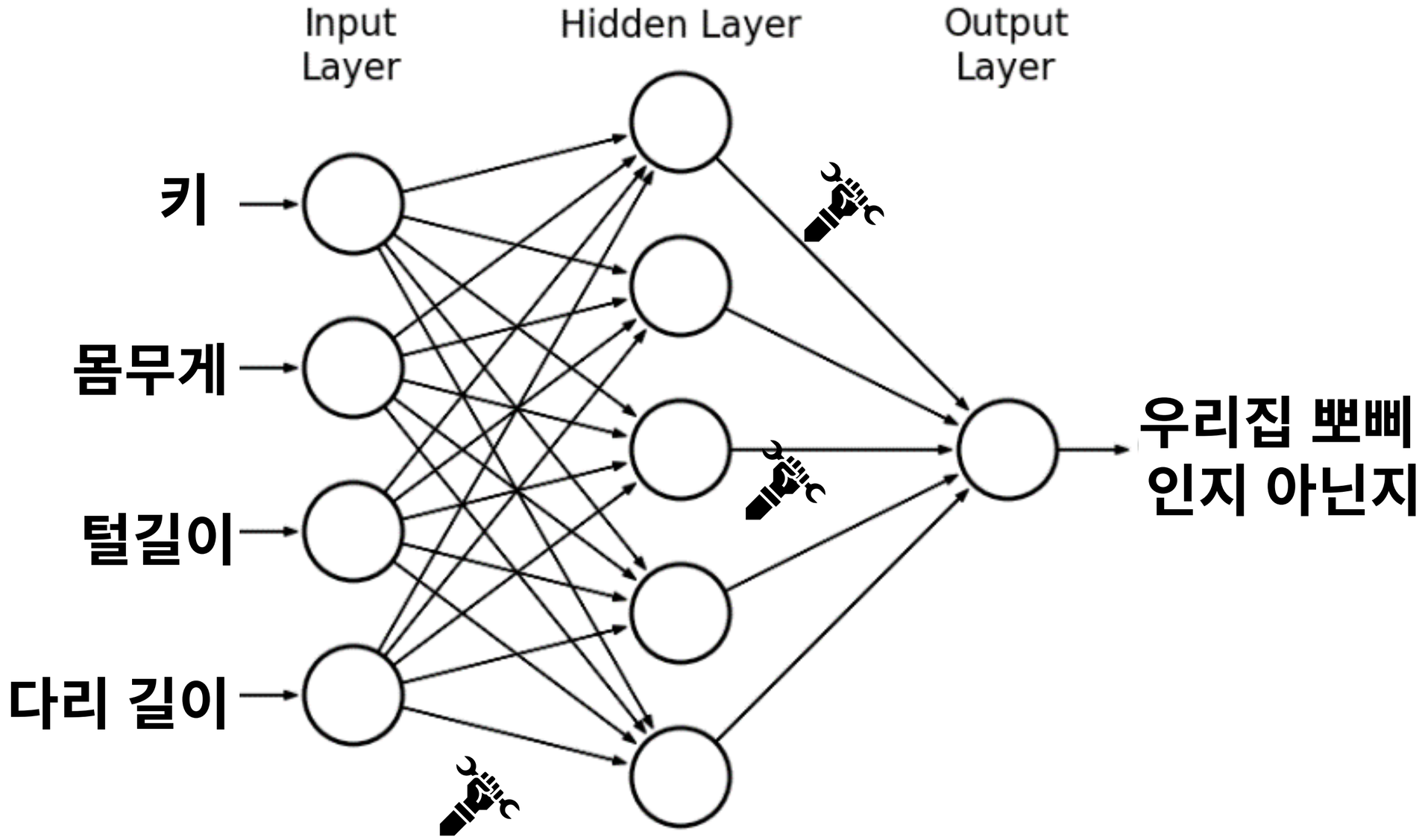
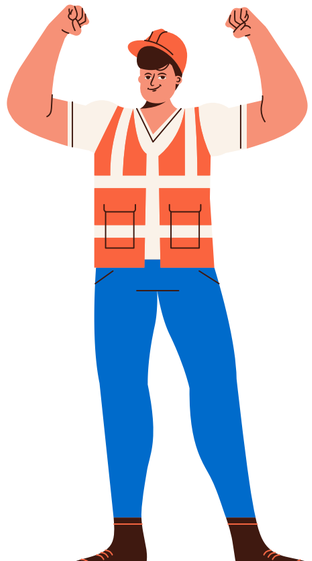
오류역전파

여전히..HOW?

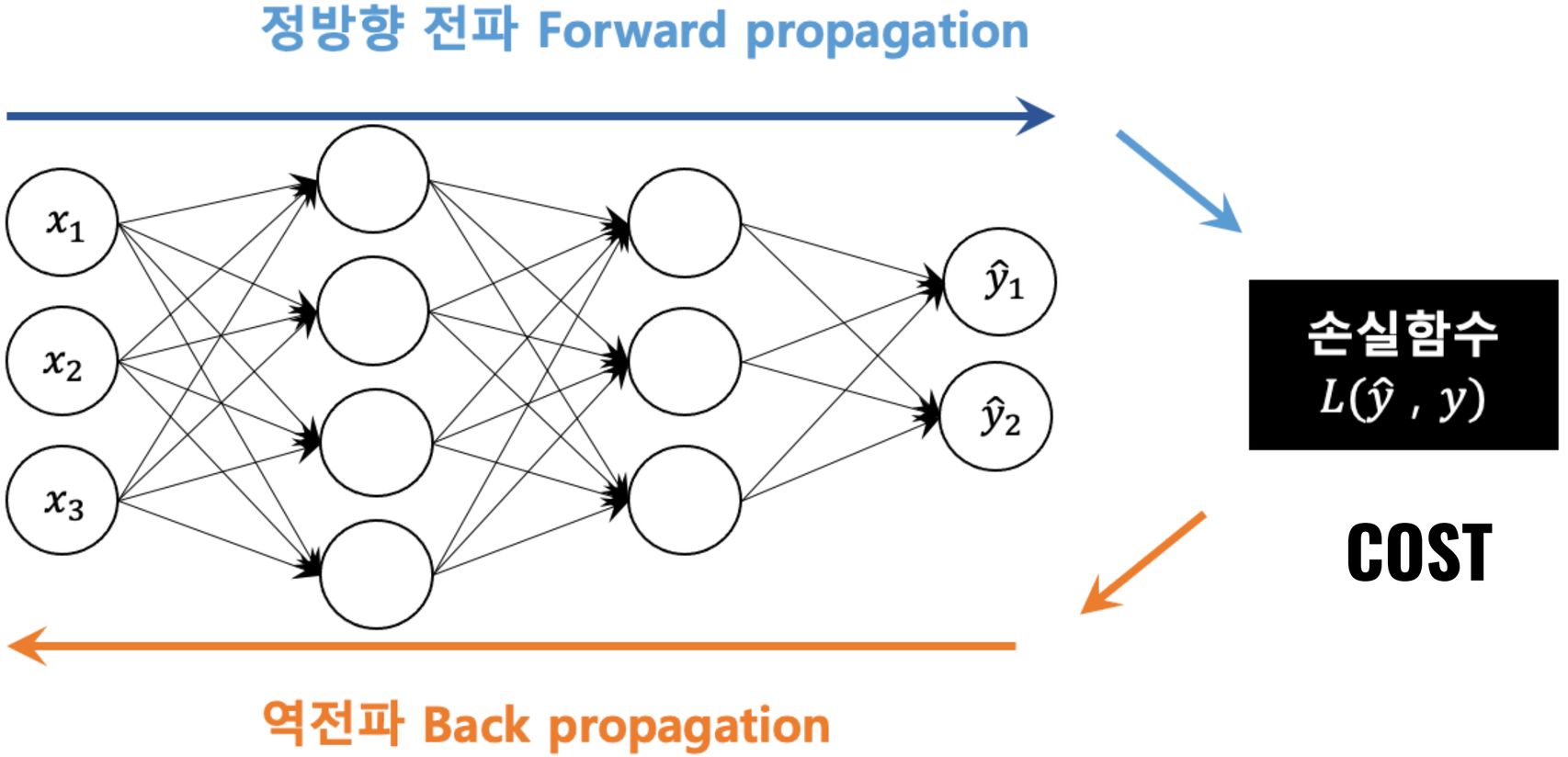



Input Layer
Hidden Layer
Output Layer
키
몸무게
털길이
다리 길이
우리집 뽀삐 인지 아닌지



ERROR BACKPROPAGATION (오류 역-전파)

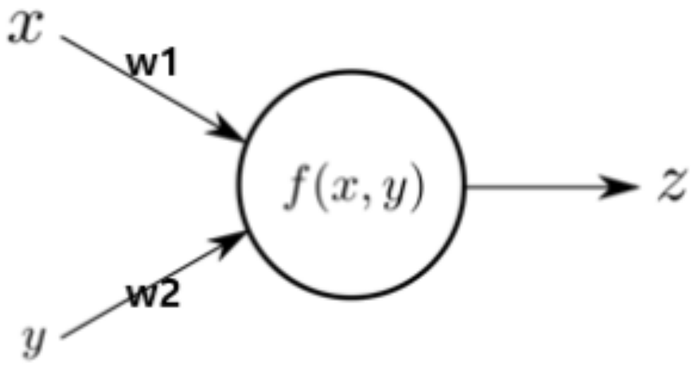
제프리 힌튼



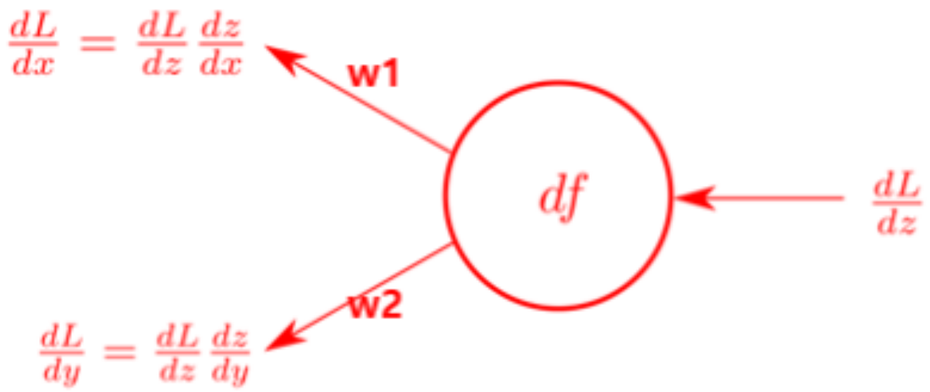
오류에 기여한 만큼 조정하면되지!

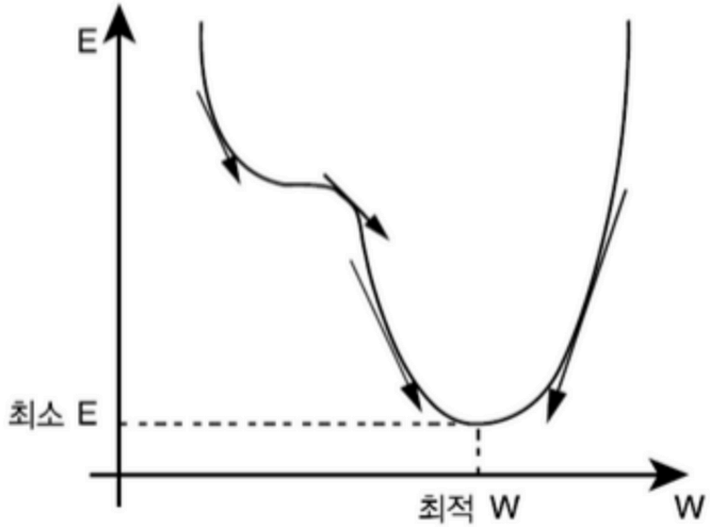
손실함수
$L(\hat{y}, y)$

Forwardpass



Backwardpass





$$w(n+1) = w(n) - \alpha \frac{\partial E}{\partial W}$$

미분: 각 항목이 손실에 기여한 정도를 구해

(어짜피 계산 가능한 수식으로 진행되었을테니 각각을 미분해!)

그리고 그 값에서 빼!

패널티를 주는거죠

<u>그리고 CHAIN RULE로 다 연결하면 돼!!!</u>

미분의 정의

$$\frac{df(x)}{dx} = \lim_{h \to 0} \frac{f(x+h) - f(x-h)}{2h}$$

합성다항식 미분

$$f = (x+y)^2 \rightarrow f' = 2(x+y) \cdot 1$$

$$f = \{(x+y)^3 + z\}^2 \rightarrow f' = 2\{(x+y)^3 + z\} \cdot 3(x+y) \cdot 1$$

CHAIN RULE

$$z = g(y), y = f(x)$$

$$z = (g \circ f)(x) = g(f(x))$$

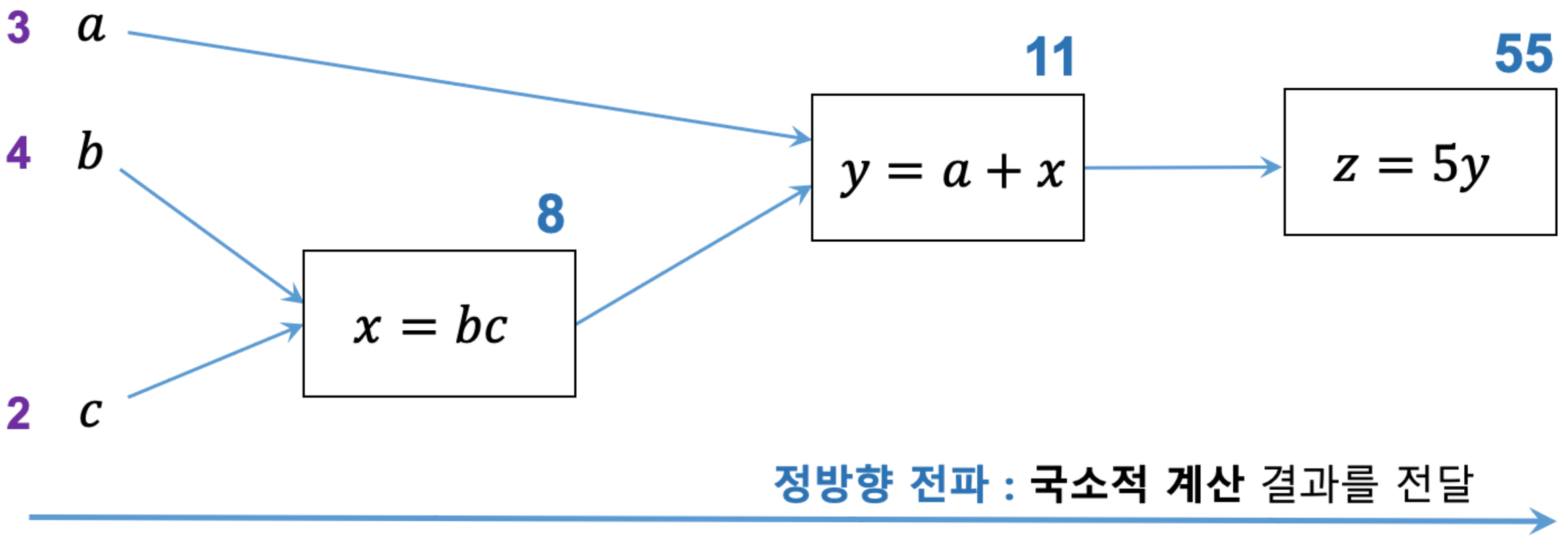
미분 →

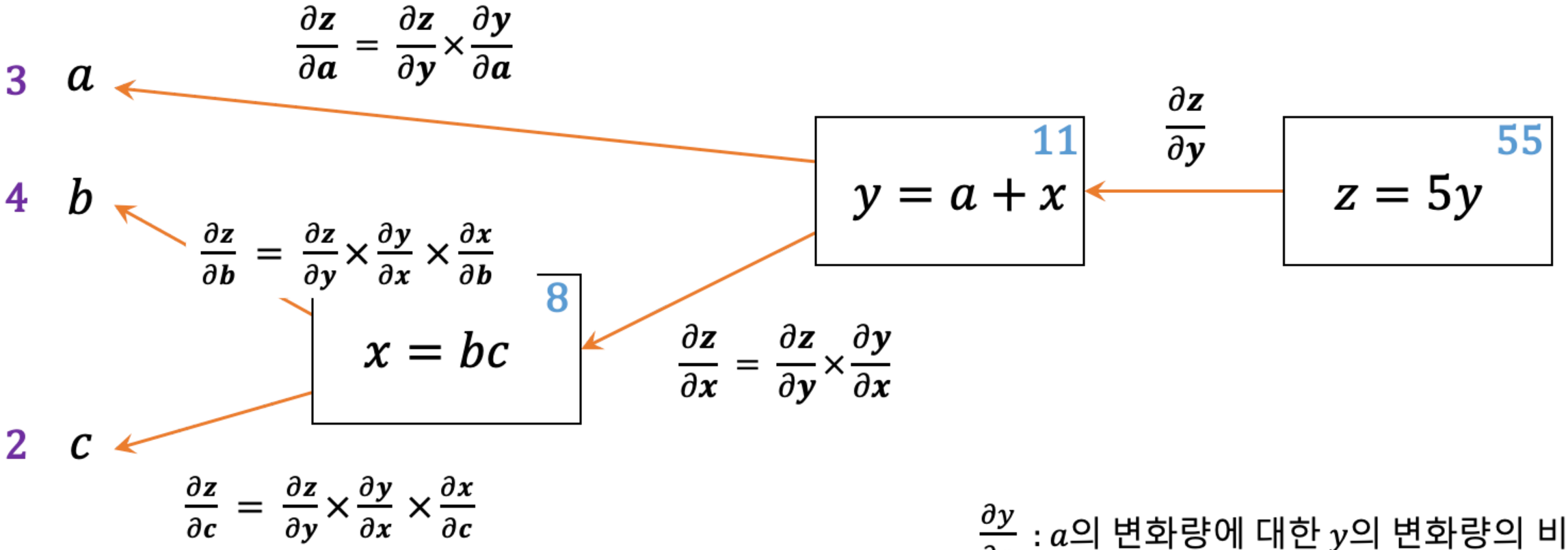
$$(g \circ f)'(x) = g'(y)f'(x)$$

↓ Chain rule

$$\frac{dz}{dx} = \frac{dz}{dy} \cdot \frac{dy}{dx}$$

HTTPS://AMBER-CHAEEUNK.TISTORY.COM/18


3 a
4 b
2 c
8
x = bc
11
y = a + x
55
z = 5y
정방향 전파 : 국소적 계산 결과를 전달

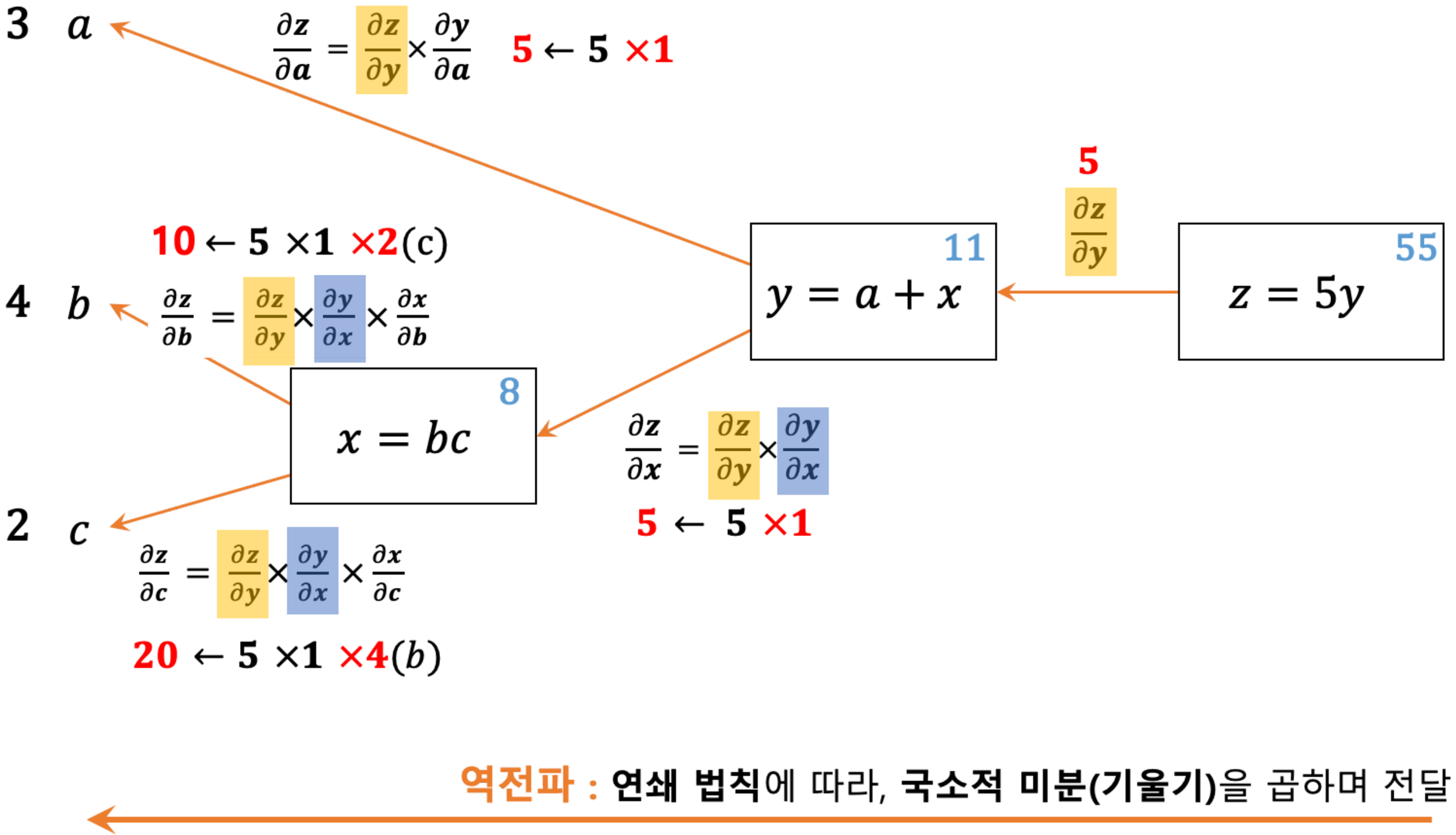


$\frac{\partial y}{\partial a}$: a의 변화량에 대한 y의 변화량의 비율

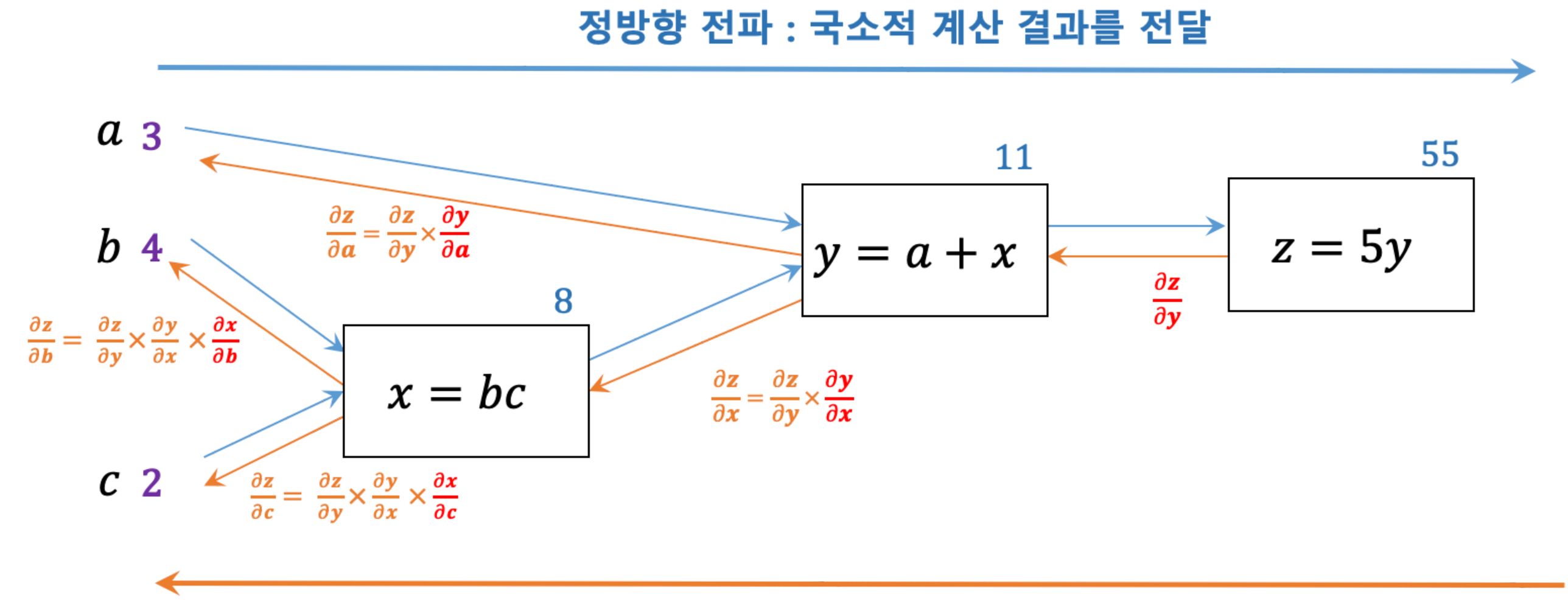
역전파 : **연쇄 법칙**에 따라, **국소적 미분(기울기)**을 곱하며 전달

합성함수의 미분은 합성함수를 구성하는 각 함수의 미분의 곱으로 나타낼 수 있다.

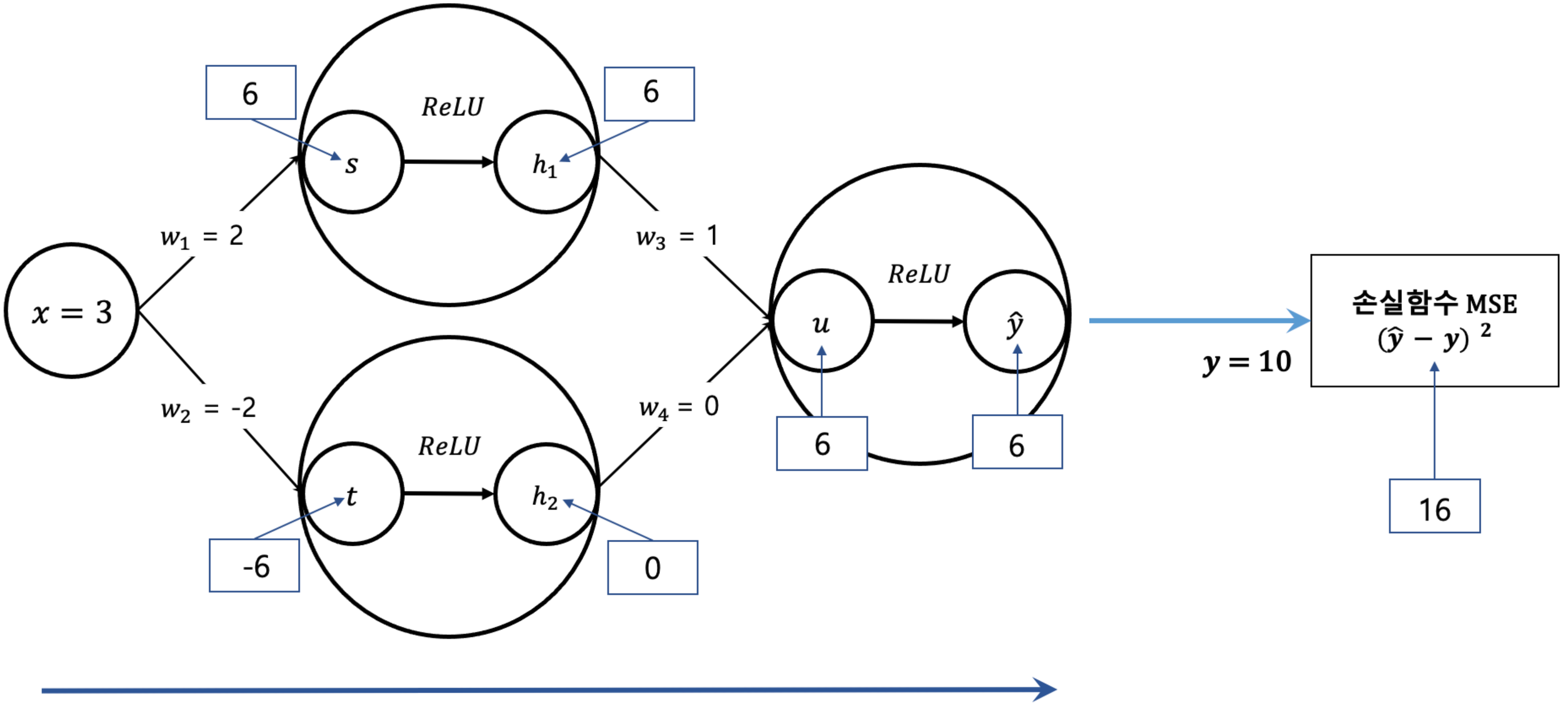
$ex)$ $z = z(y(a))$ 일 때, $\frac{\partial z}{\partial a} = \frac{\partial z}{\partial y} \times \frac{\partial y}{\partial a}$

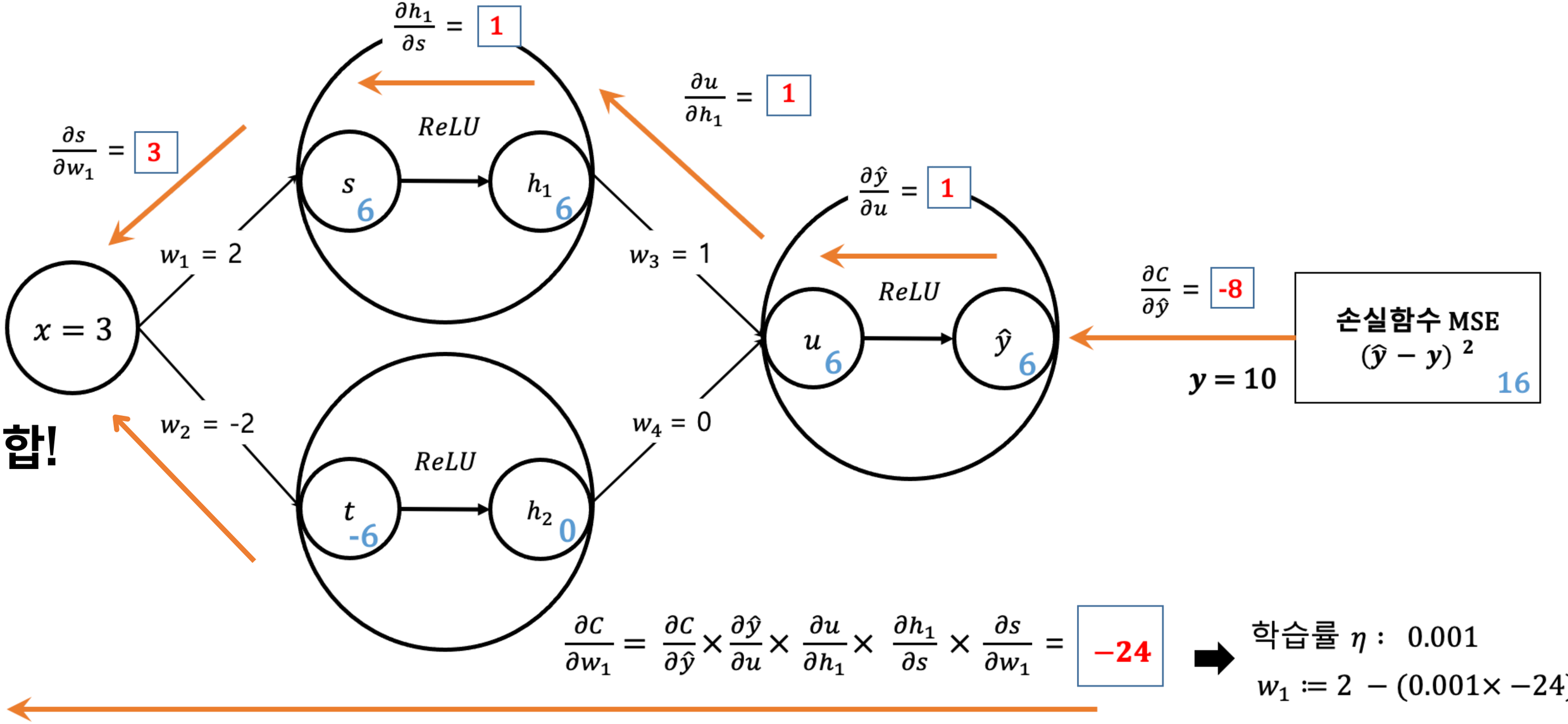


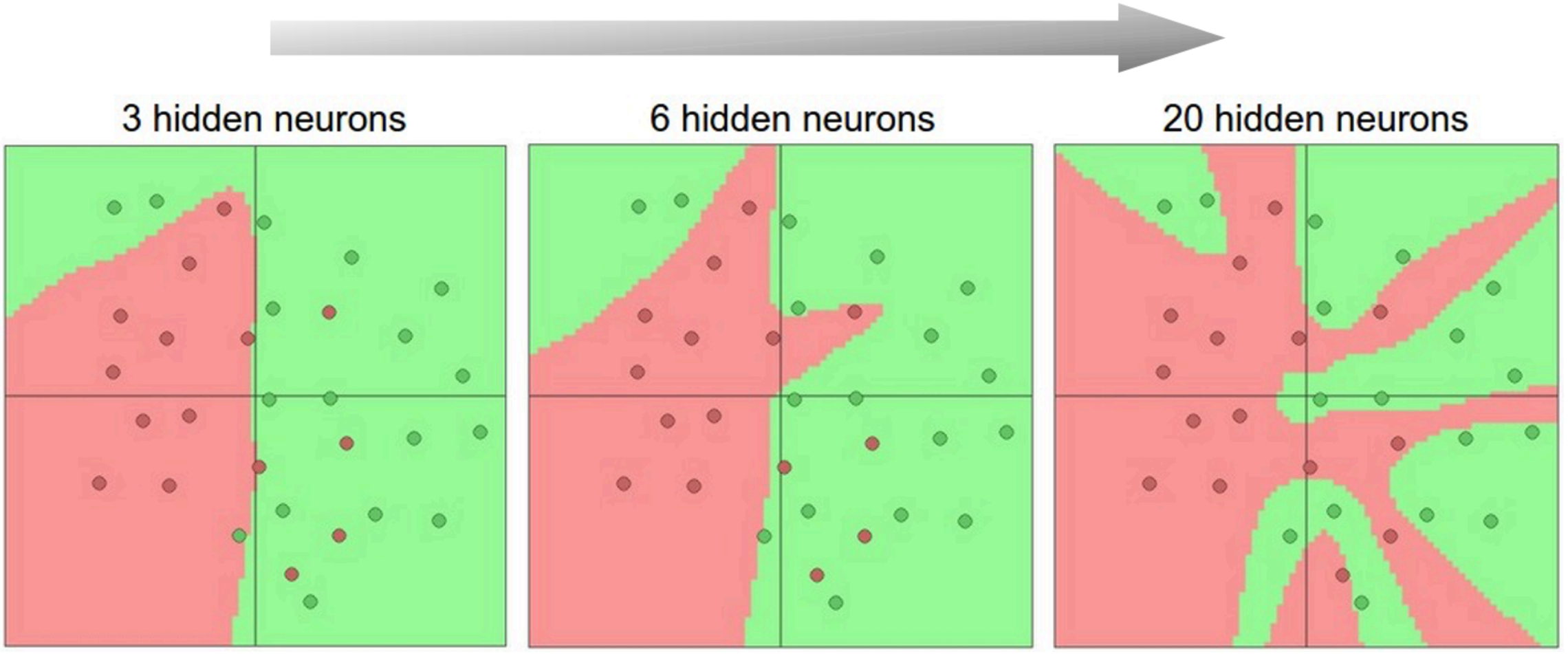
역전파 : 연쇄 법칙에 따라, **국소적 미분(기울기)**을 곱하며 전달


정방향 전파 : 국소적 계산 결과를 전달
a 3
b 4
c 2
11
55
8
$y = a + x$
$z = 5y$
$x = bc$
$\frac{\partial z}{\partial a} = \frac{\partial z}{\partial y} \times \frac{\partial y}{\partial a}$
$\frac{\partial z}{\partial b} = \frac{\partial z}{\partial y} \times \frac{\partial y}{\partial x} \times \frac{\partial x}{\partial b}$
$\frac{\partial z}{\partial y}$
$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y} \times \frac{\partial y}{\partial x}$
$\frac{\partial z}{\partial c} = \frac{\partial z}{\partial y} \times \frac{\partial y}{\partial x} \times \frac{\partial x}{\partial c}$
역전파 : 연쇄 법칙에 따라, 국소적 미분(기울기)을 곱하며 전달

신경망에서는?


6
ReLU
6
s
h_1
w_1 = 2
w_3 = 1
$x = 3$
ReLU
u
$\hat{y}$
w_2 = -2
w_4 = 0
ReLU
t
h_2
6
6
-6
0
$y = 10$
손실함수 MSE
$(\hat{y} - y)^2$
16


$\frac{\partial h_1}{\partial s} = 1$
$\frac{\partial u}{\partial h_1} = 1$
$\frac{\partial \hat{y}}{\partial u} = 1$
$\frac{\partial s}{\partial w_1} = 3$
ReLU
s 6
h_1 6
$w_1 = 2$
$w_3 = 1$
$x = 3$
u 6
$\hat{y}$ 6
$\frac{\partial C}{\partial \hat{y}} = -8$
손실함수 MSE
$(\hat{y} - y)^2$
16
$y = 10$
합!
$w_2 = -2$
$w_4 = 0$
ReLU
t -6
h_2 0
$\frac{\partial C}{\partial w_1} = \frac{\partial C}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial u} \times \frac{\partial u}{\partial h_1} \times \frac{\partial h_1}{\partial s} \times \frac{\partial s}{\partial w_1} = -24$
학습률 η : 0.001
$w_1 := 2 - (0.001 \times -24)$

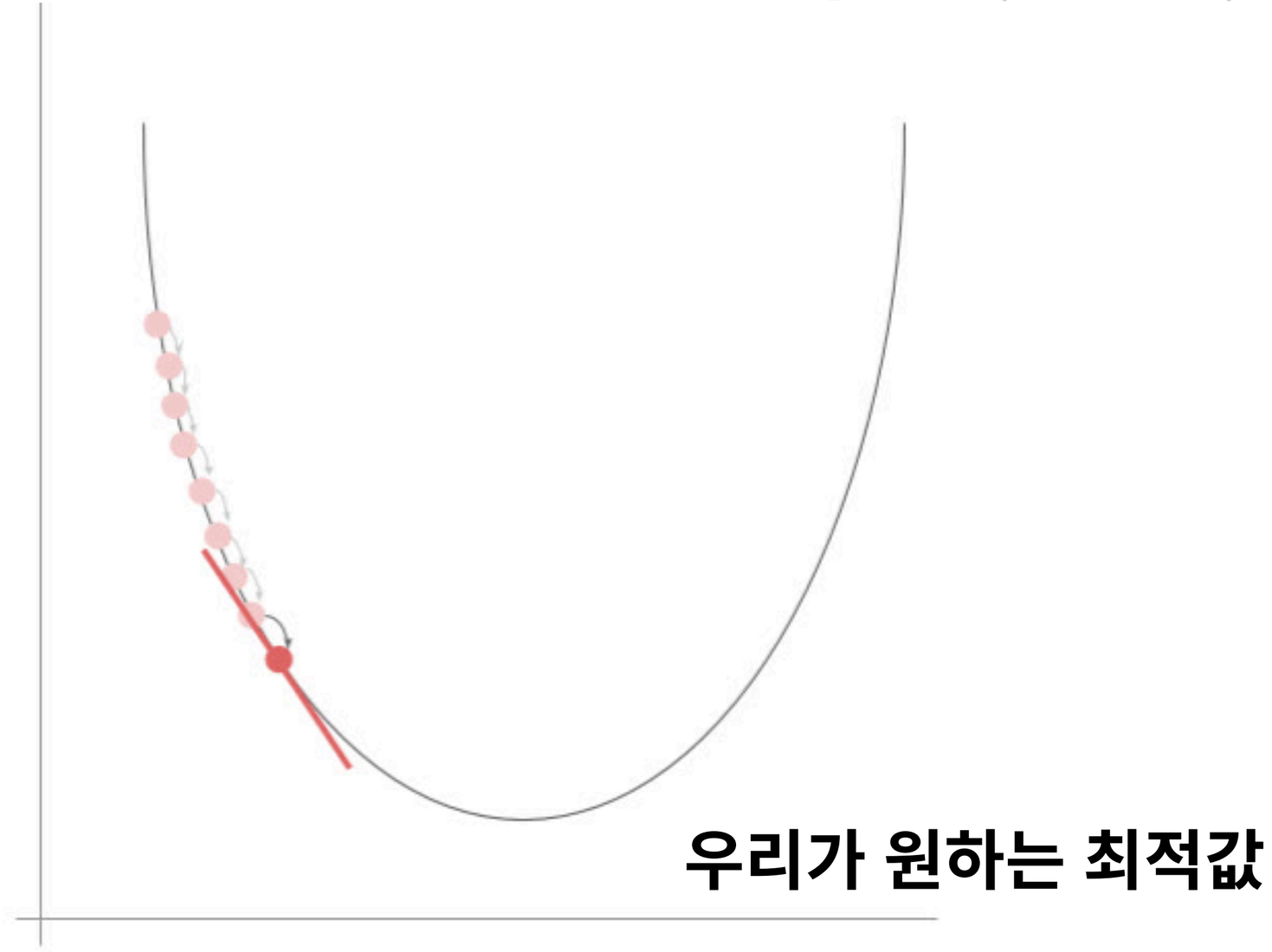


층을 쌓을수록, 복잡하게 표현할 수 있는 능력이 높아진다

$$\frac{\partial C}{\partial w_1} = \frac{\partial C}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial u} \times \frac{\partial u}{\partial h_1} \times \frac{\partial h_1}{\partial s} \times \frac{\partial s}{\partial w_1} = \boxed{-24}$$

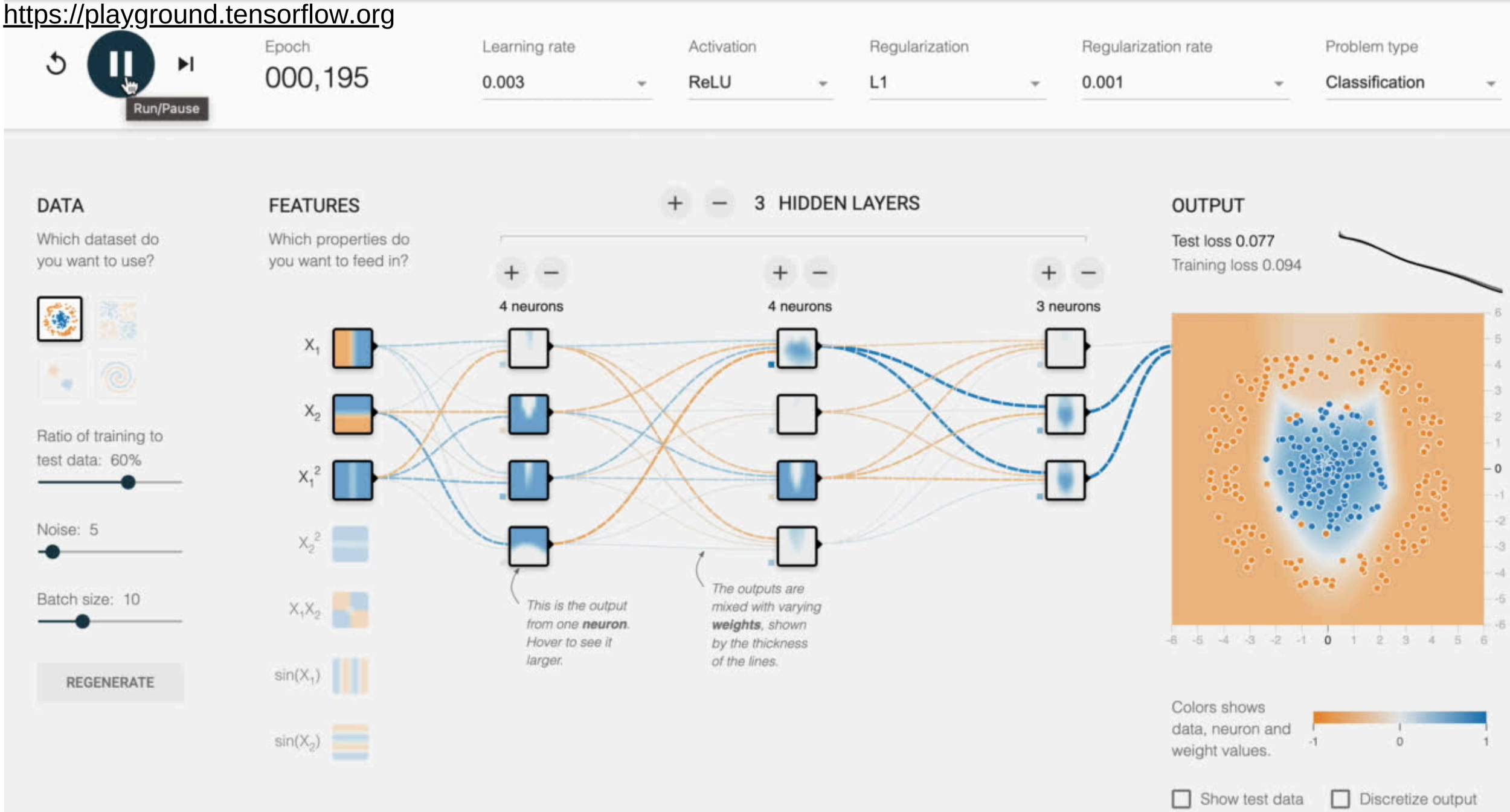
학습률 η : 0.001

$w_1 := 2 - (0.001 \times -24)$

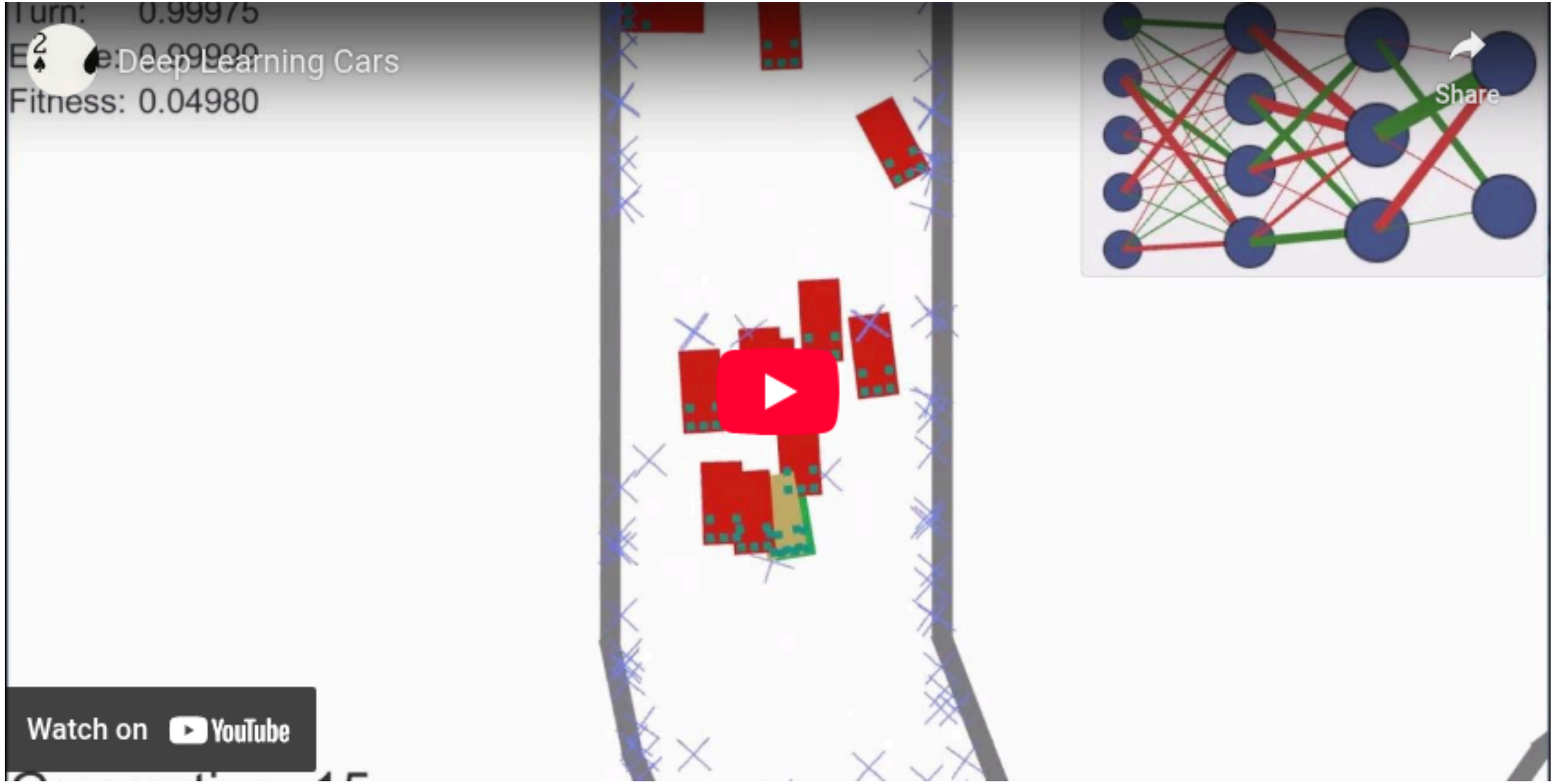


우리가 원하는 최적값

https://playground.tensorflow.org



Epoch
000,066
Learning rate
0.01
Activation
ReLU
Regularization
L1
Regularization rate
0.001
Problem type
Classification
DATA
Which dataset do you want to use?
Ratio of training to test data: 70%
Noise: 5
Batch size: 10
REGENERATE
FEATURES
Which properties do you want to feed in?
6 HIDDEN LAYERS
5 neurons
5 neurons
5 neurons
4 neurons
4 neurons
3 neurons
X1
X2
X1²
X2²
X1X2
sin(X1)
sin(X2)
This is the output from one neuron. Hover to see it larger.
The outputs are mixed with varying weights, shown by the thickness of the lines.
OUTPUT
Test loss 0.495
Training loss 0.490
Colors shows data, neuron and weight values.
-1
0
1
Show test data
Discretize output


Turn: 0.99975
Deep Learning Cars
Fitness: 0.04980
Share
Watch on YouTube

이걸 만들어야 합니다.

파이썬 + 코랩

colab

인터프리터식

실행하면 한줄씩 실행하는 방식임

동적 타이핑

선언하는 순간 변수의 타입이 결정됨

객체지향적

객체적 성질을 이용하여
기능을 자유롭게 차용

대화형 언어

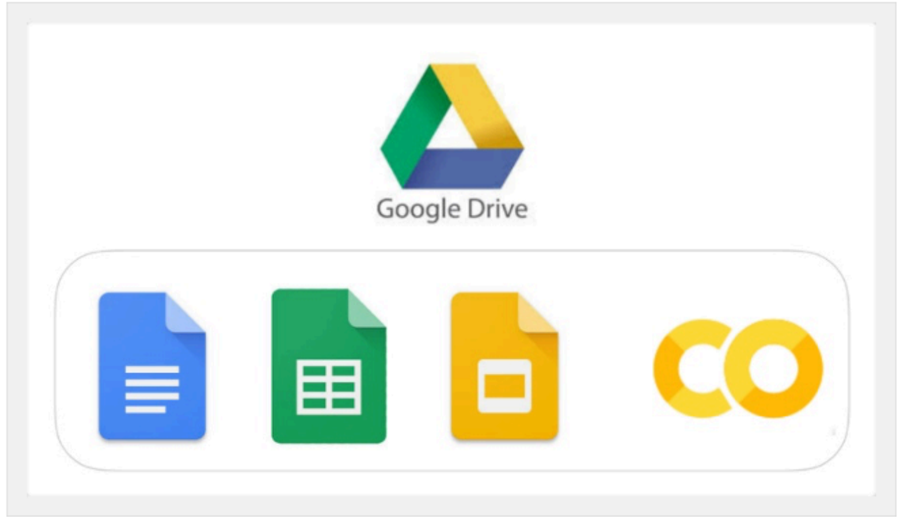
코랩 주피터 노트북 이용

무료 GPU 사용

클라우드 기반

jupyter notebook 호환

구글 드라이브와 호환



웹 브라우저에서 텍스트와 프로그램 코드를 자유롭게 작성할 수 있는 Google Colaboratory (일명 코랩)

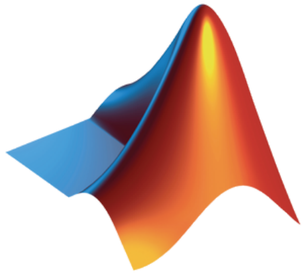
- 로컬 *PC* 가 아닌 클라우드 환경에서 작업이 된다

- 따라서 사용 중인 컴퓨터의 성능과 관계없이
  구글에서 제공하는 GPU, TPU 성능을 활용할 수 있다

“공짜로”

TORCH?

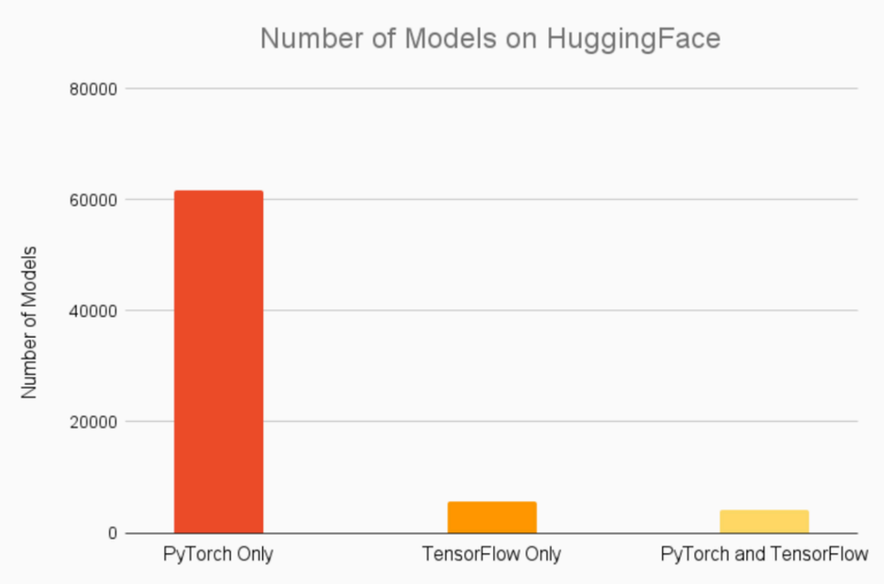
파이썬으로 딥러닝을 만드는 프레임워크

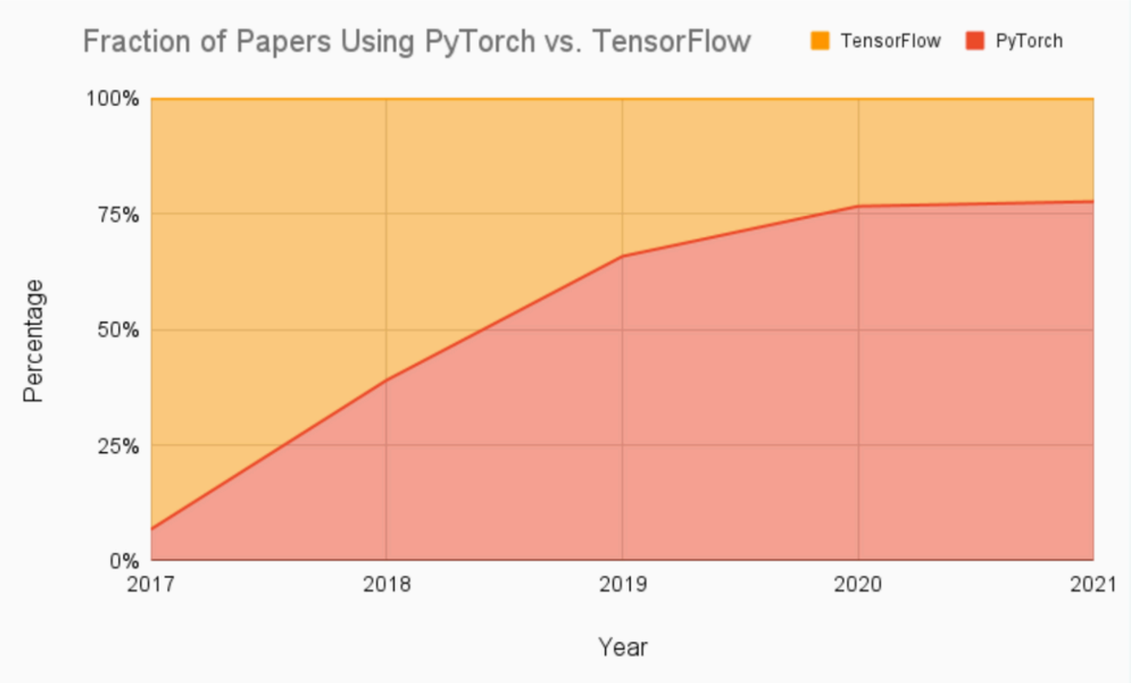


https://pytorch.org

https://tutorials.pytorch.kr

PyTorch





Dataset & Dataloader

데이터 읽기 및 전처리
배치 생성과 증강 등

torch.utils
torchvision
torchaudio

Neural network

뉴럴네트워크 구성
모듈형, 함수형 모두 지원

torch.nn
torch.nn.functional

Loss & Optimizer

오차계산과 모델 업데이트 지원

torch.autograd
torch.optim

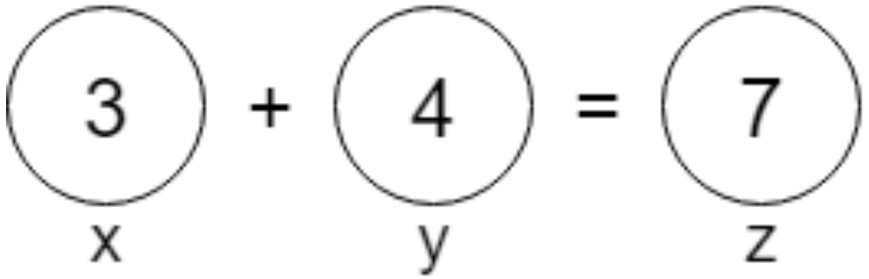
추가 기능 지원

병렬처리, 직렬화 등

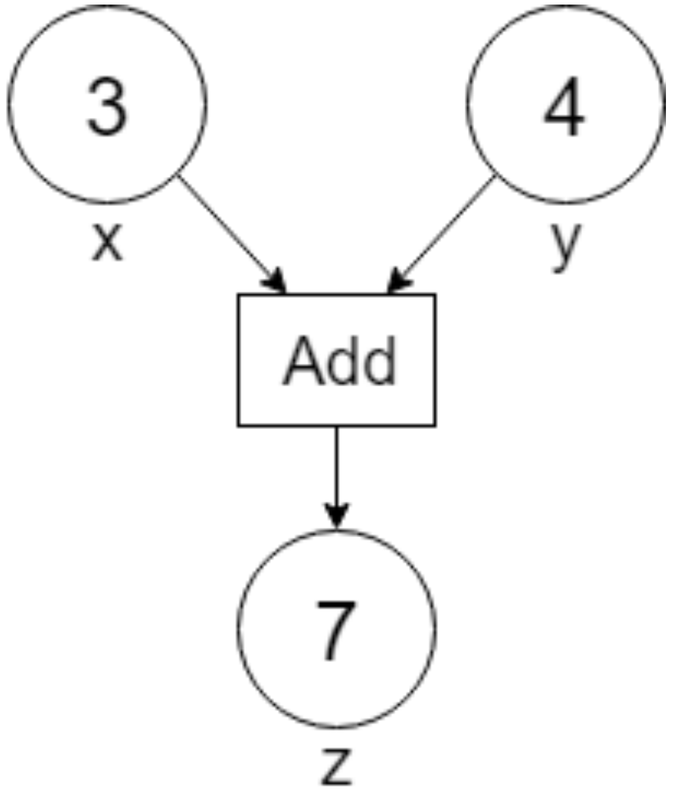
torch.multiprocessing
torch.onnx

PyTorch?

DAG: Directed Acyclic Graph



VS

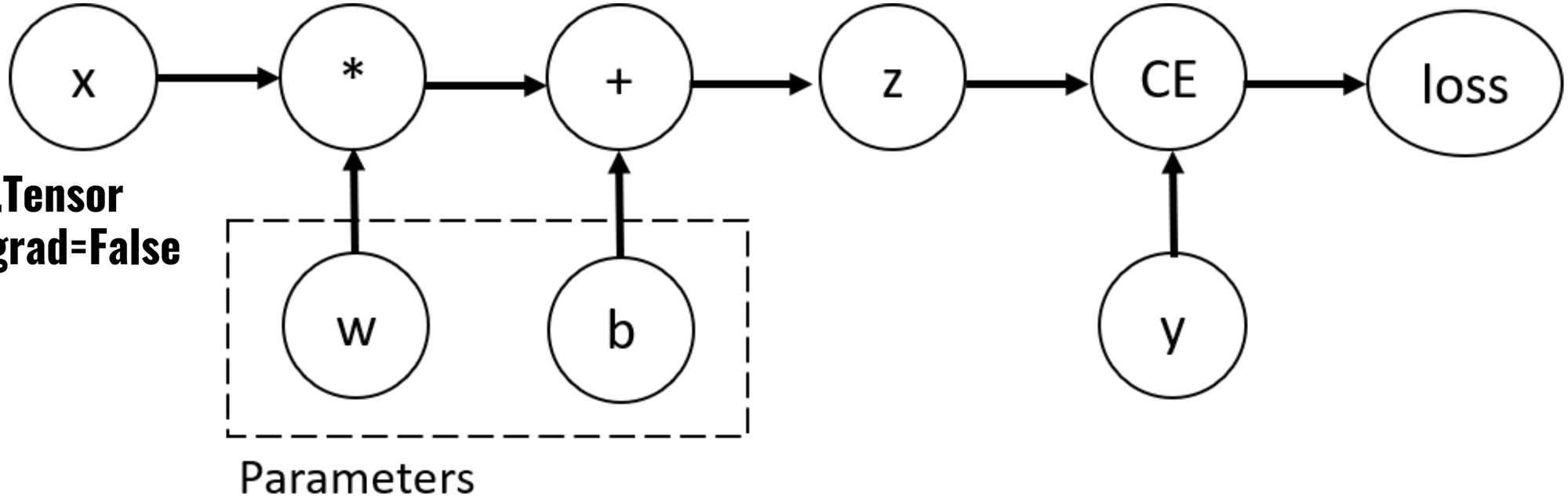


DEFINED BY RUN: DYNAMIC COMPUTATION GRAPH

PyTorch?

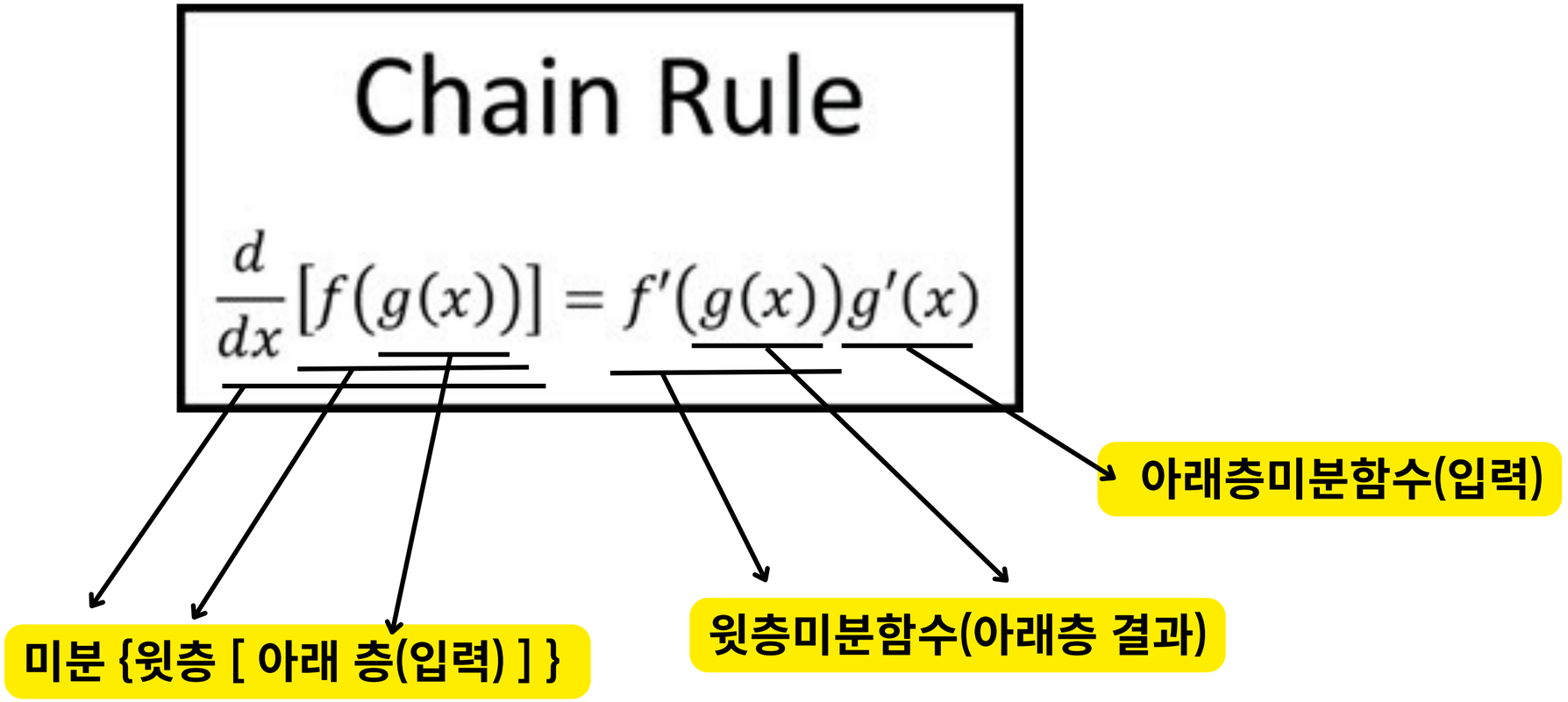
Dynamic computation graph
retain_graph = False

DAG



torch.Tensor
require_grad=False

torch.Tensor[torch.nn]
require_grad=True



if x = k(z)? if z = q(j) ?

계속해서 연장가능!!!

$$\vec{y} = f(\vec{x})$$

$\vec{x}$ 에 대한 $\vec{y}$ 의 변화도는 야코비안 행렬(Jacobian Matrix) $\boldsymbol{J}$:

$$\boldsymbol{J} = \begin{pmatrix} \frac{\partial \mathbf{y}}{\partial x_1} & \cdots & \frac{\partial \mathbf{y}}{\partial x_n} \end{pmatrix} = \begin{pmatrix} \frac{\partial y_1}{\partial x_1} & \cdots & \frac{\partial y_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_m}{\partial x_1} & \cdots & \frac{\partial y_m}{\partial x_n} \end{pmatrix}$$

$$l = g\left(\vec{y}\right) \quad \vec{y} = f(\vec{x})$$

만약 $\vec{v}$ 가 스칼라 함수 $l = g\left(\vec{y}\right)$ 의 변화도(gradient)인 경우:

$$\vec{v} = \begin{pmatrix} \frac{\partial l}{\partial y_1} & \cdots & \frac{\partial l}{\partial y_m} \end{pmatrix}^T$$

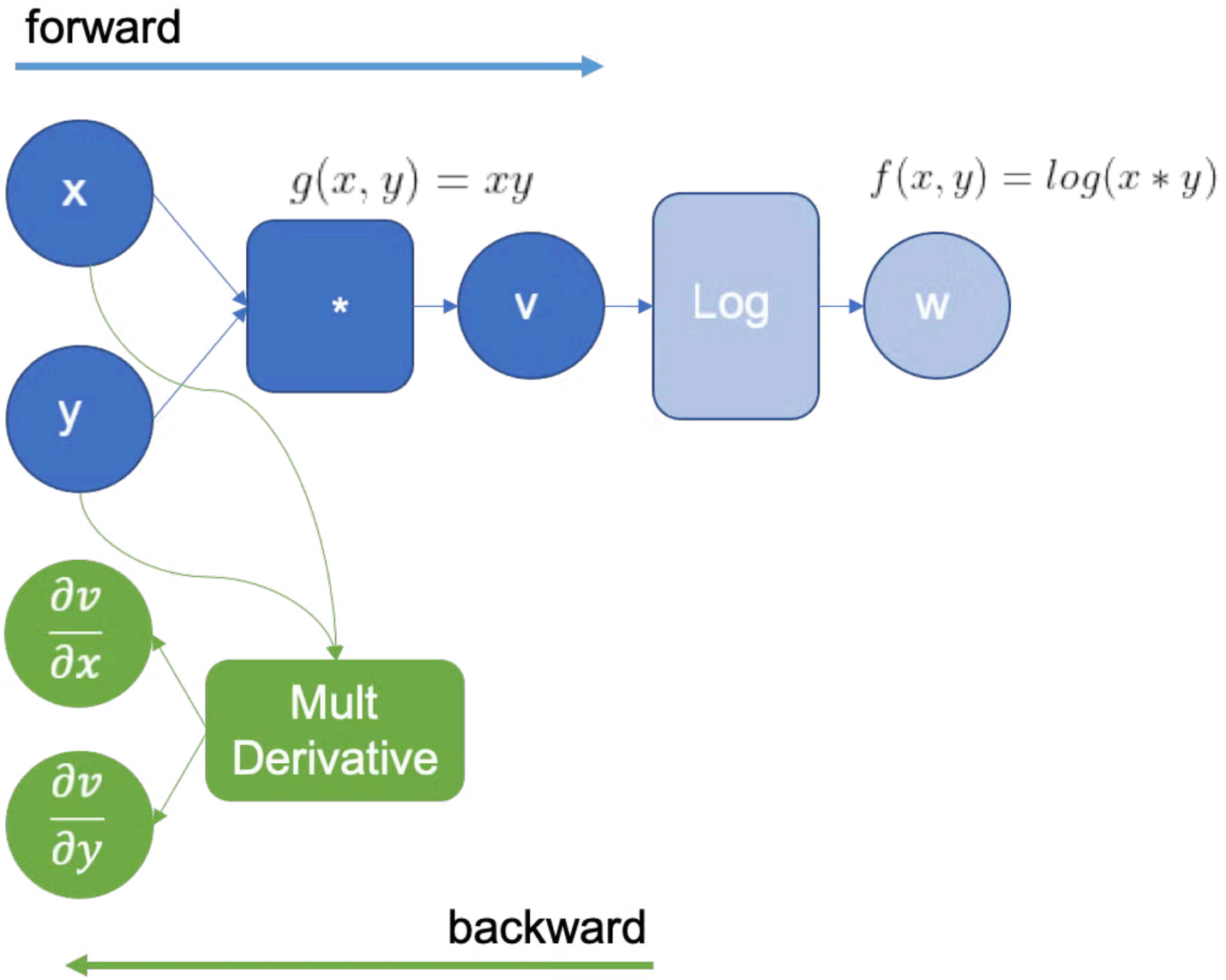
연쇄 법칙(chain rule)에 따라, 벡터-야코비안 곱은 $\vec{x}$ 에 대한 l 의 변화도(gradient)

$$J^T \cdot \vec{v} = \begin{pmatrix} \frac{\partial y_1}{\partial x_1} & \cdots & \frac{\partial y_m}{\partial x_1} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_1}{\partial x_n} & \cdots & \frac{\partial y_m}{\partial x_n} \end{pmatrix} \begin{pmatrix} \frac{\partial l}{\partial y_1} \\ \vdots \\ \frac{\partial l}{\partial y_m} \end{pmatrix} = \begin{pmatrix} \frac{\partial l}{\partial x_1} \\ \vdots \\ \frac{\partial l}{\partial x_n} \end{pmatrix}$$

autograd

Chain Rule

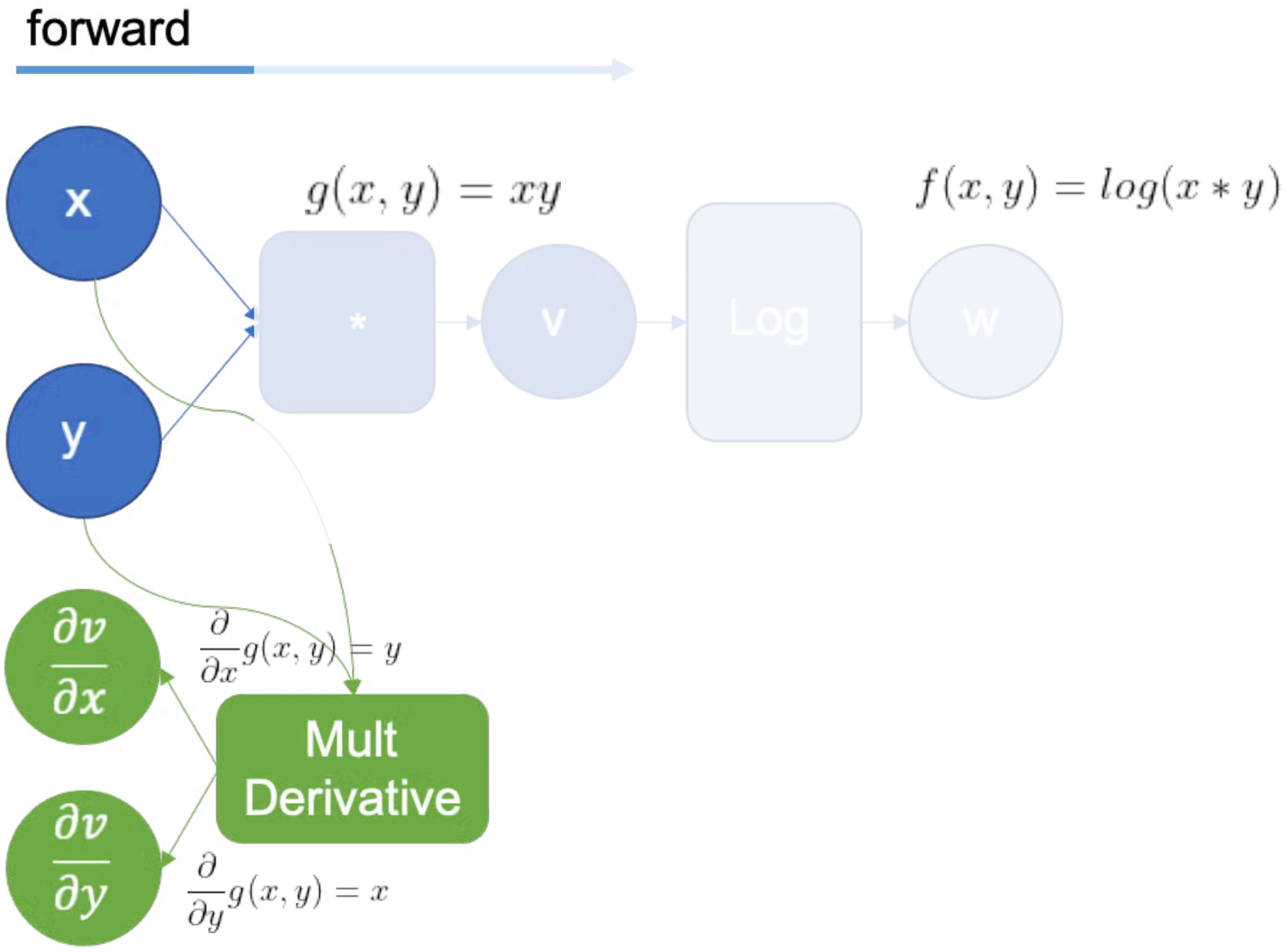
$$\frac{d}{dx}[f(g(x))] = f'(g(x))g'(x)$$



autograd

Chain Rule

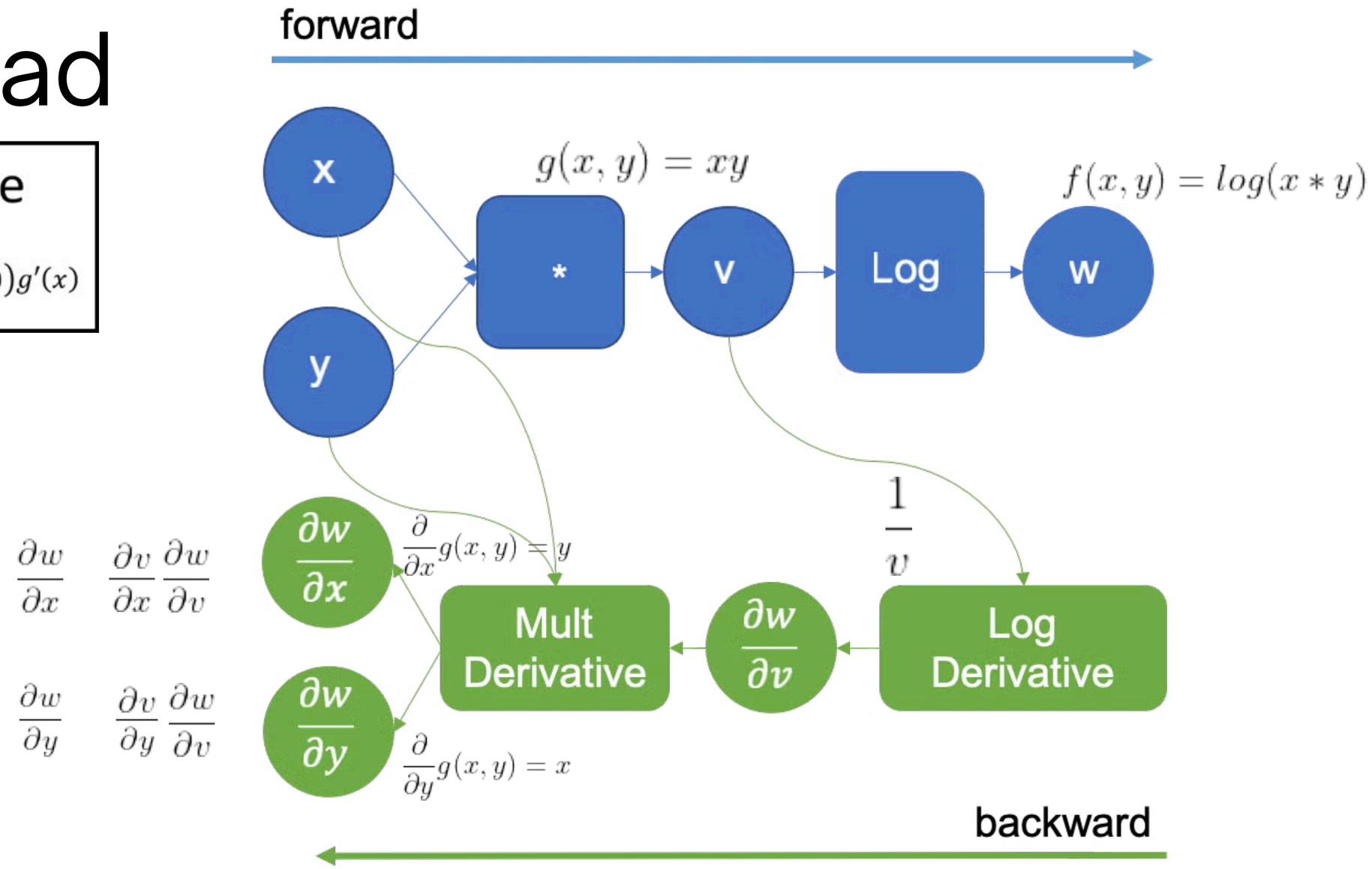
$$\frac{d}{dx}[f(g(x))] = f'(g(x))g'(x)$$



autograd

Chain Rule

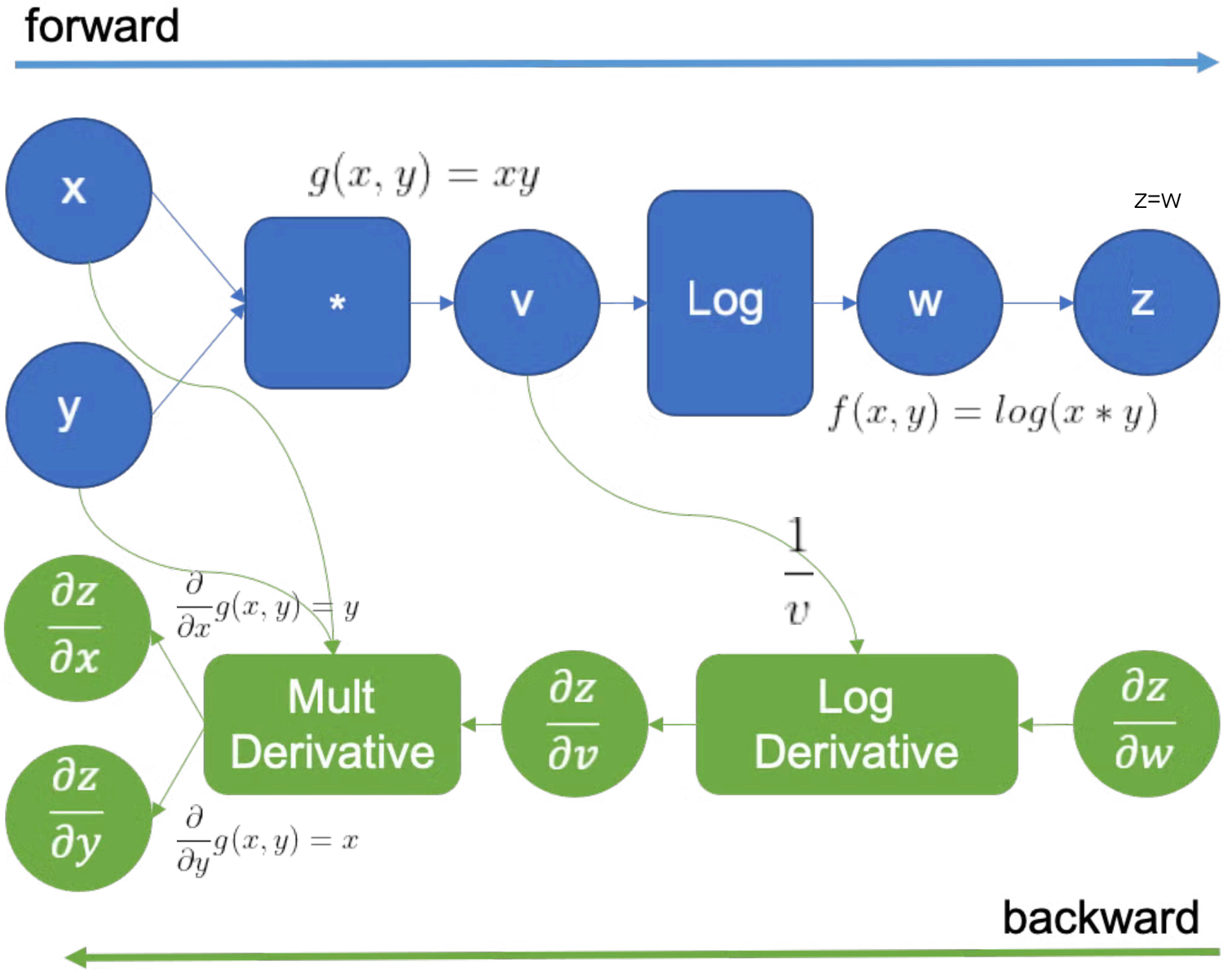
$$\frac{d}{dx}[f(g(x))] = f'(g(x))g'(x)$$



autograd

Chain Rule

$$\frac{d}{dx}[f(g(x))] = f'(g(x))g'(x)$$



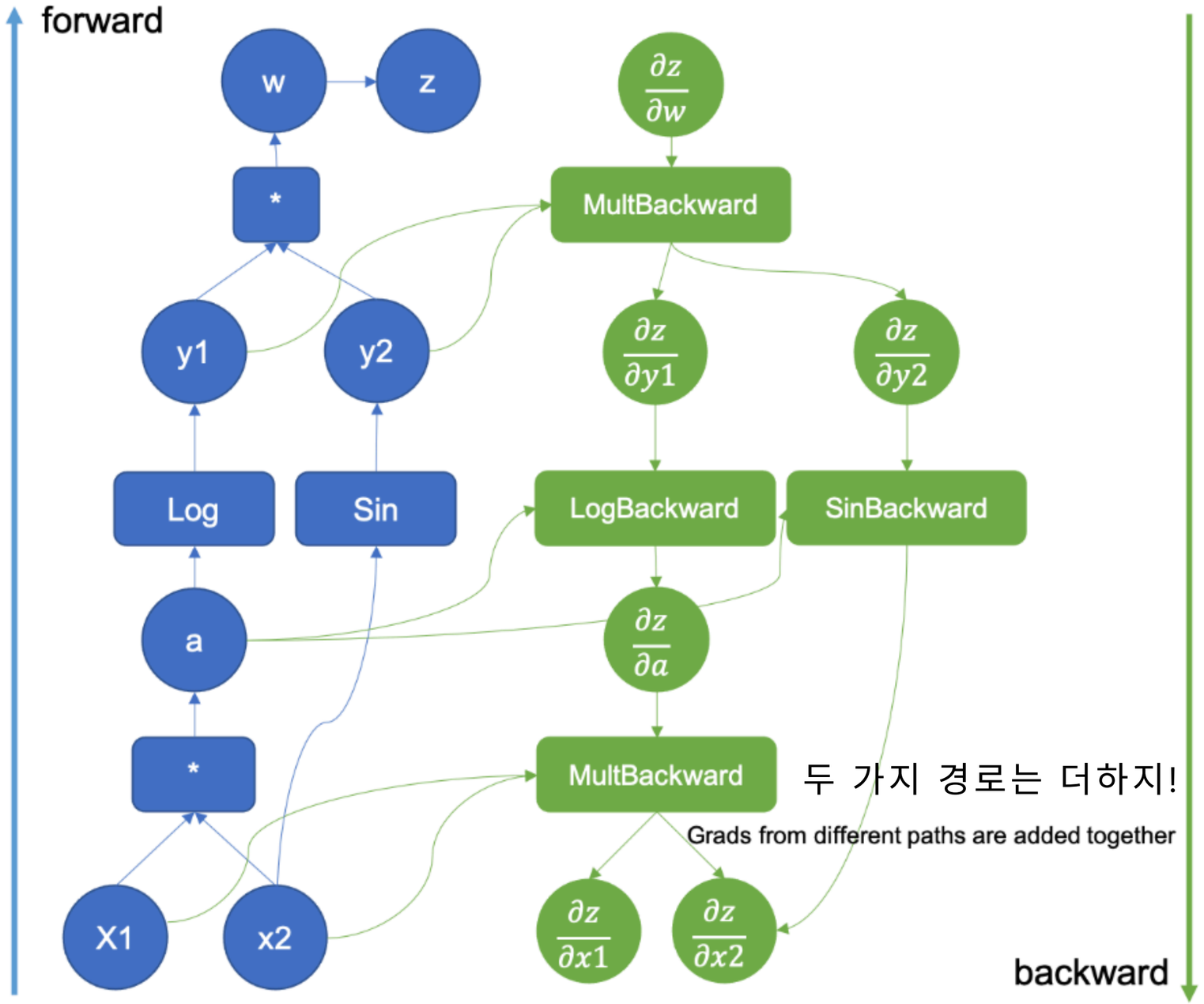
autograd

만약 연산이 확장된다고 해도

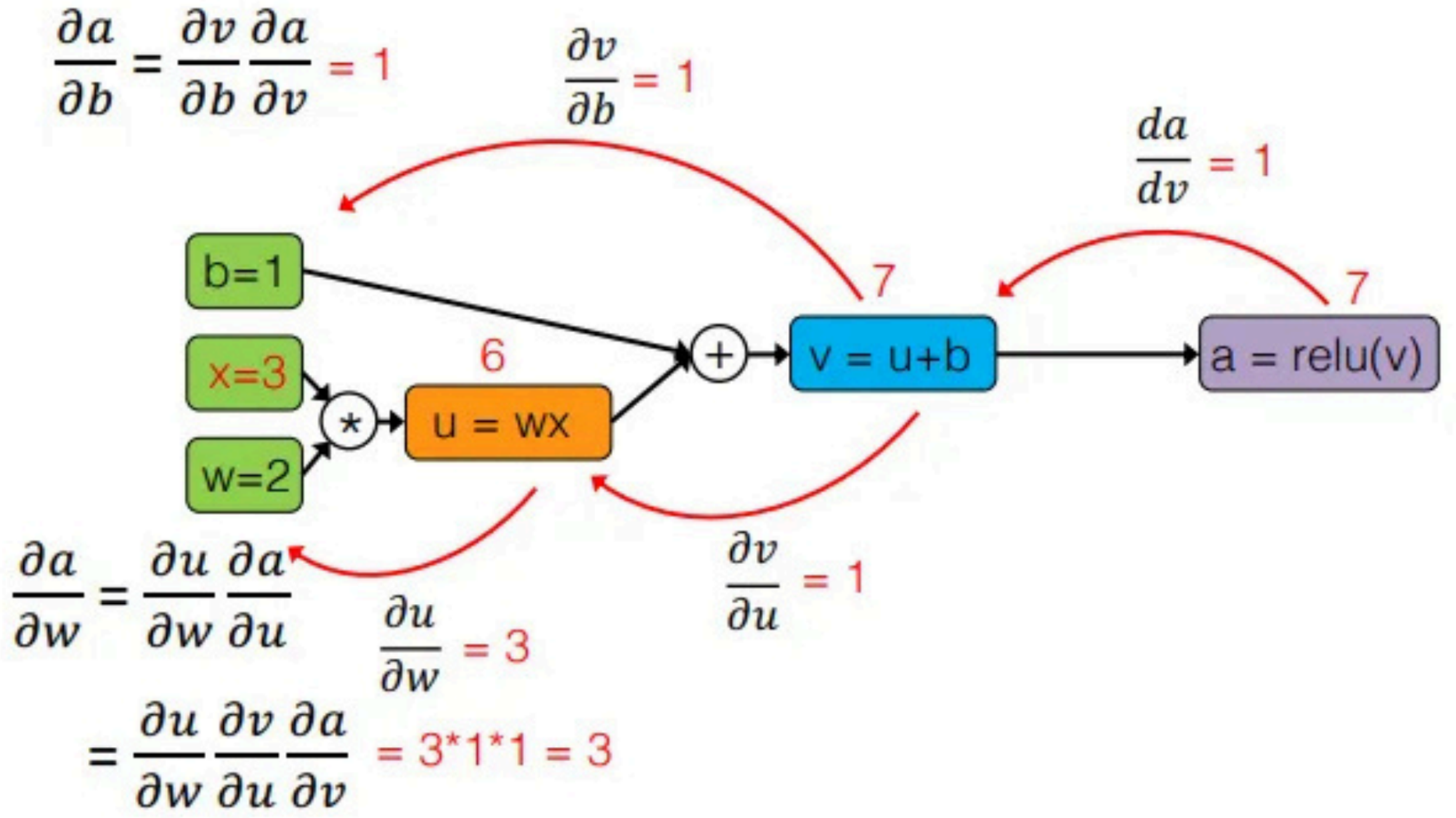
연산이 더 길어진다고 해도

그래프를 따라 하나씩

계산할 수 있을거에요!

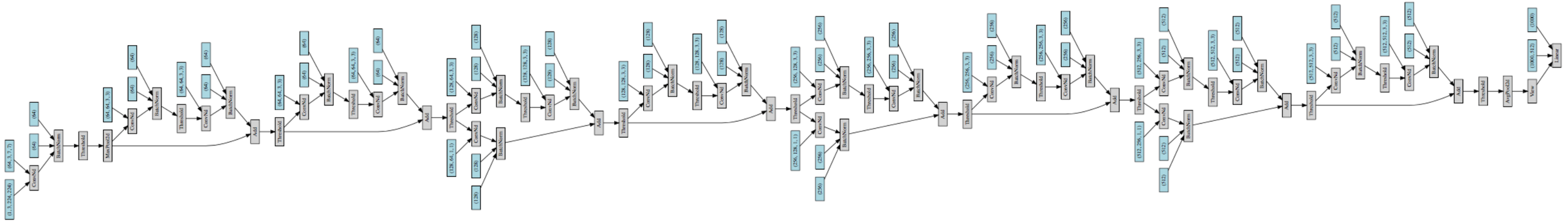


이 그림이 이해가시나요?



COMPUTATION GRAPH, SOURCE: HTTPS://SEBASTIANRASCHKA.COM/PDF/LECTURE-NOTES/STAT453SS21/L06_PYTORCH_SLIDES.PDF

더 길어도 상관없나요?



손실함수와 성능지표

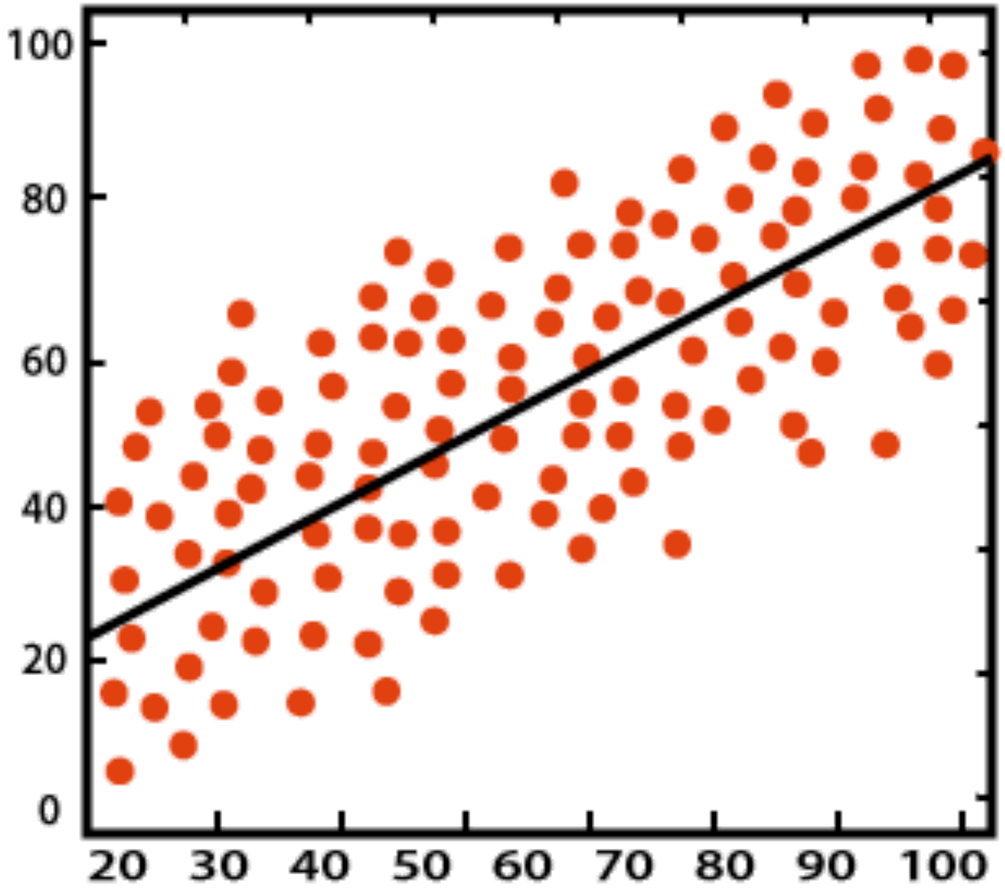
Regression

회귀란?

- 값을 예측하는 것
- 연봉은?
- 기대수명은?
- 당신이 신상을 구매할 확률은?
- *KDA?*

- 정성 데이터*(Qualitative)*

숫자로 측정가능한 값



Regression

```python
# 선형 회귀 모델 정의
class LinearRegressionModel(nn.Module):
    def __init__(self):
        super(LinearRegressionModel, self).__init__()
        self.linear = nn.Linear(1, 1)  # 입력 차원 1, 출력 차원 1

    def forward(self, x):
        return self.linear(x)

# 모델, 손실 함수 및 옵티마이저 초기화
model = LinearRegressionModel()
criterion = nn.MSELoss()  # 평균 제곱 오차 손실
optimizer = optim.SGD(model.parameters(), lr=0.01)  # 확률적 경사 하강법
```

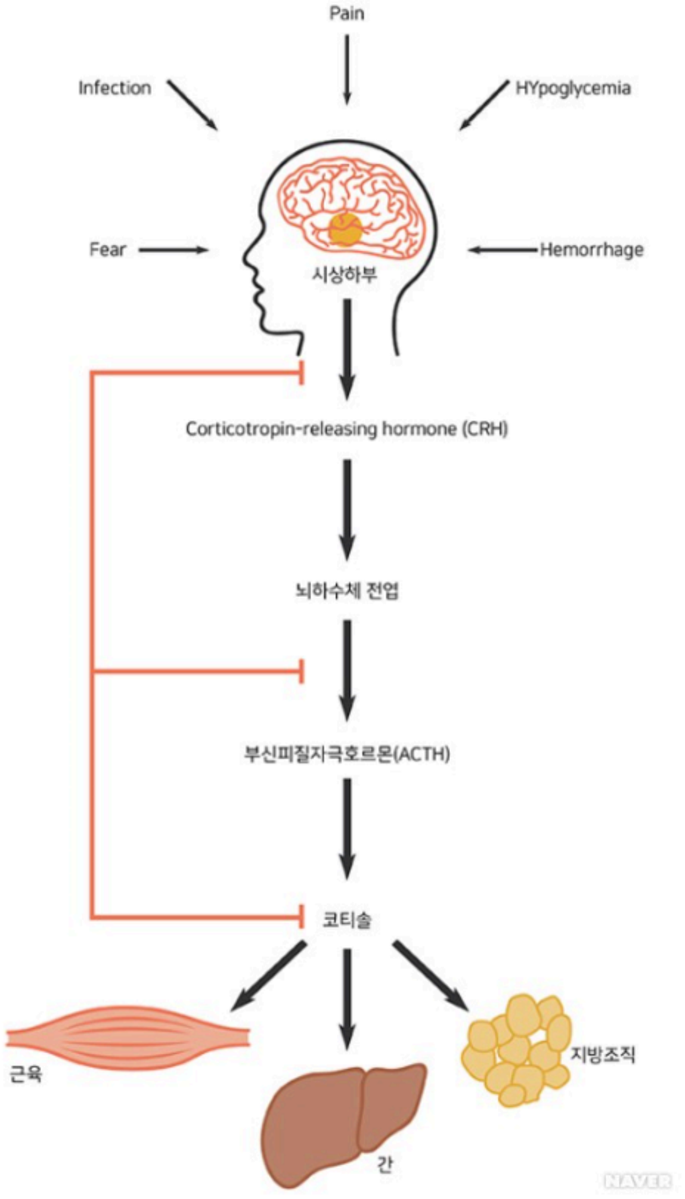
```python
# 임의의 데이터 생성
x_train = torch.randn(100, 1)  # 100개의 훈련 데이터
y_train = x_train * 2 + 3 + torch.randn(100, 1) * 0.1  # y = 2x + 3에 노이즈 추가

# 훈련 과정
num_epochs = 100
for epoch in range(num_epochs):
    # 순전파: 입력을 모델에 전달하여 예측값 계산
    predictions = model(x_train)

    # 손실 계산
    loss = criterion(predictions, y_train)

    # 역전파
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    if (epoch + 1) % 10 == 0:
        print(f'Epoch [{epoch+1}/{num_epochs}], Loss: {loss.item():.4f}')

print("Finished Training")
```

주식 종가 예측?

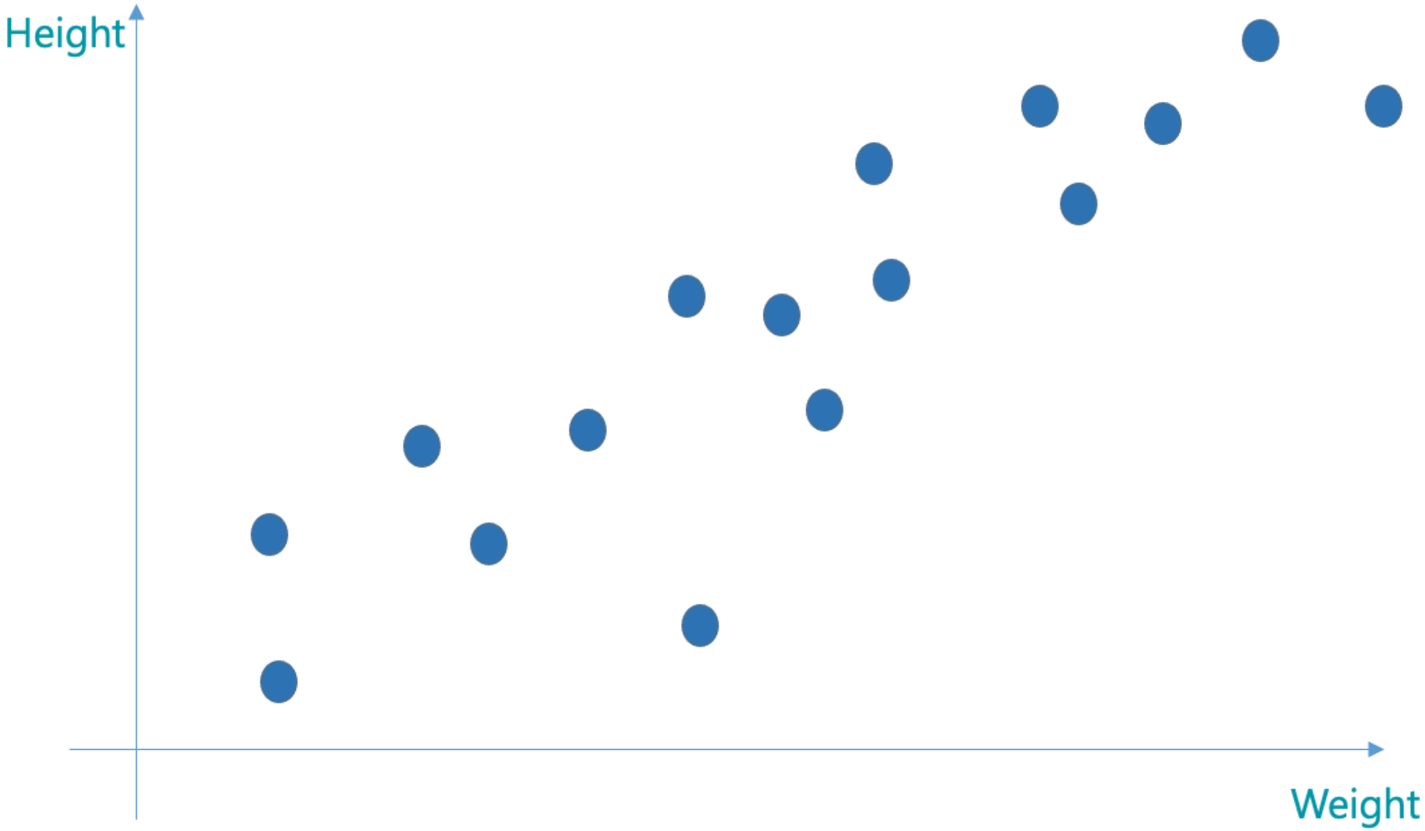
Credit: https://www.tradingsetupsreview.com/quick-trade-using-linear-regression-channel/

코티졸과 스트레스

https://www.nhimc.or.kr/ilsan_news/Hello_2019_Autumn/html/sub01_04.html



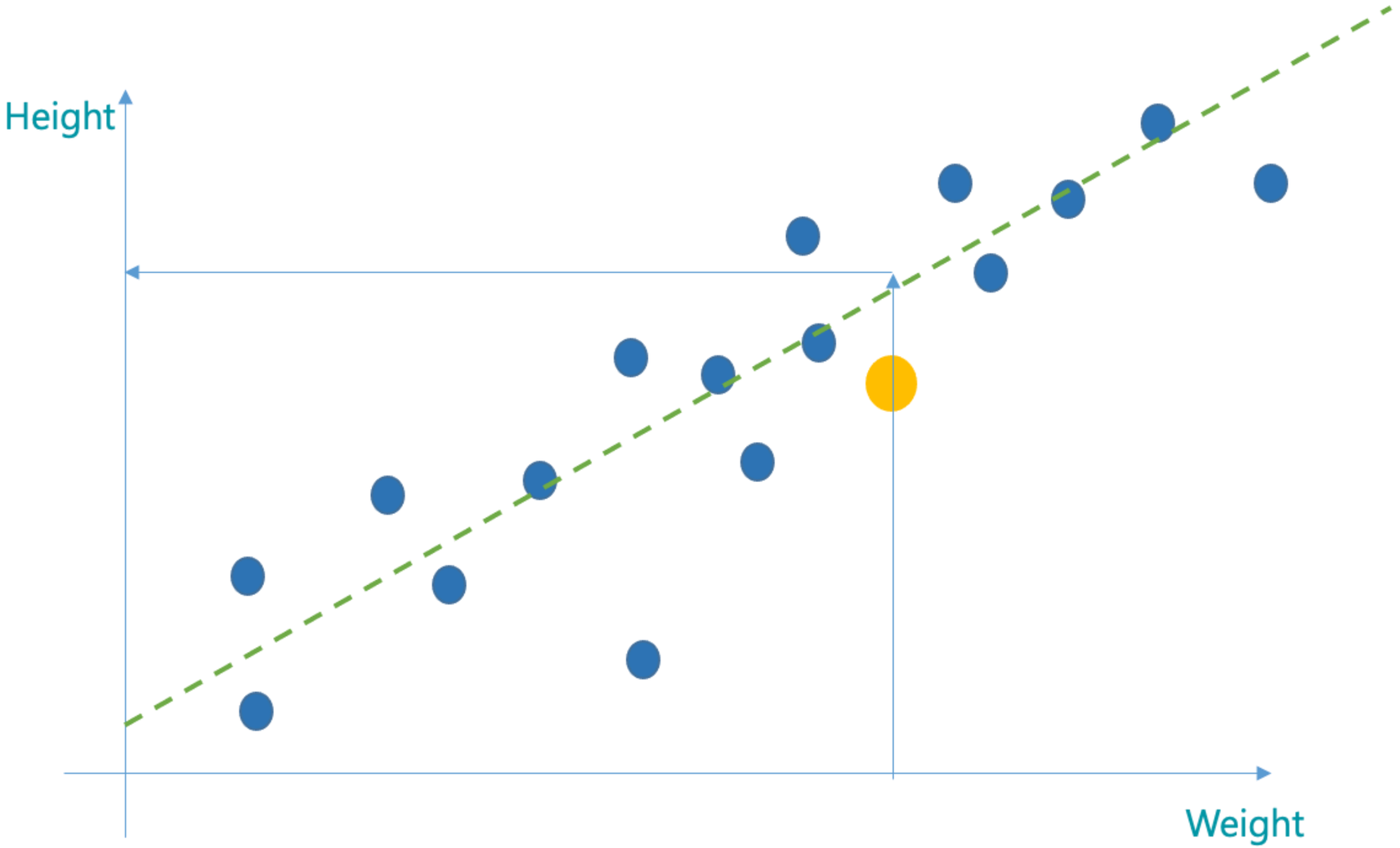
https://m.blog.naver.com/sohoon1002/222033614363



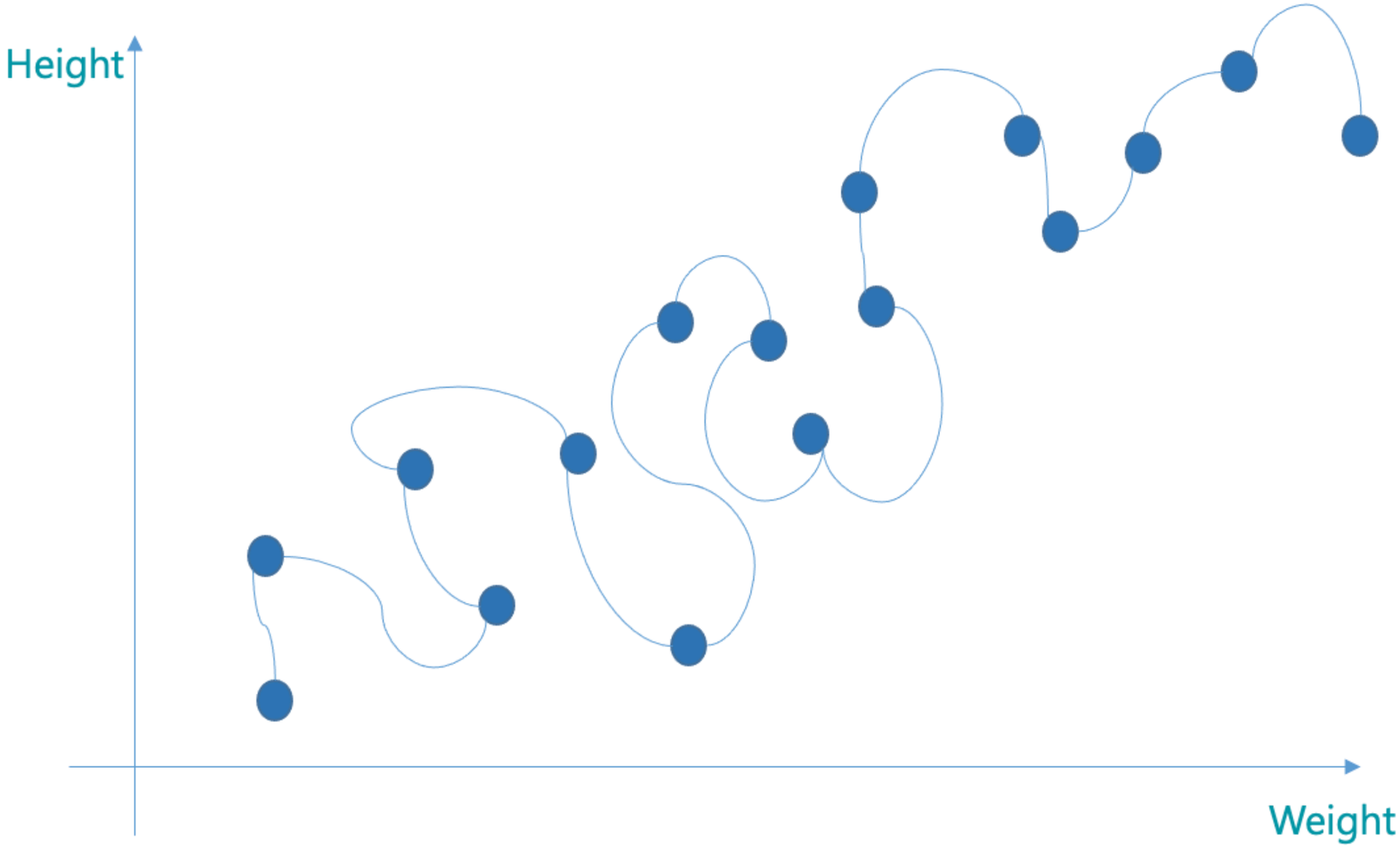
회귀 문제:

데이터가 이렇게 주어져 있을 때
새로운 데이터의 결과값을 어떻게 예측할까?

ml 쪽에서 지표 내용 가져오자

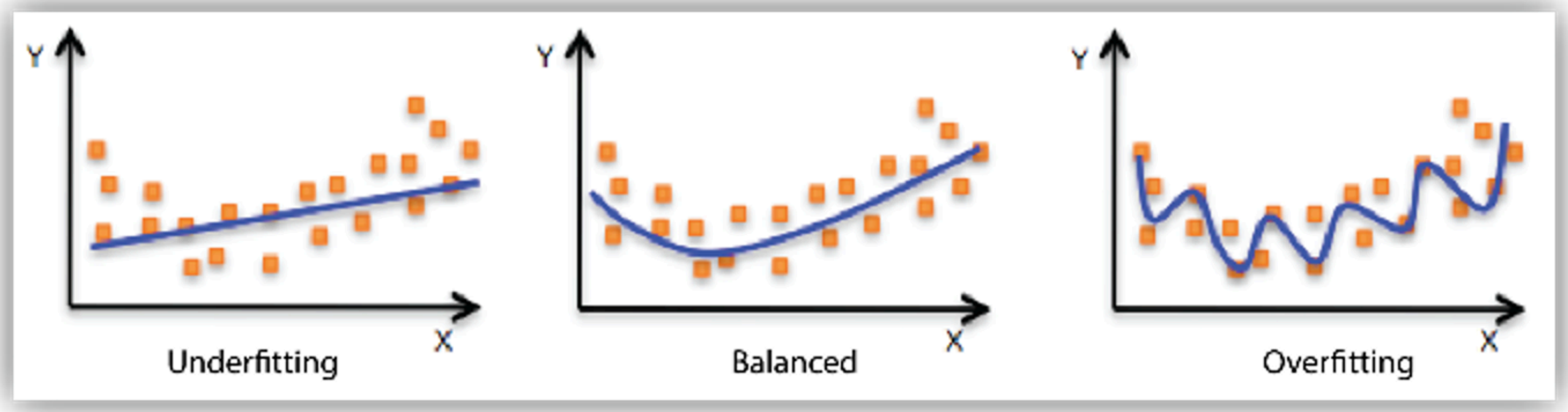


새로운 몸무게 데이터가 들어왔을 때,
키를 예측할 수 있다



이런 선을 찾으면 어떻게 될까?

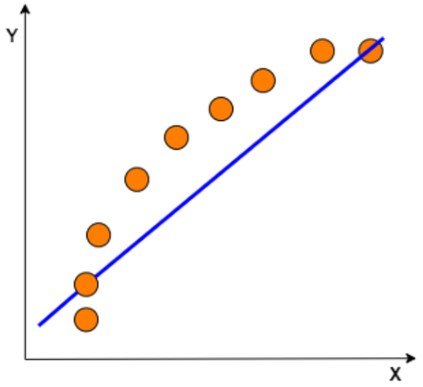
-> **overfitting(오버피팅)**



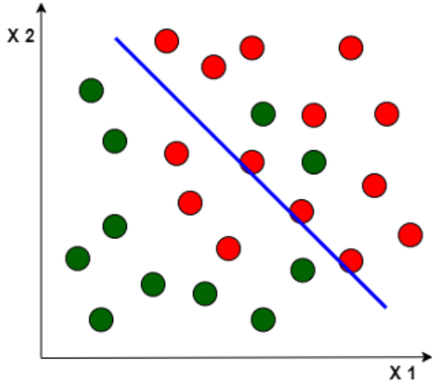
최소한의 예제도 모르면..
시험을 잘칠리가 없겠죠?

문제집을 달달 암기한다고
무조건 시험을 잘치진 않죠?

Generalization

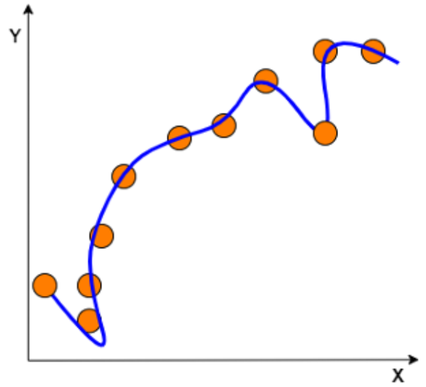


Linear Regression

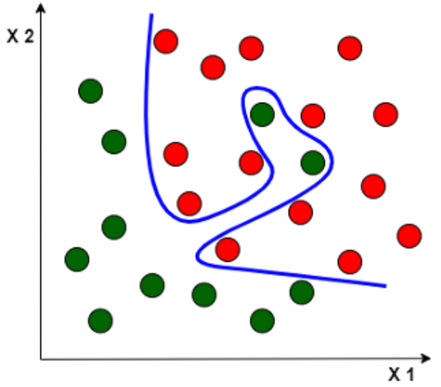


Logistic Regression

Underfitting(언더피팅)
: 학습 데이터의 특성을 제대로 잡지 못하고
피팅이 잘 되지 않은 상태



Linear Regression



Logistic Regression

Overfitting(오버피팅)
: 학습 데이터에 너무 많이 피팅되어
일반화할 수 없어 새로운 데이터에 대한 예측력이 낮아지는 상태

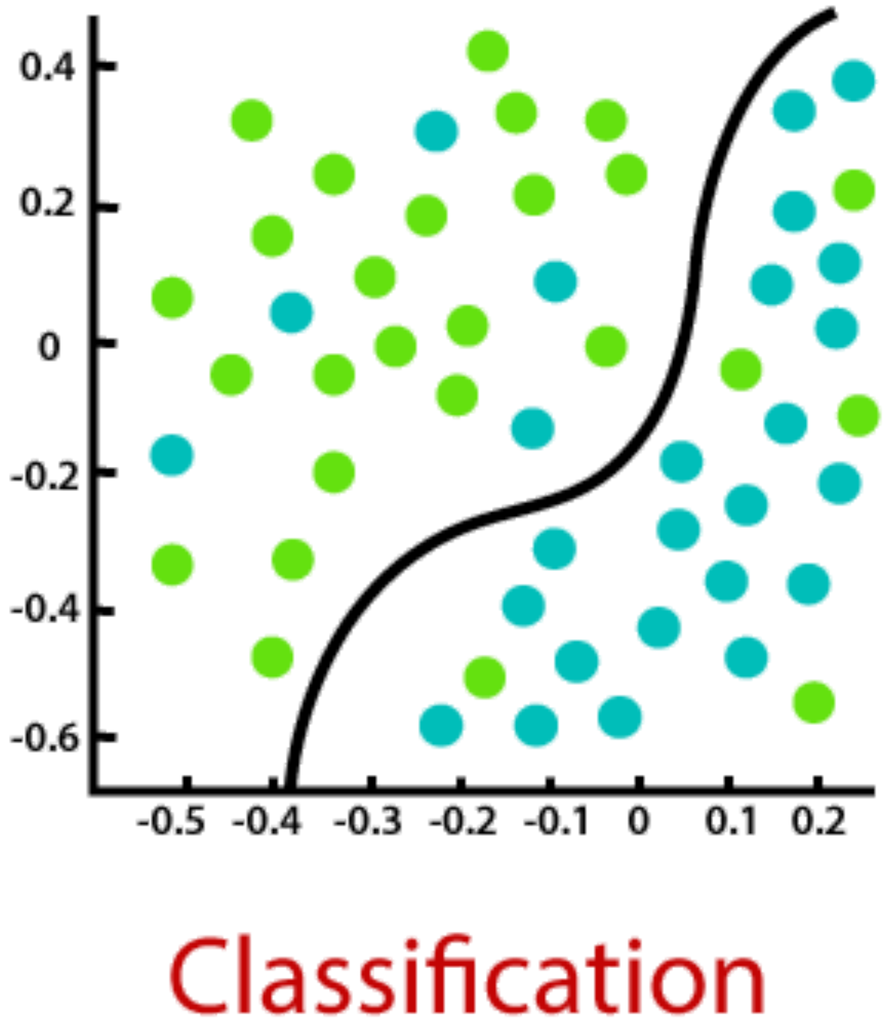
https://satishgunjal.com/underfitting_overfitting/

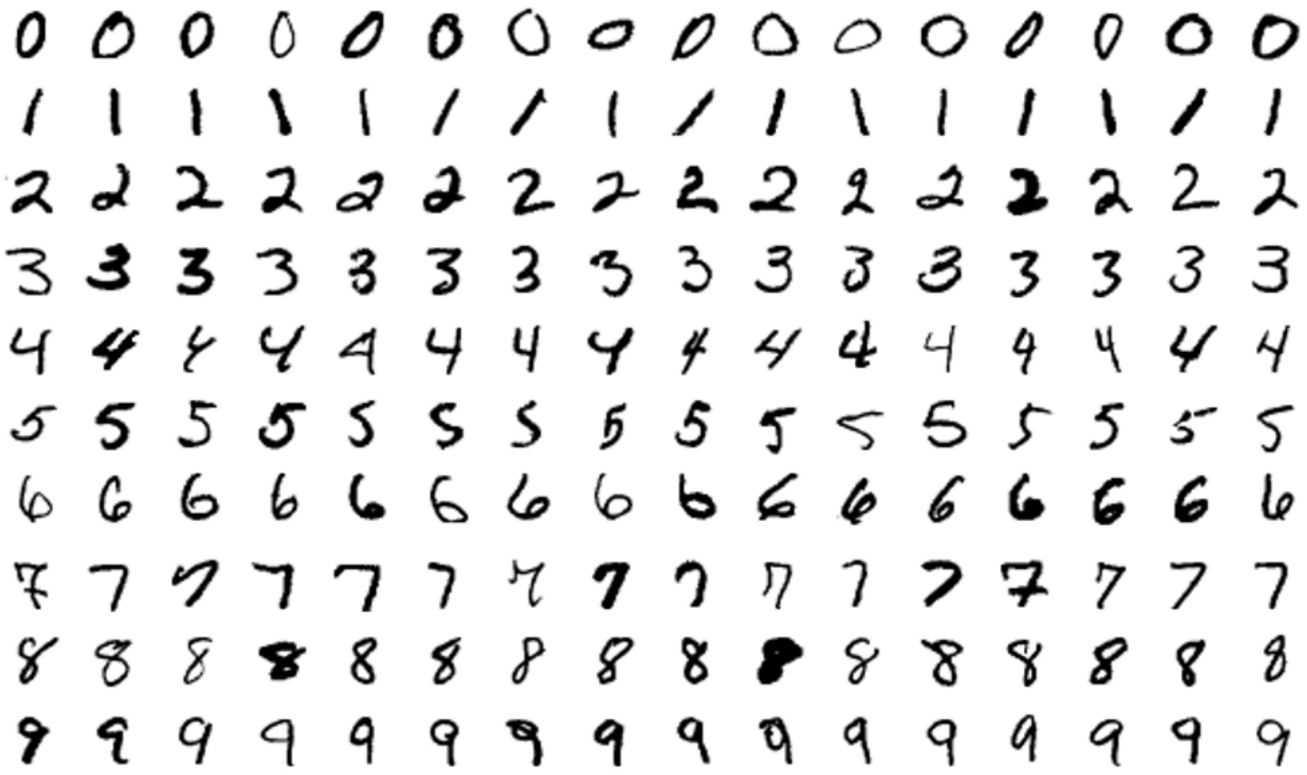
Classification

분류란?

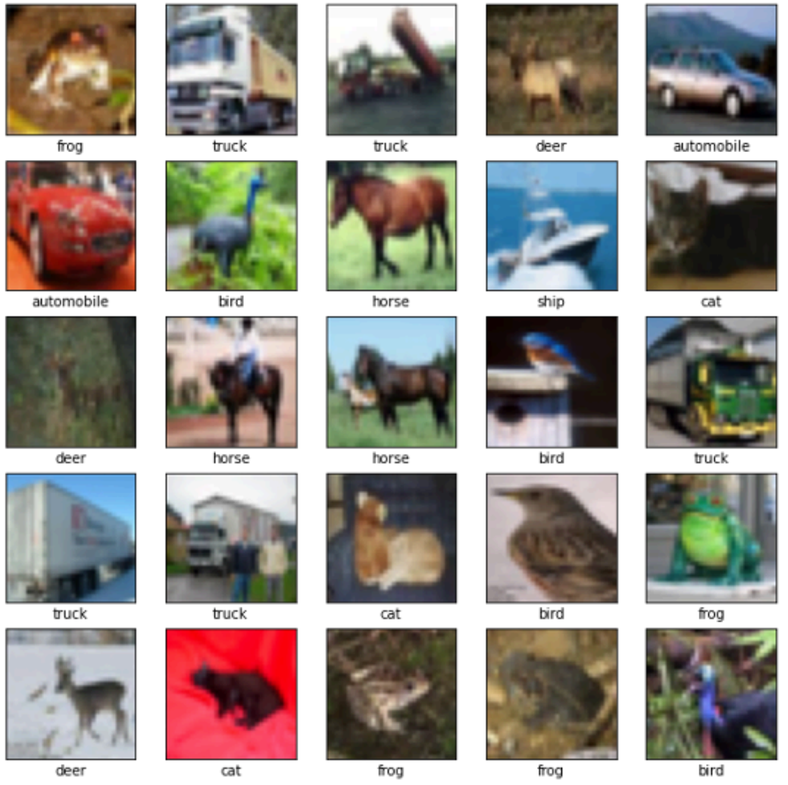
- 분류란 나누는 것
- 개? 고양이?
- 남자? 여자?
- 종양이 악성인지? 아닌지?
- 이메일이 스팸인지? 아닌지?

- 범주*(category)*





MNIST 손글씨 분류



CIFAR 사진 대상 분류

텍스트나 표정의 감정 분석

https://monkeylearn.com/sentiment-analysis/

네이버 영화 리뷰 감정분석

	id	document	label
0	9976970	아 더빙 진짜 짜증나네요 목소리	0
1	3819312	흠포스터보고 초딩영화줄오버연기조차 가볍지 않구나	1
2	10265843	너무재밌었다그래서보는것을추천한다	0
3	9045019	교도소 이야기구먼 솔직히 재미는 없다평점 조정	0
4	6483659	사이몬페그의 익살스런 연기가 돋보였던 영화스파이더맨에서 늙어보이기만 했던 커스틴 던...	1

LOSS

얼마나 다른지를 숫자로 표현하는 방법

Regression Loss Functions

Mean Absolute Error (L1 Loss)

$$MAE = \frac{\sum_{i=1}^{n} | y_i - \hat{y}_i |}{n}$$

Mean Absolute Error (MAE) Loss Function

OUTLIER에 강인함, UNSTABLE..

Mean-Squared Error (L2 Loss)

$$MSE = \frac{\sum_{i=1}^{n} (y_i - \hat{y}_i)^2}{n}$$

Mean Squared Error (MSE) Loss Function

OUTLIER에 취약, STABLE..

nn.L1loss(MAE), nn.MSEloss, nn.HuberLoss, nn.SmoothL1Loss

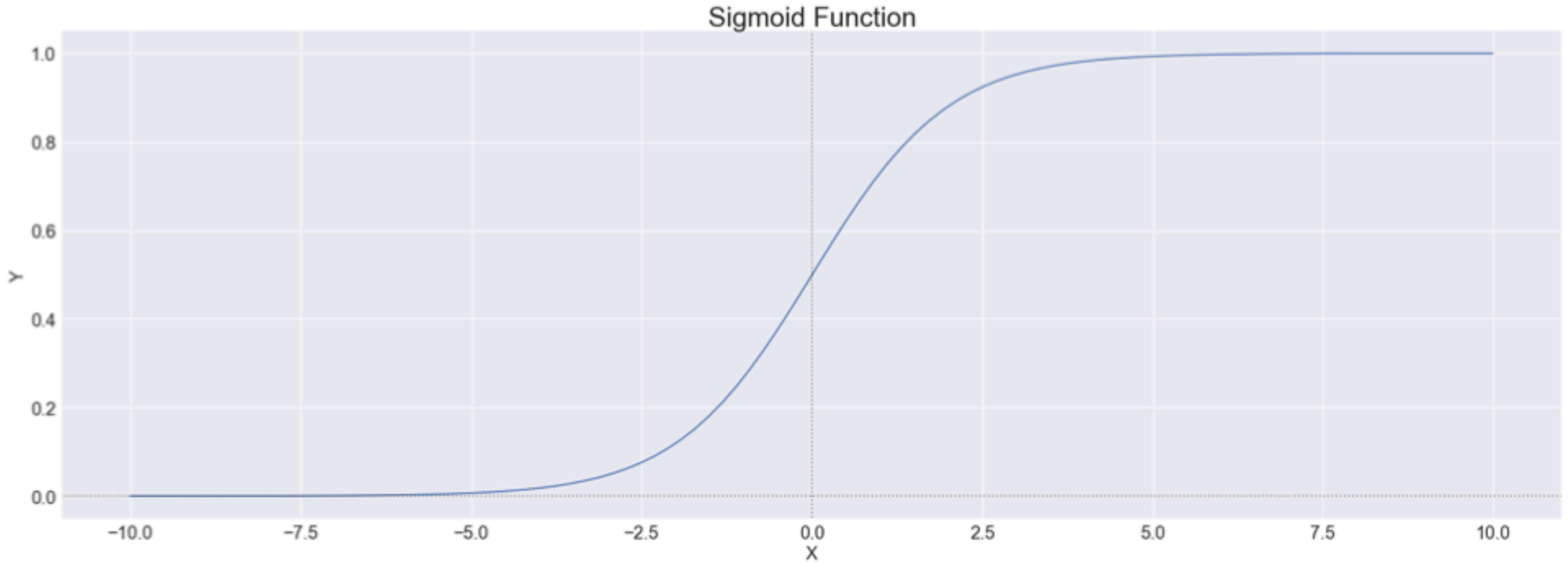
학습 = “LOSS”가 줄어들 수 있도록 주어진 PARAMETER를 바꾸는 것

LOSS = 오차

우리가 원하는 결과가 나오면 오차는 적게

우리가 원하지 않는 결과가 나오면 오차는 많게!

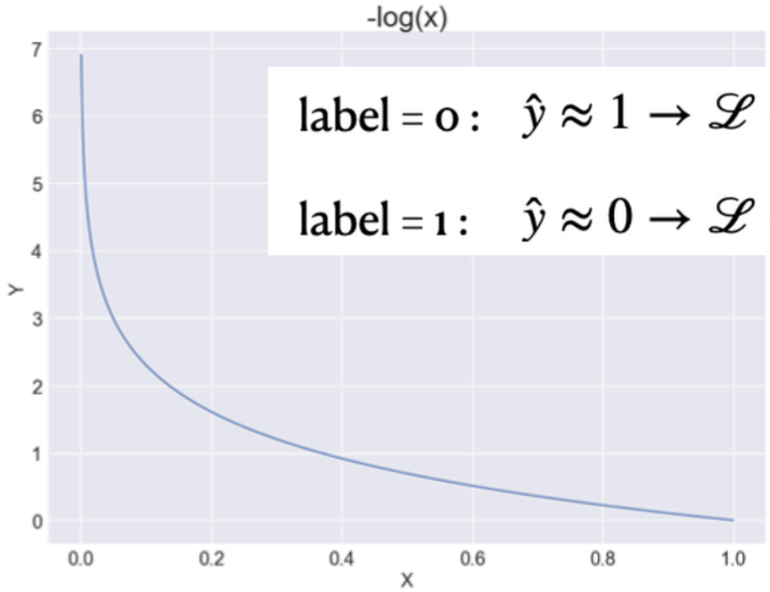
BINARY CLASSIFICATION



$$\text{label} = 0: \quad \hat{y} \approx 1 \rightarrow \mathscr{L} \Uparrow \qquad \hat{y} \approx 0 \rightarrow \mathscr{L} \Downarrow$$

$$\text{label} = 1: \quad \hat{y} \approx 0 \rightarrow \mathscr{L} \Downarrow \qquad \hat{y} \approx 1 \rightarrow \mathscr{L} \Uparrow$$

BCELOSS: BINARY CROSS ENTROPY LOSS

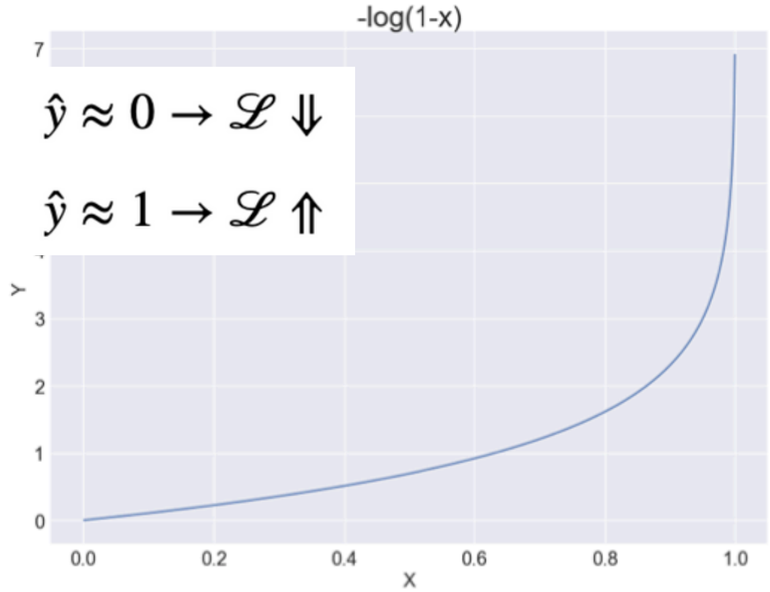


$\text{label} = 0: \quad \hat{y} \approx 1 \rightarrow \mathscr{L} \Uparrow$

$\text{label} = 0: \quad \hat{y} \approx 0 \rightarrow \mathscr{L} \Downarrow$

$\text{label} = 1: \quad \hat{y} \approx 0 \rightarrow \mathscr{L} \Downarrow$

$\text{label} = 1: \quad \hat{y} \approx 1 \rightarrow \mathscr{L} \Uparrow$



$$BCE = \begin{cases} -log(\hat{y}), where\ y = 1 \\ -log(1-\hat{y}), where\ y = 0 \end{cases}$$

$$= -\left[ylog(\hat{y}) + (1-y)log(1-\hat{y})\right]$$

HTTPS://SOSOEASY.TISTORY.COM/351

EXAMPLE: 암 구분

$$L = -[y \log(p) + (1 - y) \log(1 - p)]$$

1. 90% 암으로 추측한 암인 경우

$$L = -[1 \log(0.9) + (1 - 1) \log(1 - 0.9)]$$

$$L = -[\log(0.9) + 0]$$

$$L = -\log(0.9)$$

2. 90% 암으로 추측한, 정상

$$L = -[0 \log(0.9) + (1 - 0) \log(1 - 0.9)]$$

$$L = -[0 + \log(0.1)]$$

$$L = -\log(0.1)$$

3. 10% 암으로 추측한 암

$$L = -[1 \log(0.1) + (1 - 1) \log(1 - 0.1)]$$

$$L = -[\log(0.1) + 0]$$

$$L = -\log(0.1)$$

4. 10% 암으로 추측한 정상

$$L = -[0 \log(0.1) + (1 - 0) \log(1 - 0.1)]$$

$$L = -[0 + \log(0.9)]$$

$$L = -\log(0.9)$$

Y는 결과, P는 확률

Binary classification

```
#importing the libraries

import torch
import numpy as np
import matplotlib.pyplot as plt
```

```
#importing the dataset

from sklearn.datasets import load_breast_cancer
data = load_breast_cancer()
x = data['data']
y = data['target']
print("shape of x: {}\nshape of y: {}".format(x.shape,y.shape))
```

output:

```
shape of x: (569, 30)
shape of y: (569,)
```

```
#feature scaling

from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
x = sc.fit_transform(x)
```

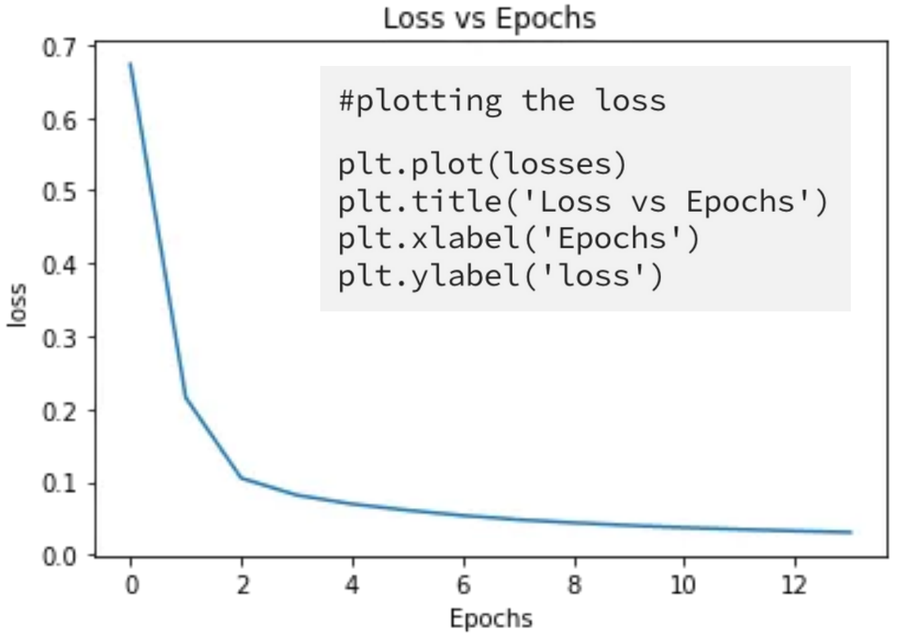
```
#defining dataset class

from torch.utils.data import Dataset, DataLoader

class dataset(Dataset):
  def __init__(self,x,y):
    self.x = torch.tensor(x,dtype=torch.float32)
    self.y = torch.tensor(y,dtype=torch.float32)
    self.length = self.x.shape[0]

  def __getitem__(self,idx):
    return self.x[idx],self.y[idx]

  def __len__(self):
    return self.length

trainset = dataset(x,y)

#DataLoader
trainloader = DataLoader(trainset,batch_size=64,shuffle=False)
```

HTTPS://MEDIUM.COM/ANALYTICS-VIDHYA/PYTORCH-FOR-DEEP-LEARNING-BINARY-CLASSIFICATION-LOGISTIC-REGRESSION-382ABD97FB43

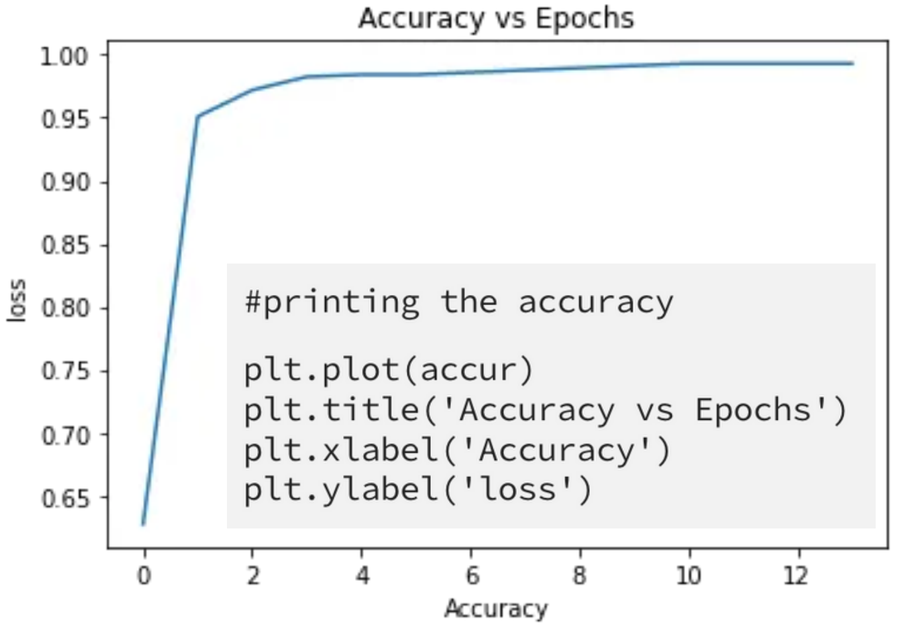
```
#defining the network

from torch import nn
from torch.nn import functional as F

class Net(nn.Module):
  def __init__(self,input_shape):
    super(Net,self).__init__()
    self.fc1 = nn.Linear(input_shape,32)
    self.fc2 = nn.Linear(32,64)
    self.fc3 = nn.Linear(64,1)

  def forward(self,x):
    x = torch.relu(self.fc1(x))
    x = torch.relu(self.fc2(x))
    x = torch.sigmoid(self.fc3(x))
    return x
```

```
#hyper parameters

learning_rate = 0.01
epochs = 700

# Model , Optimizer, Loss

model = Net(input_shape=x.shape[1])
optimizer = torch.optim.SGD(model.parameters(),lr=learning_rate)
loss_fn = nn.BCELoss()
```

```
#forward loop

losses = []
accur = []

for i in range(epochs):
  for j,(x_train,y_train) in enumerate(trainloader):

    #calculate output
    output = model(x_train)

    #calculate loss
    loss = loss_fn(output,y_train.reshape(-1,1))

    #accuracy
    predicted = model(torch.tensor(x,dtype=torch.float32))
    acc = (predicted.reshape(-1).detach().numpy().round() ==
y).mean()

    #backprop
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()


  if i%50 == 0:
    losses.append(loss)
    accur.append(acc)
    print("epoch {}\tloss : {}\t accuracy : {}".format(i,loss,acc))
```




```
#plotting the loss

plt.plot(losses)
plt.title('Loss vs Epochs')
plt.xlabel('Epochs')
plt.ylabel('loss')
```




```
#printing the accuracy

plt.plot(accur)
plt.title('Accuracy vs Epochs')
plt.xlabel('Accuracy')
plt.ylabel('loss')
```

그런데 나는 가위, 바위, 보를 구분하고 싶은데요?

MULTI-CLASS CLASSICIATION

다중 클래스 크로스 엔트로피

$$L = -\sum_{i=1}^{C} y_i \log(p_i)$$

고양이, 개, 새를 분류하는 문제의 클래스 C는 3입니다(고양이, 개, 새).

실제 출력 레이블은 '개' = (원-핫 인코딩) y는 [0, 1, 0]

실제 출력 확률은 = (SOFTMAX) [0.2, 0.7, 0.1].

$$L = -[(0 \cdot \log(0.2)) + (1 \cdot \log(0.7)) + (0 \cdot \log(0.1))]$$

$$L = -[0 + \log(0.7) + 0]$$

$$L = -\log(0.7)$$

```python
# 신경망 정의
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.fc1 = nn.Linear(10, 50)  # 입력 차원 10, 출력 차원 50
        self.fc2 = nn.Linear(50, 3)   # 출력 차원을 클래스 수인 3으로 설정

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = self.fc2(x)
        return x

# 모델, 손실 함수 및 옵티마이저 초기화
model = Net()
criterion = nn.CrossEntropyLoss()  # 소프트맥스 로직 포함
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

```python
# 임의의 입력과 레이블 생성
inputs = torch.randn(1, 10)  # 배치 크기 1, 특성 10
labels = torch.tensor([2])   # 실제 레이블 (클래스 인덱스 2)

# 순전파: 입력을 모델에 전달하여 예측값 계산
outputs = model(inputs)

# 손실 계산
loss = criterion(outputs, labels)

# 역전파
optimizer.zero_grad()
loss.backward()
optimizer.step()

print("Loss: {:.3f}".format(loss.item()))
```

In torch; CrossEntropyLoss = CE + Softmax

```python
# 신경망 정의
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.fc1 = nn.Linear(10, 50)  # 입력 차원 10, 출력 차원 50
        self.fc2 = nn.Linear(50, 3)   # 출력 차원을 클래스 수인 3으로 설정
        self.softmax = nn.Softmax(dim=1)  # 마지막 계층에 적용할 소프트맥스

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = self.fc2(x)
        x = self.softmax(x)  # 소프트맥스 적용
        return x

# 모델, 손실 함수 및 옵티마이저 초기화
model = Net()
criterion = nn.NLLLoss()  # 소프트맥스 적용된 출력에 적합한 손실 함수
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

```python
# 임의의 입력과 레이블 생성
inputs = torch.randn(1, 10)  # 배치 크기 1, 특성 10
labels = torch.tensor([2])   # 실제 레이블 (클래스 인덱스 2)

# 순전파: 입력을 모델에 전달하여 예측값 계산
outputs = model(inputs)

# 손실 계산: 소프트맥스 출력을 로그 확률로 변환
log_probs = torch.log(outputs)  # NLLLoss는 로그 확률을 입력으로 필요로 함
loss = criterion(log_probs, labels)

# 역전파
optimizer.zero_grad()
loss.backward()
optimizer.step()

print("Loss: {:.3f}".format(loss.item()))
```

NLLLOSS = NEGATIVE LOG LIKELIHOOD

Loss Functions

`nn.L1Loss`	Creates a criterion that measures the mean absolute error (MAE) between each element in the input x and target y.
`nn.MSELoss`	Creates a criterion that measures the mean squared error (squared L2 norm) between each element in the input x and target y.
`nn.CrossEntropyLoss`	This criterion computes the cross entropy loss between input logits and target.
`nn.CTCLoss`	The Connectionist Temporal Classification loss.
`nn.NLLLoss`	The negative log likelihood loss.
`nn.PoissonNLLLoss`	Negative log likelihood loss with Poisson distribution of target.
`nn.GaussianNLLLoss`	Gaussian negative log likelihood loss.
`nn.KLDivLoss`	The Kullback-Leibler divergence loss.
`nn.BCELoss`	Creates a criterion that measures the Binary Cross Entropy between the target and the input probabilities:
`nn.BCEWithLogitsLoss`	This loss combines a *Sigmoid* layer and the *BCELoss* in one single class.
`nn.MarginRankingLoss`	Creates a criterion that measures the loss given inputs $x1$, $x2$, two 1D mini-batch or 0D *Tensors*, and a label 1D mini-batch or 0D *Tensor* y (containing 1 or -1).
`nn.HingeEmbeddingLoss`	Measures the loss given an input tensor x and a labels tensor y (containing 1 or -1).
`nn.MultiLabelMarginLoss`	Creates a criterion that optimizes a multi-class multi-classification hinge loss (margin-based loss) between input x (a 2D mini-batch *Tensor*) and output y (which is a 2D *Tensor* of target class indices).
`nn.HuberLoss`	Creates a criterion that uses a squared term if the absolute element-wise error falls below delta and a delta-scaled L1 term otherwise.
`nn.SmoothL1Loss`	Creates a criterion that uses a squared term if the absolute element-wise error falls below beta and an L1 term otherwise.
`nn.SoftMarginLoss`	Creates a criterion that optimizes a two-class classification logistic loss between input tensor x and target tensor y (containing 1 or -1).
`nn.MultiLabelSoftMarginLoss`	Creates a criterion that optimizes a multi-label one-versus-all loss based on max-entropy, between input x and target y of size (N, C).
`nn.CosineEmbeddingLoss`	Creates a criterion that measures the loss given input tensors x_1, x_2 and a *Tensor* label y with values 1 or -1.
`nn.MultiMarginLoss`	Creates a criterion that optimizes a multi-class classification hinge loss (margin-based loss) between input x (a 2D mini-batch *Tensor*) and output y (which is a 1D tensor of target class indices, $0 \leq y \leq \mathrm{x.size}(1) - 1$):
`nn.TripletMarginLoss`	Creates a criterion that measures the triplet loss given an input tensors $x1$, $x2$, $x3$ and a margin with a value greater than 0.
`nn.TripletMarginWithDistanceLoss`	Creates a criterion that measures the triplet loss given input tensors a, p, and n (representing anchor, positive, and negative examples, respectively), and a nonnegative, real-valued function ("distance function") used to compute the relationship between the anchor and positive example ("positive distance") and the anchor and negative example ("negative distance").

LOSS는 여러가지를 더해도 되지

L1 Regularization

$$\text{Cost} = \sum_{i=0}^{N}(y_i - \sum_{j=0}^{M} x_{ij}W_j)^2 + \lambda \sum_{j=0}^{M} |W_j|$$

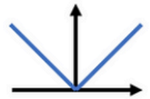
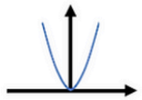
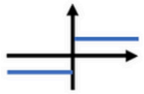
L2 Regularization

$$\text{Cost} = \sum_{i=0}^{N}(y_i - \sum_{j=0}^{M} x_{ij}W_j)^2 + \lambda \sum_{j=0}^{M} W_j^2$$

Loss function

Regularization Term

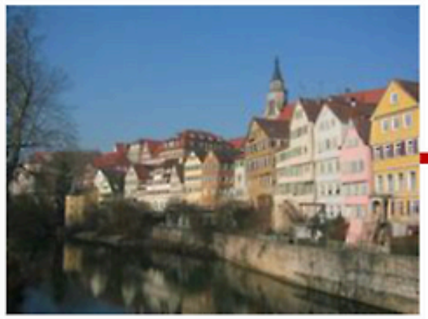
W가 뭐죠 여기서?

구분	Lasso	Ridge
수식	$\|\boldsymbol{w}\|_1$ 	$\frac{1}{2}\|\boldsymbol{w}\|^2$ 
미분	$\begin{cases} 1 & w < 0 \\ -1 & w > 0 \end{cases}$ 	$\boldsymbol{w}$
특성	• 가중치 값을 정확하게 0으로 만듦 • 중요한 특징을 '선택'하는 효과 • 모델에 Sparsity를 가함.	• 큰 가중치의 값을 작게 만듦 • 모델 전반적인 복잡도를 감소시키는 효과 • 가중치의 값이 0이 되게 하지는 못함

여러 목적을 합쳐서 줄일 수 있구나

$$L_{total}(\vec{p}, \vec{a}, \vec{x}) = \alpha L_{content}(\vec{p}, \vec{x}) + \beta L_{style}(\vec{a}, \vec{x})$$

α, β는 반영비율이고!!



Content Image $\vec{p}$

$L_{content}$

Style Image $\vec{a}$

L_{style}

Update

$\vec{x}$

Result $\vec{x}$

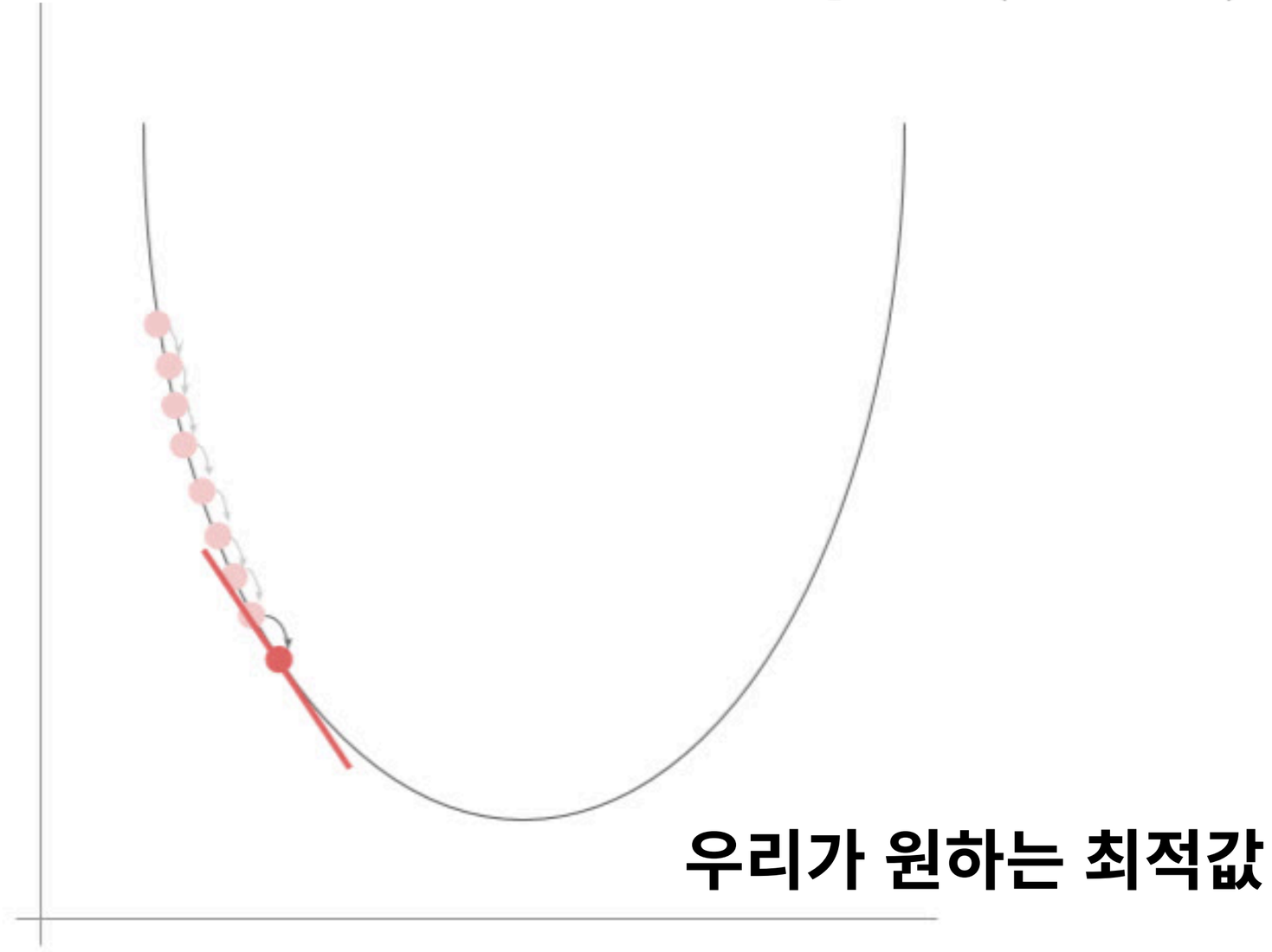
두 가지 목적을 동시에 이루는?!

옵티마이저와 스케쥴링

$$\frac{\partial C}{\partial w_1} = \frac{\partial C}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial u} \times \frac{\partial u}{\partial h_1} \times \frac{\partial h_1}{\partial s} \times \frac{\partial s}{\partial w_1} = \boxed{-24}$$

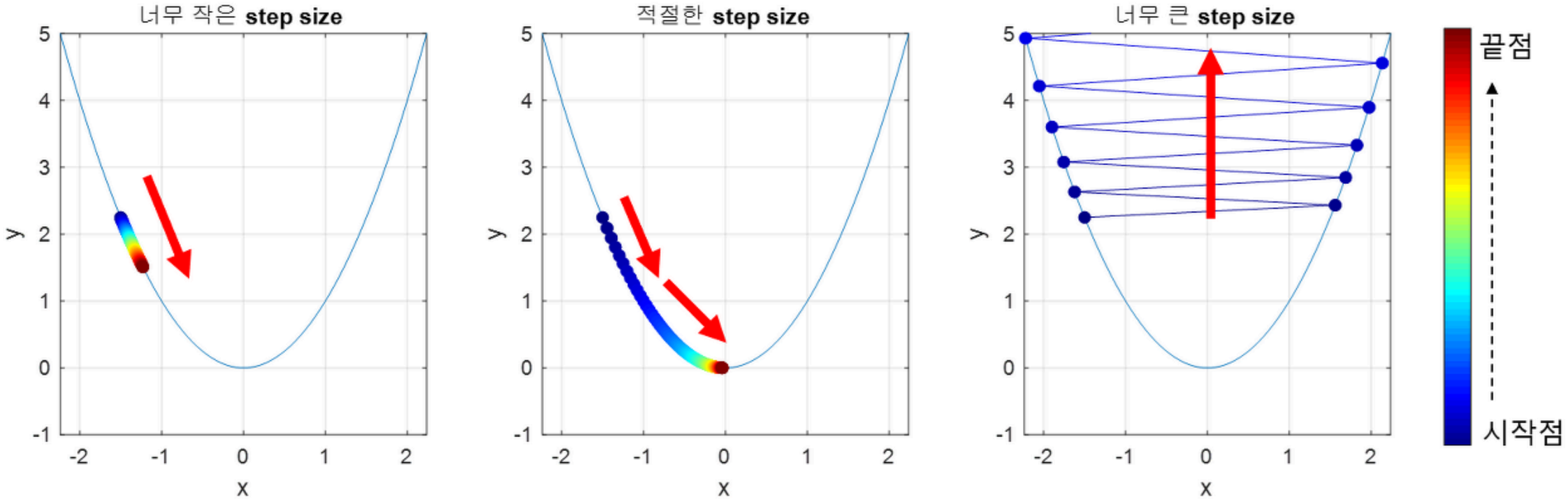
학습률 η : 0.001

$w_1 := 2 - (0.001 \times -24)$



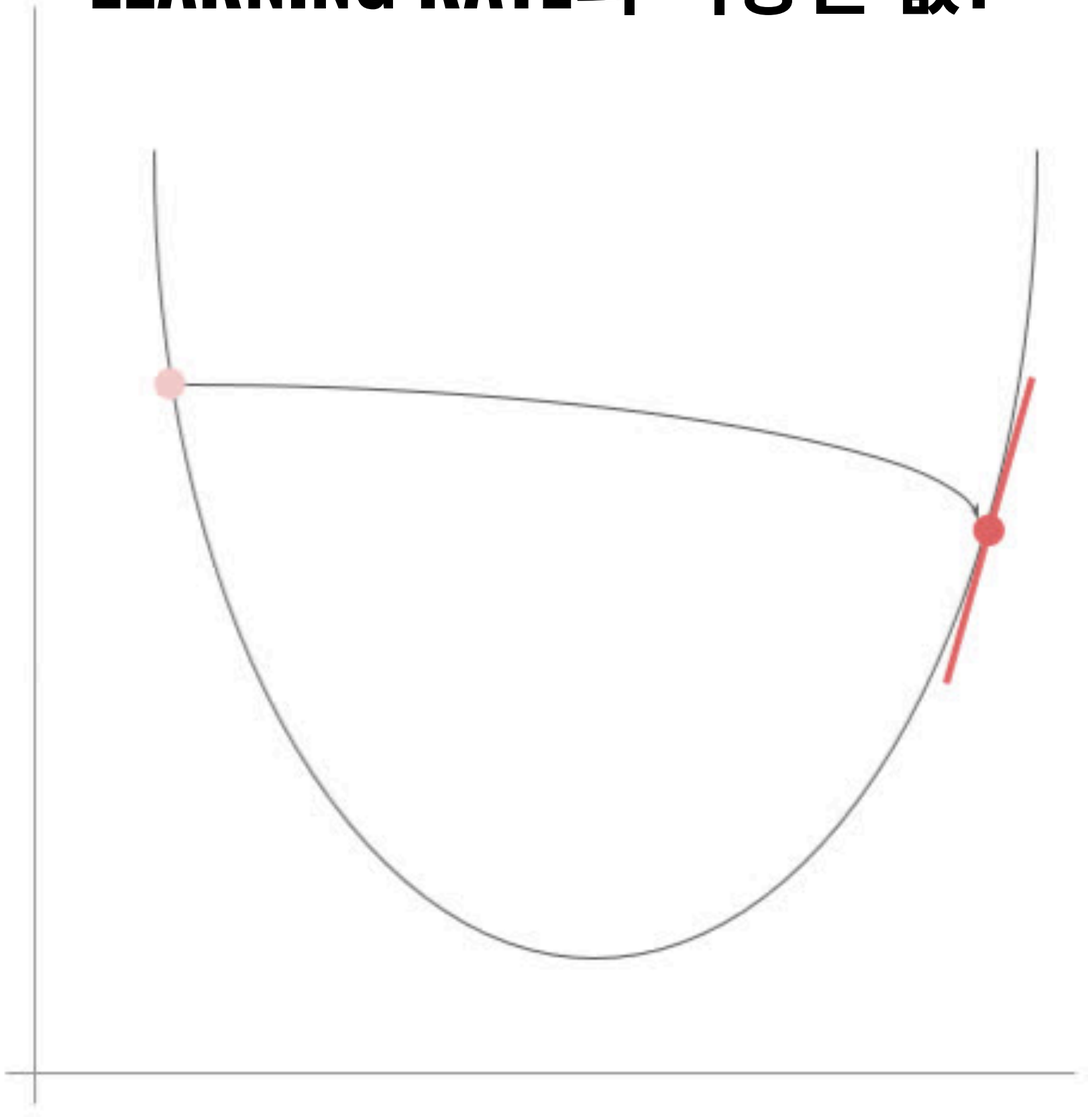
우리가 원하는 최적값

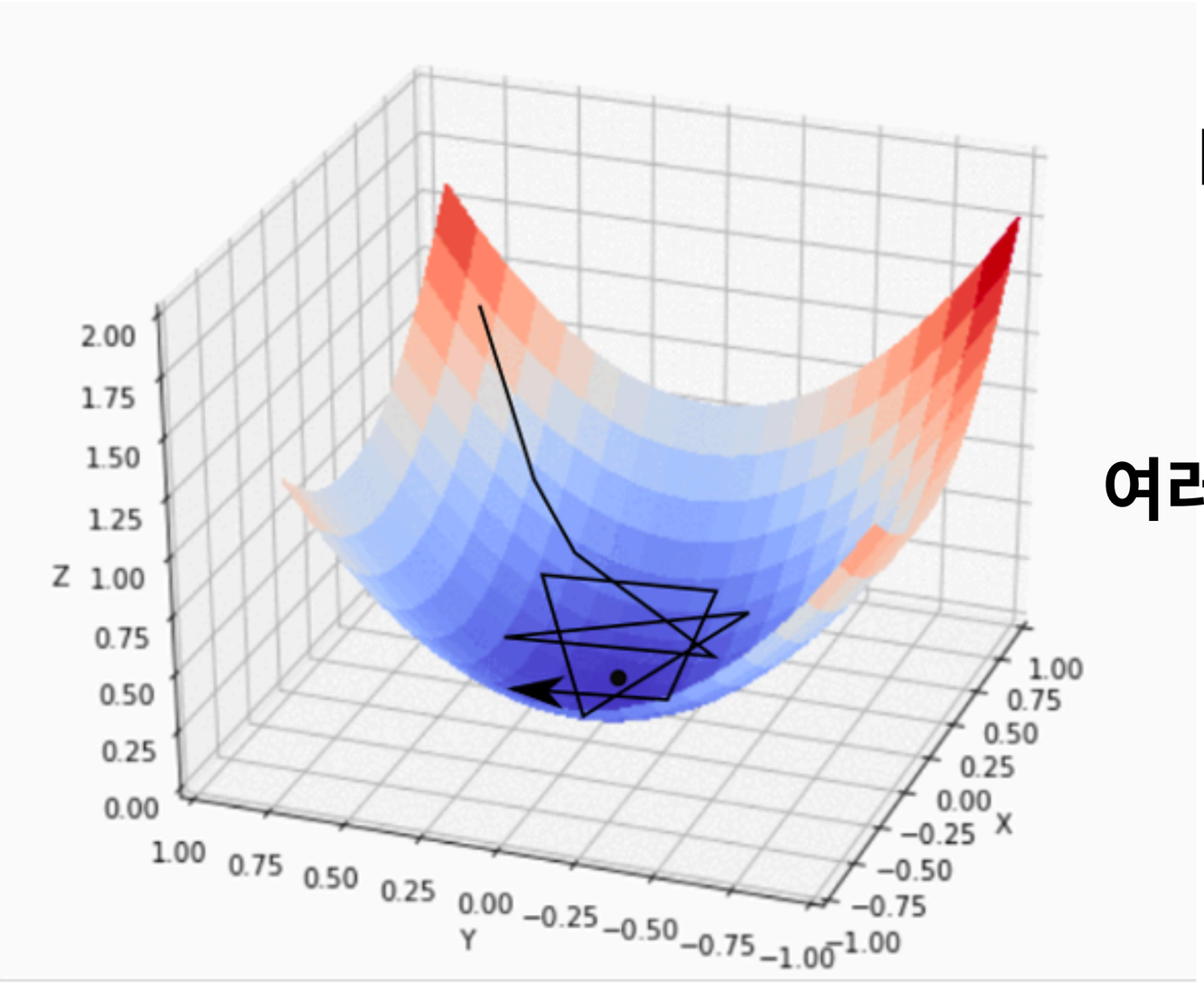
어떻게 적절하지!!!



HTTPS://HEEYA-STUPIDBUTSTUDYING.TISTORY.COM/ENTRY/ML-%EC%8B%A0%EA%B2%BD%EB%A7%9D%EC%97%90%EC%84%9C%EC%9D%98-OPTIMIZER-%EC%97%AD%ED%95%A0%EA%B3%BC-%EC%A2%85%EB%A5%98

LEARNING RATE의 적당한 값?





CONVEX

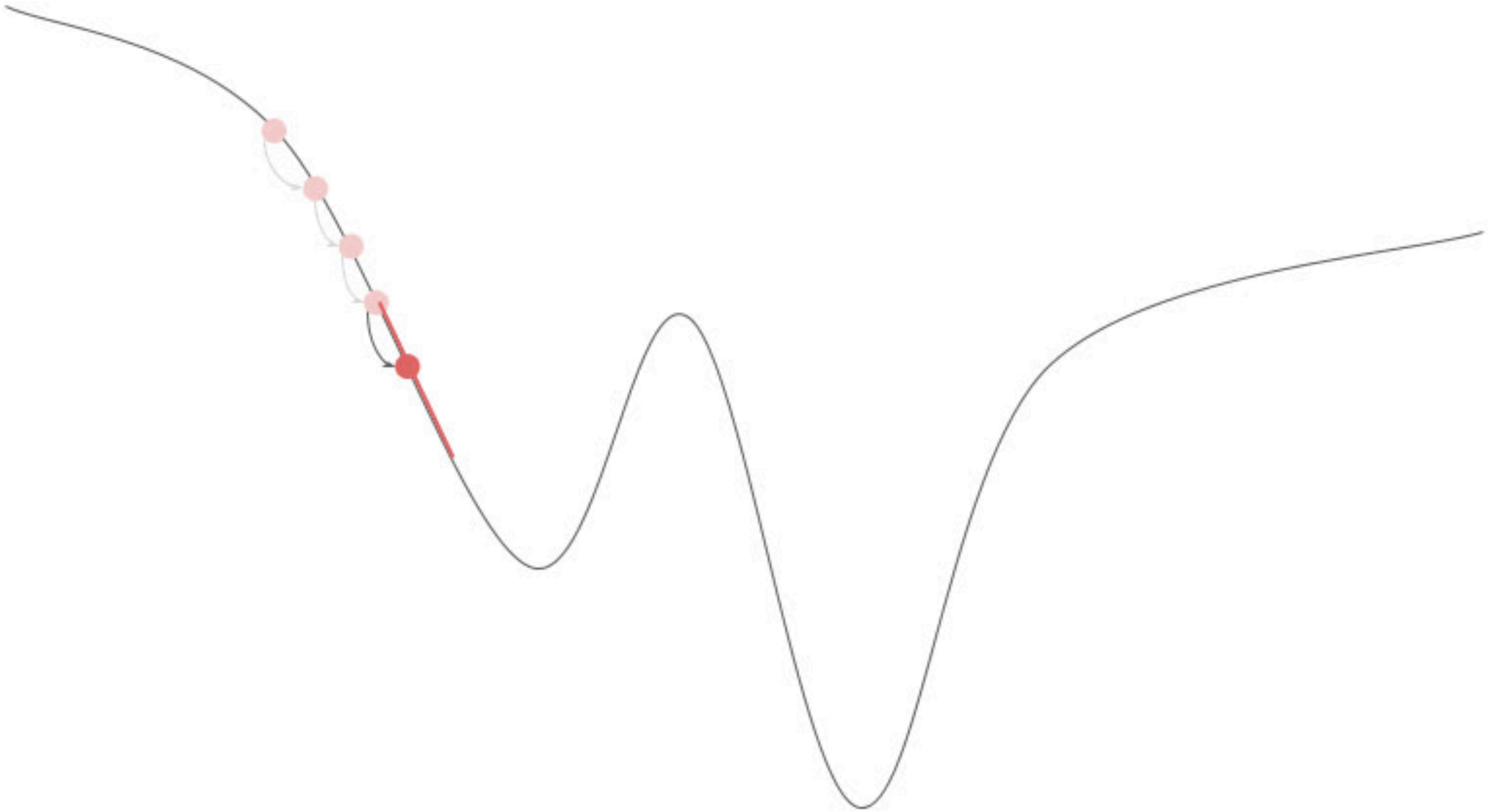
Q. 여러분은 짙은 안개 속에서
LOSS라는 산을 내려오고 있습니다.

이 산은 어떻게 생겼을까요?

여러분이 도착한 곳이 가장 최저점일까요?

어쩌면 좋을까요?

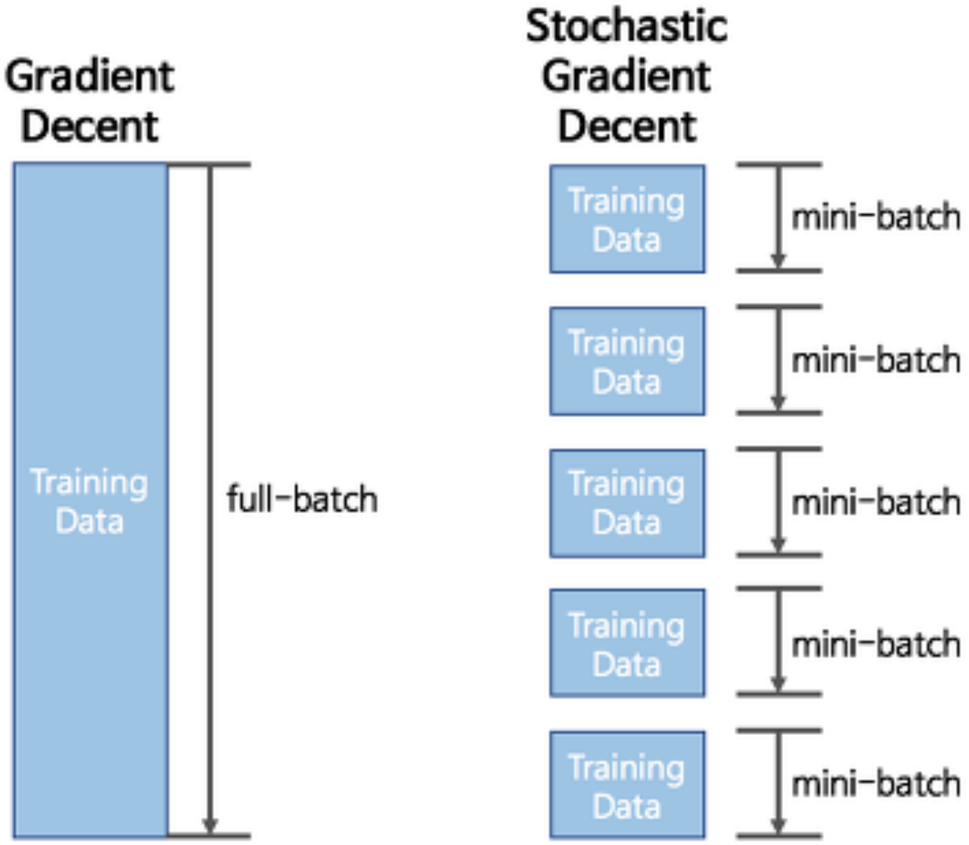
이럴수도 있자나요?



SGD (STOCHASTIC GRADIENT DESCENT; 확률적 경사하강법)

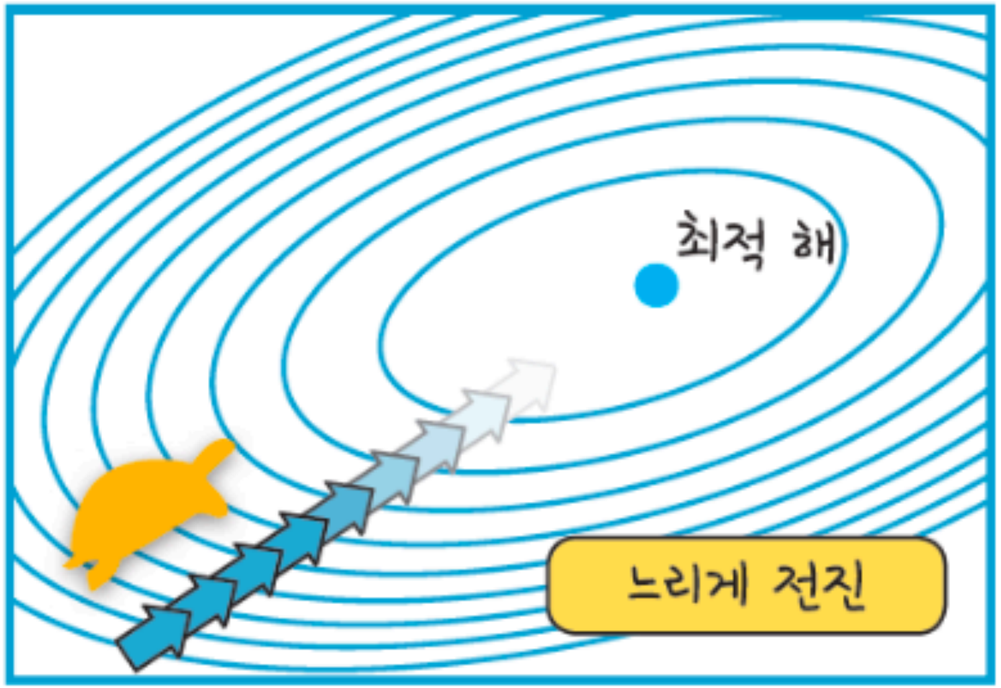
“학습데이터
전체를 다 보고
한번 업데이트”

근데 데이터가 너무
많아서... 힘들어..

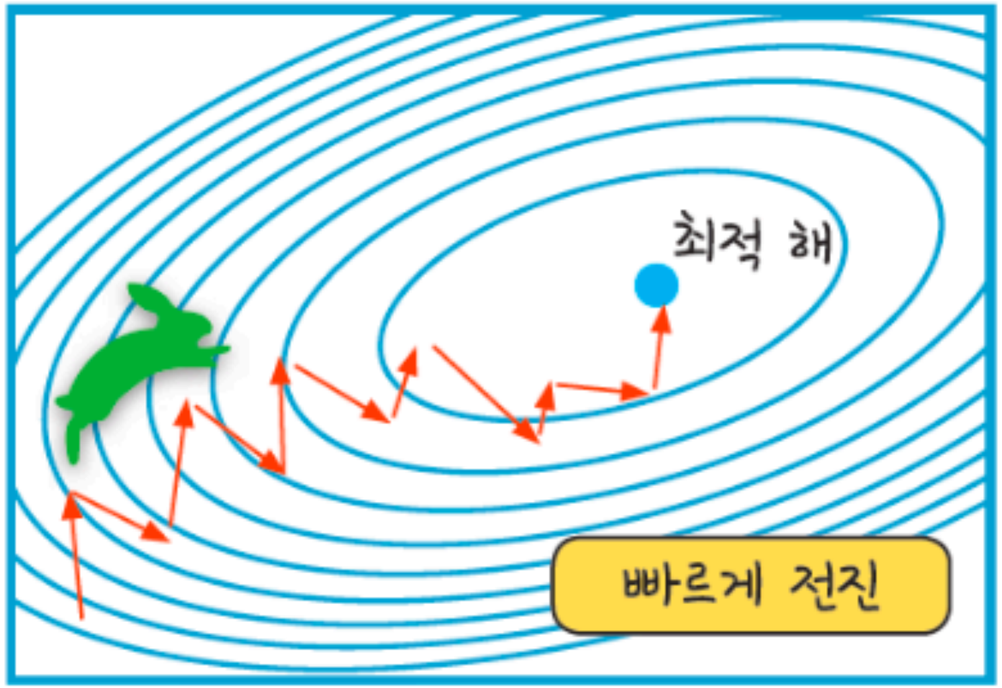


‘그러면.. 좀 나눠서 부분별로
보고 자주 업데이트하면?”

이게 되네...

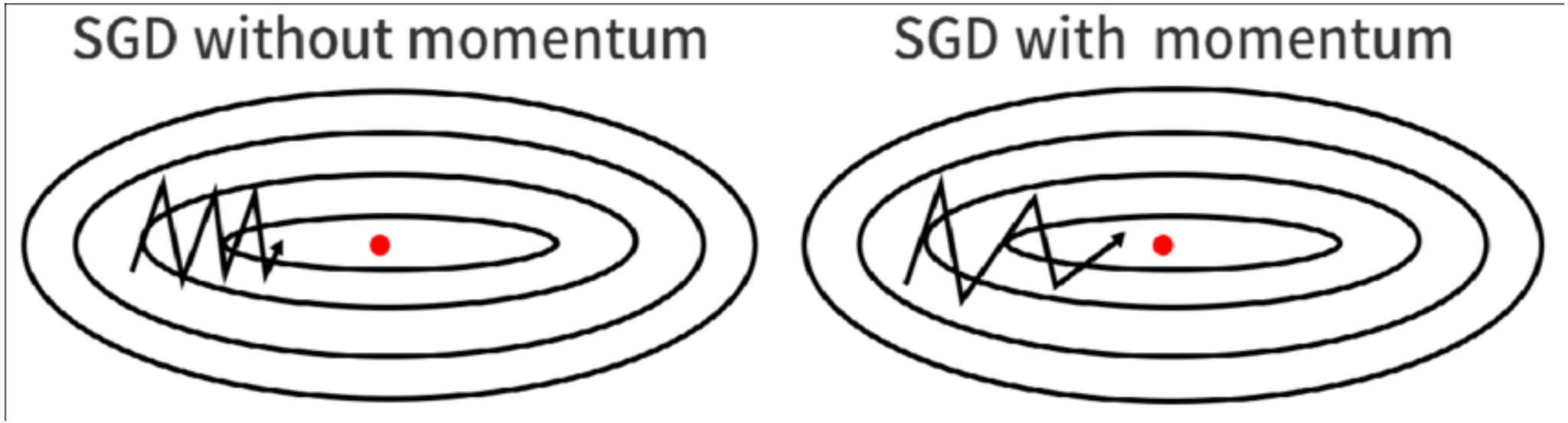


경사 하강법



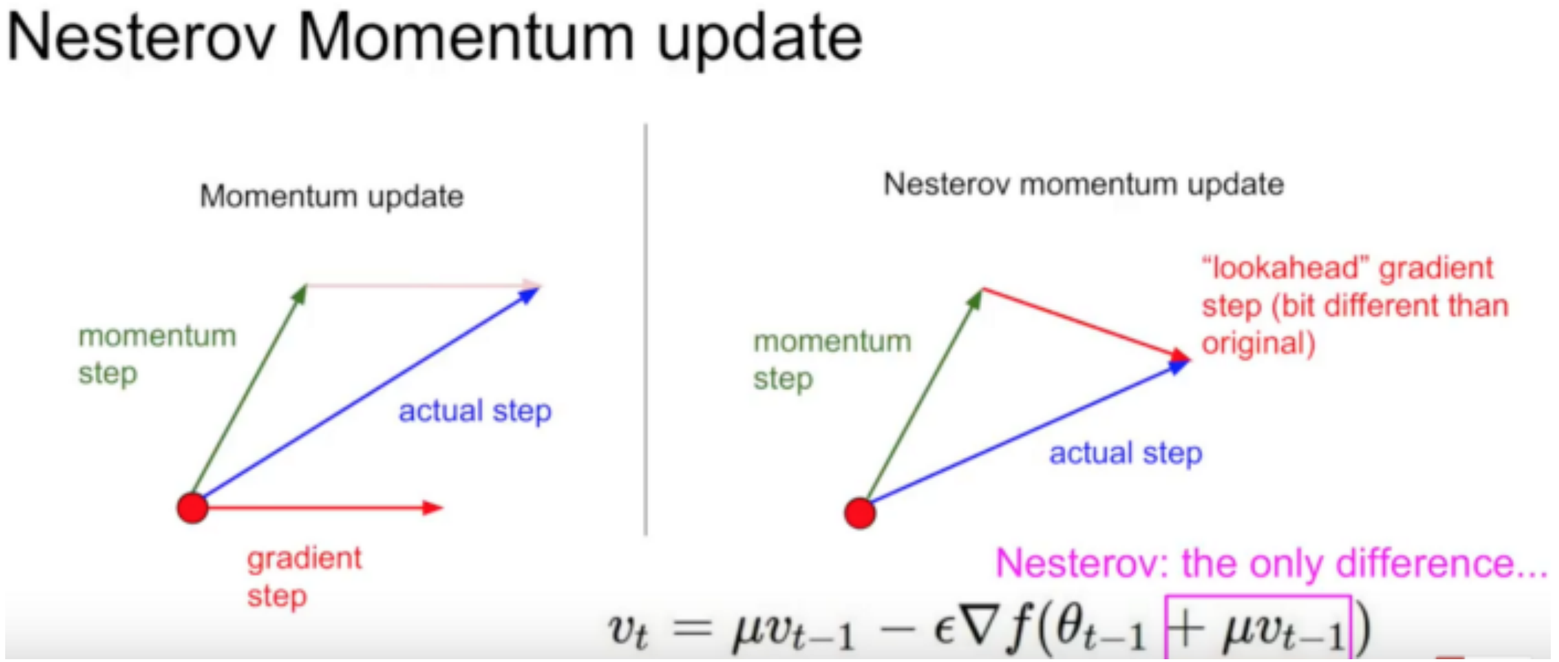
확률적 경사 하강법

모멘텀 (MOMENTUM)



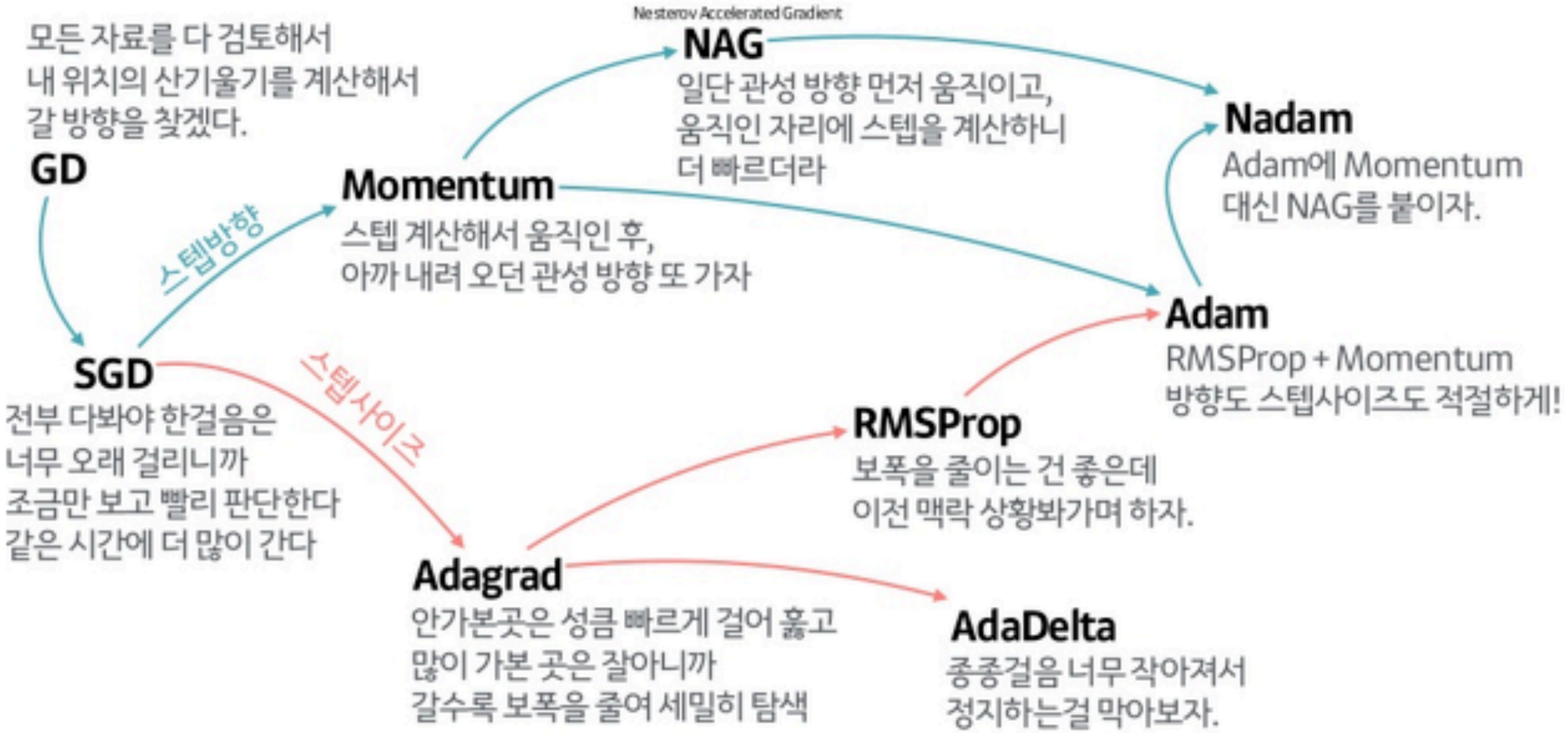
일단 SGD로 움직이는데... 이전에 걸어 오던 방향으로 좀 더 가지뭐..

NESTREROV MOMENTUM

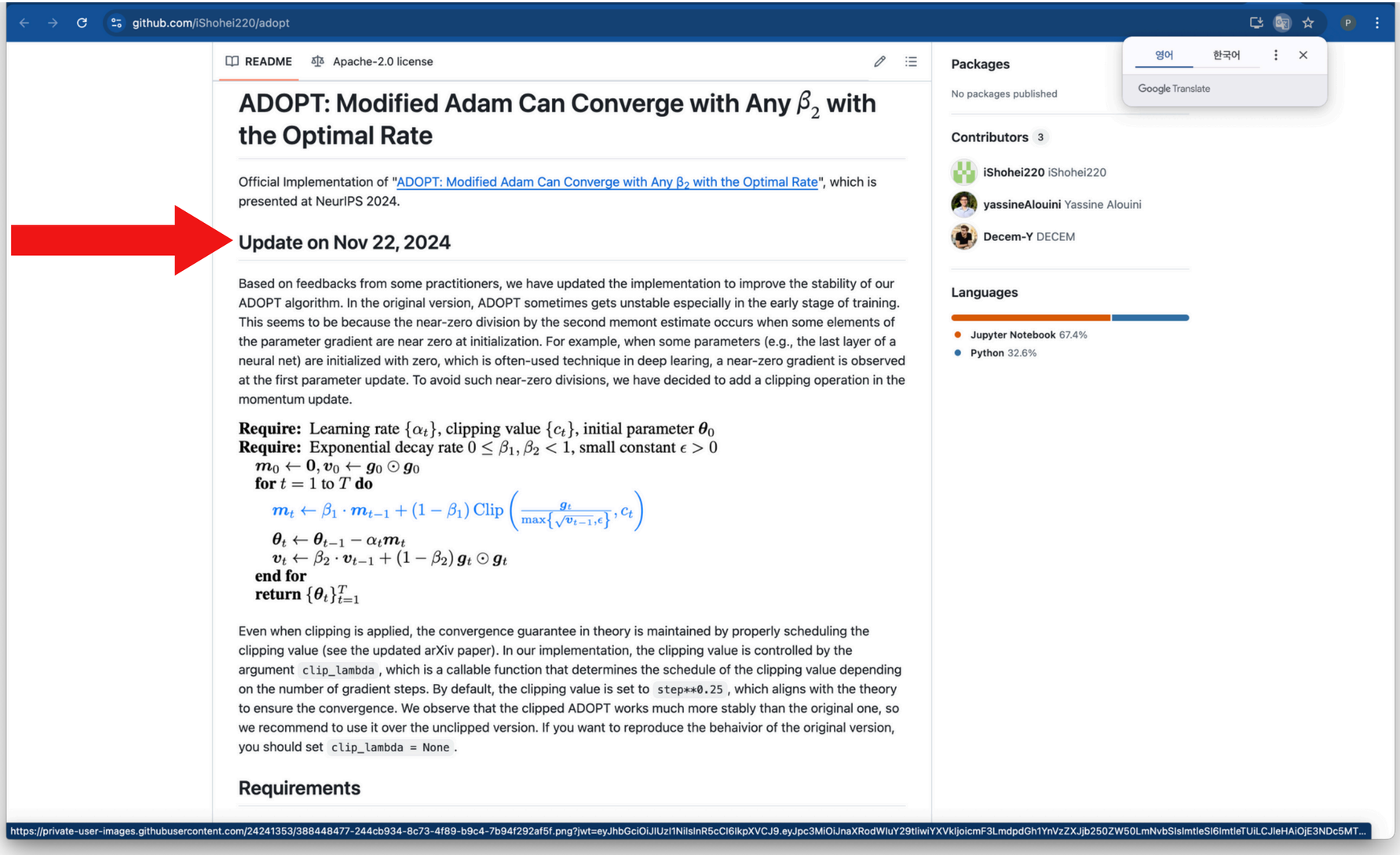


MOMENTUM 방식이긴한데 순서를 바꿔서.. 이전에 걷던방향으로 먼저하고 움직이니까 좋은데?

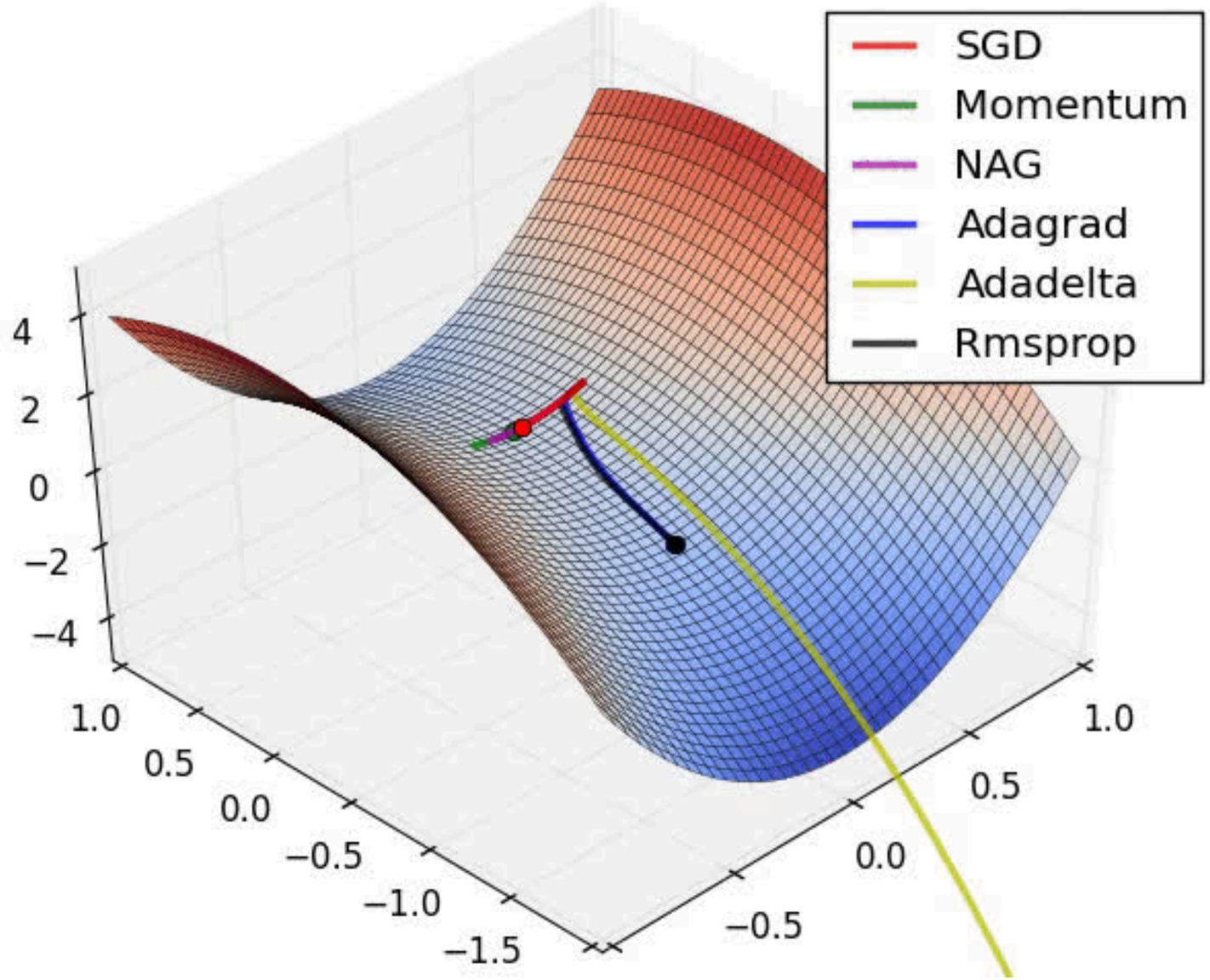
다양한 방법이 존재합니다!

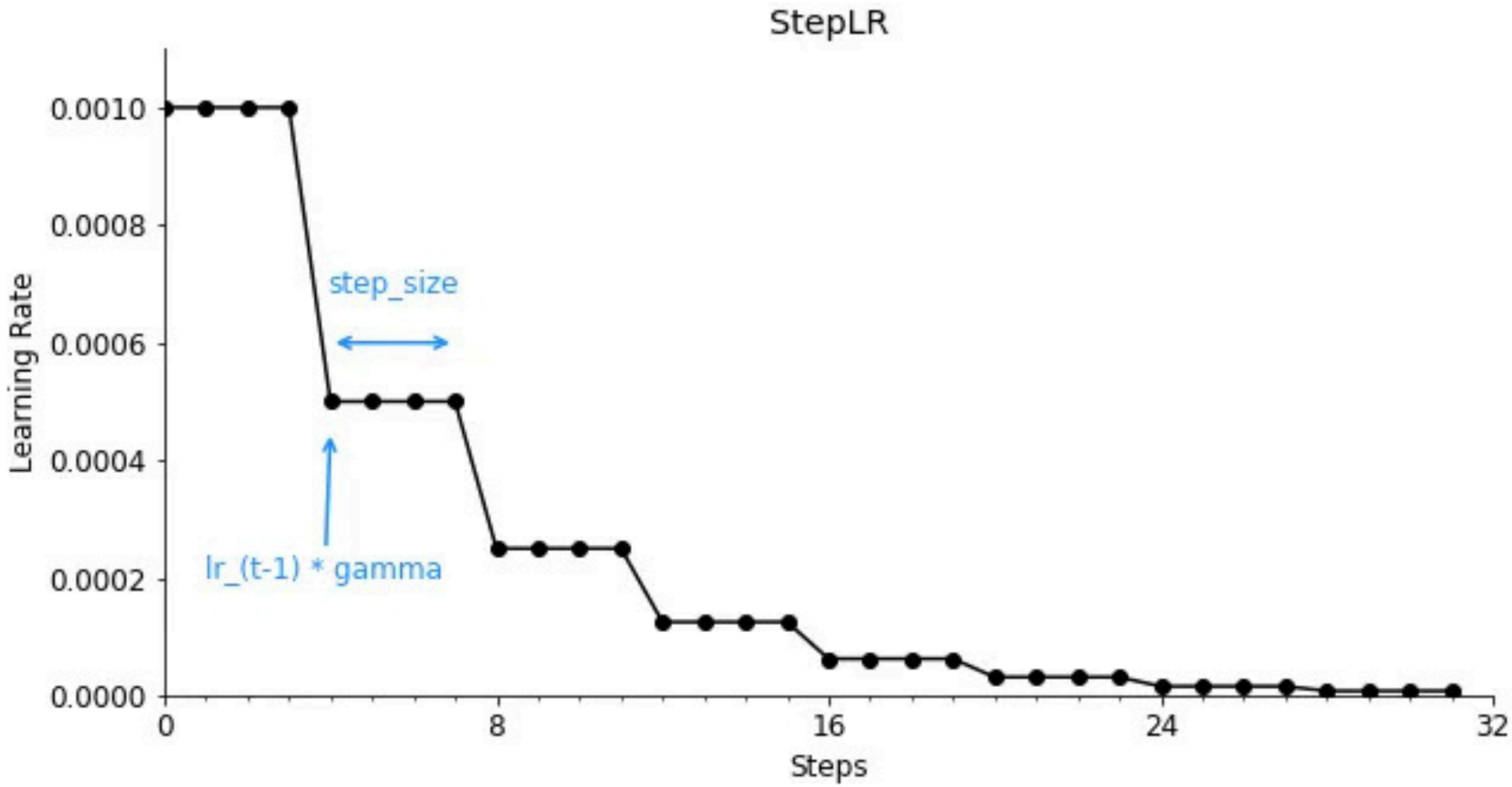


출처 : https://www.slideshare.net/yongho/ss-79607172

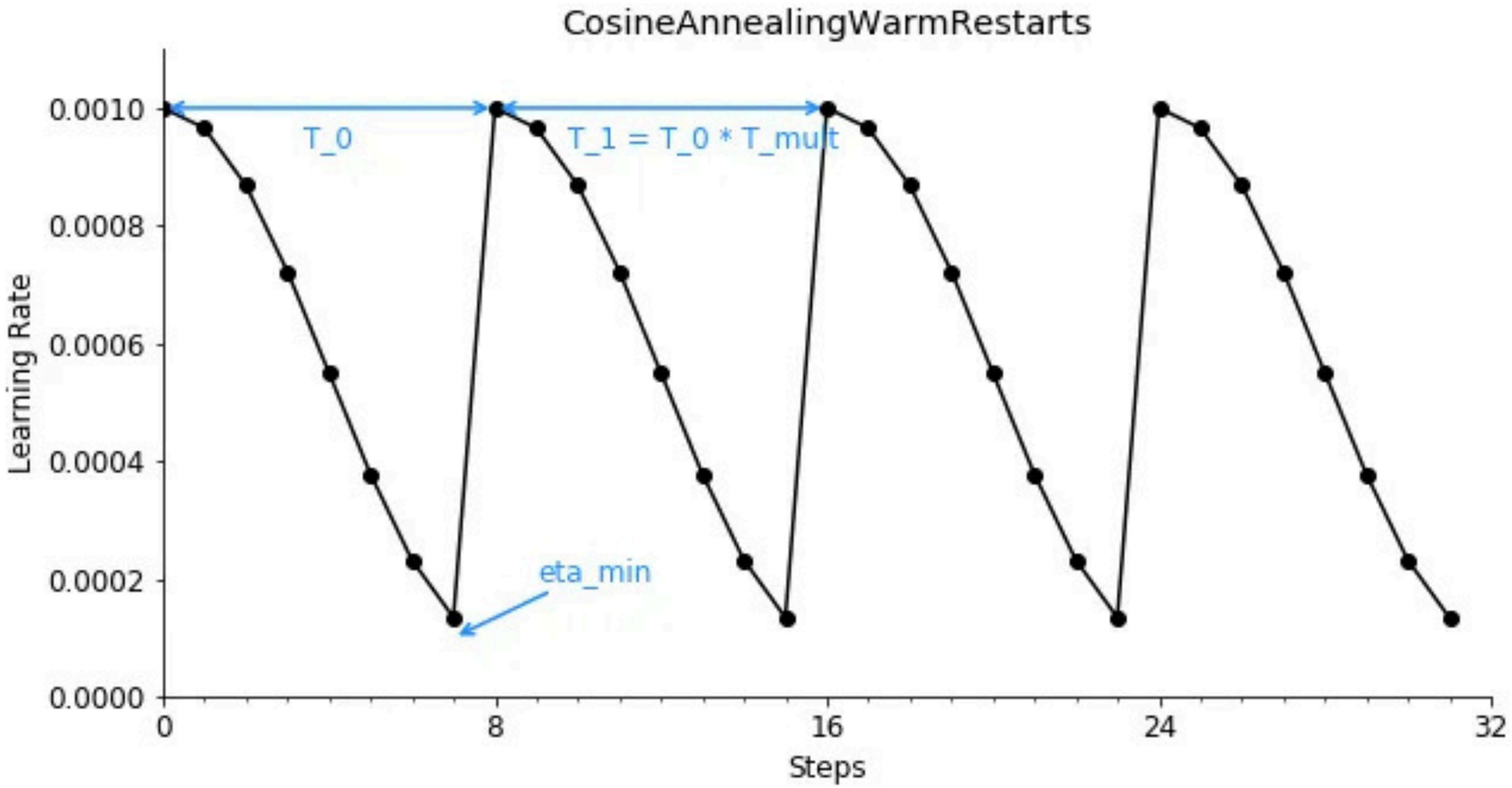

github.com/iShohei220/adopt
README
Apache-2.0 license
ADOPT: Modified Adam Can Converge with Any β_2 with the Optimal Rate
Official Implementation of "ADOPT: Modified Adam Can Converge with Any β_2 with the Optimal Rate", which is presented at NeurIPS 2024.
Update on Nov 22, 2024
Based on feedbacks from some practitioners, we have updated the implementation to improve the stability of our ADOPT algorithm. In the original version, ADOPT sometimes gets unstable especially in the early stage of training. This seems to be because the near-zero division by the second memont estimate occurs when some elements of the parameter gradient are near zero at initialization. For example, when some parameters (e.g., the last layer of a neural net) are initialized with zero, which is often-used technique in deep learing, a near-zero gradient is observed at the first parameter update. To avoid such near-zero divisions, we have decided to add a clipping operation in the momentum update.
Require: Learning rate $\{\alpha_t\}$, clipping value $\{c_t\}$, initial parameter $\boldsymbol{\theta}_0$
Require: Exponential decay rate $0 \leq \beta_1, \beta_2 < 1$, small constant $\epsilon > 0$
$\boldsymbol{m}_0 \leftarrow \mathbf{0}, \boldsymbol{v}_0 \leftarrow \boldsymbol{g}_0 \odot \boldsymbol{g}_0$
for $t = 1$ to T do
$\boldsymbol{m}_t \leftarrow \beta_1 \cdot \boldsymbol{m}_{t-1} + (1-\beta_1)\,\mathrm{Clip}\left(\frac{\boldsymbol{g}_t}{\max\{\sqrt{\boldsymbol{v}_{t-1}}, \epsilon\}}, c_t\right)$
$\boldsymbol{\theta}_t \leftarrow \boldsymbol{\theta}_{t-1} - \alpha_t \boldsymbol{m}_t$
$\boldsymbol{v}_t \leftarrow \beta_2 \cdot \boldsymbol{v}_{t-1} + (1-\beta_2)\,\boldsymbol{g}_t \odot \boldsymbol{g}_t$
end for
return $\{\boldsymbol{\theta}_t\}_{t=1}^{T}$
Even when clipping is applied, the convergence guarantee in theory is maintained by properly scheduling the clipping value (see the updated arXiv paper). In our implementation, the clipping value is controlled by the argument clip_lambda, which is a callable function that determines the schedule of the clipping value depending on the number of gradient steps. By default, the clipping value is set to step**0.25, which aligns with the theory to ensure the convergence. We observe that the clipped ADOPT works much more stably than the original one, so we recommend to use it over the unclipped version. If you want to reproduce the behaivior of the original version, you should set clip_lambda = None.
Requirements
Packages
No packages published
Contributors 3
iShohei220 iShohei220
yassineAlouini Yassine Alouini
Decem-Y DECEM
Languages
Jupyter Notebook 67.4%
Python 32.6%
영어
한국어
Google Translate
https://private-user-images.githubusercontent.com/24241353/388448477-244cb934-8c73-4f89-b9c4-7b94f292af5f.png?jwt=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJnaXRodWIuY29tIiwiYXVkIjoicmF3LmdpdGh1YnVzZXJjb250ZW50LmNvbSIsImtleSI6ImtleTUiLCJleHAiOjE3NDc5MT...

연구가 계속 진행중이다!!!


SGD
Momentum
NAG
Adagrad
Adadelta
Rmsprop
4
2
0
−2
−4
1.0
0.5
0.0
−0.5
−1.0
−1.5
1.0
0.5
0.0
−0.5

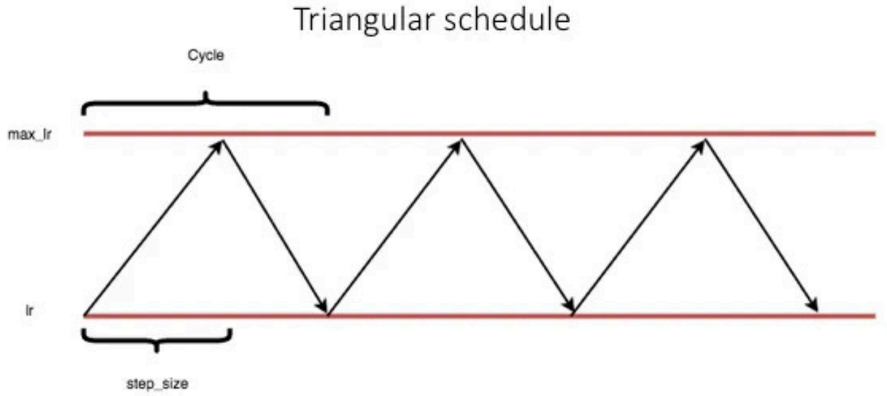


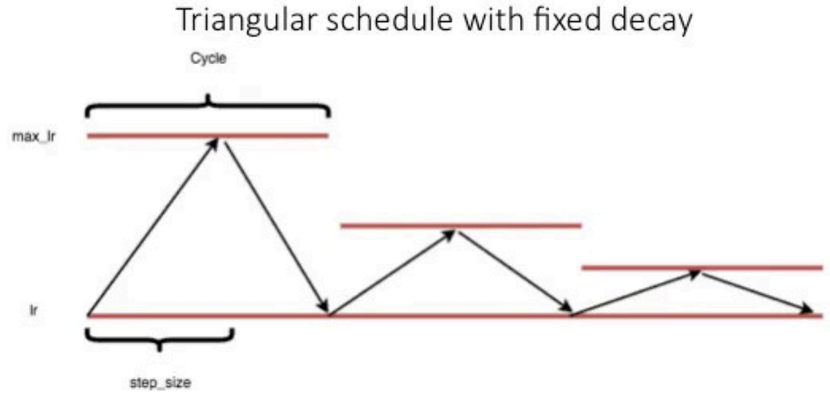
그런데... 기본적으로 학습이 진행될수록 세밀하게 움직여야되..

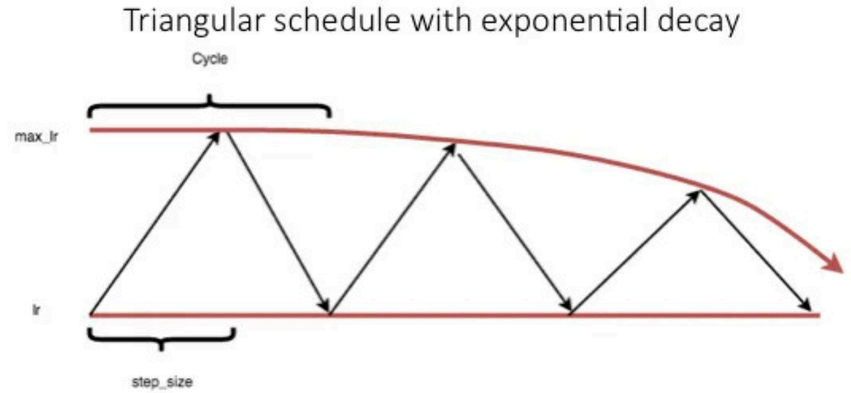


아 그런데 가끔 흔들어줘야되!!!

그런데..
관성(!)을 보니까..
되려 처음에는 과감하게
움직이면 안되.



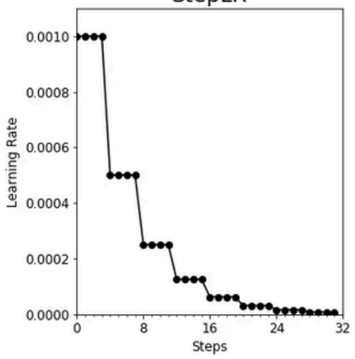




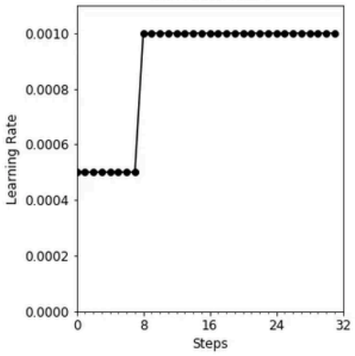
그리고 다른거랑 좀
섞으면...

HTTPS://MEDIUM.COM/DATA-SCIENCE/ADAPTIVE-AND-CYCLICAL-LEARNING-RATES-USING-PYTORCH-2BF904D18DEE

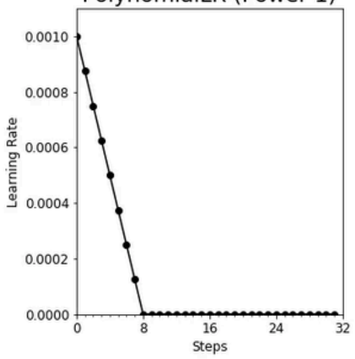
StepLR



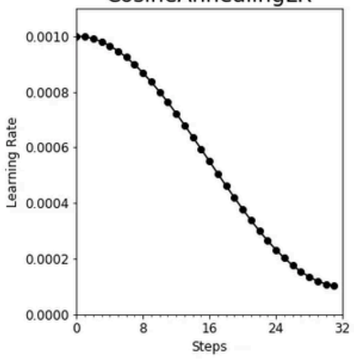
ConstantLR



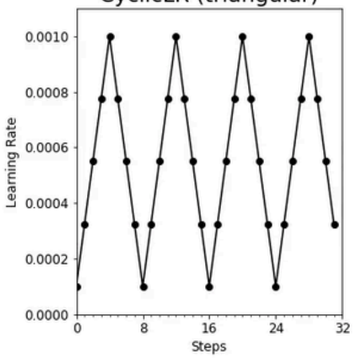
PolynomialLR (Power 1)



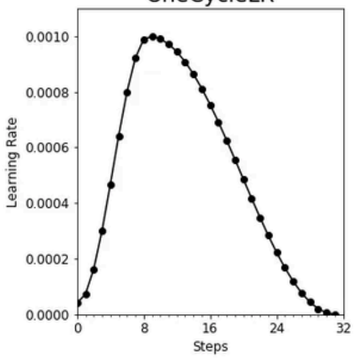
CosineAnnealingLR



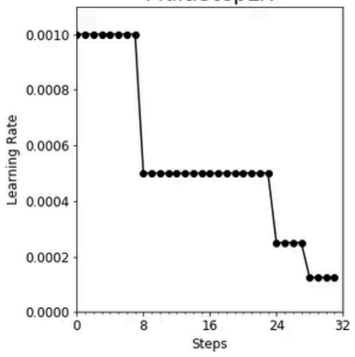
CyclicLR (triangular)



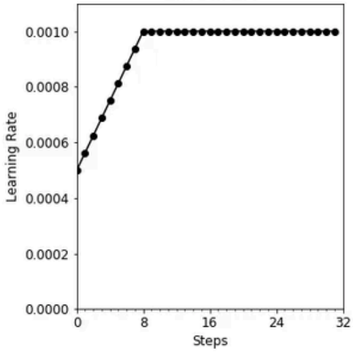
OneCycleLR



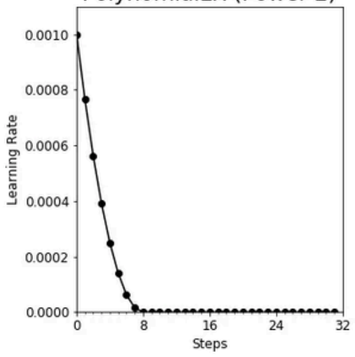
MultiStepLR



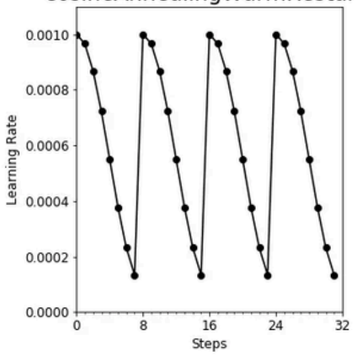
LinearLR



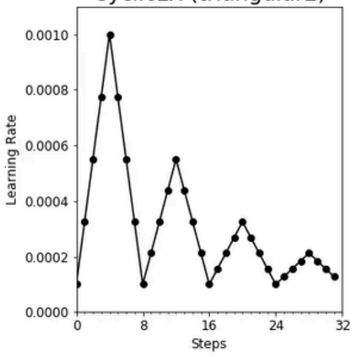
PolynomialLR (Power 2)



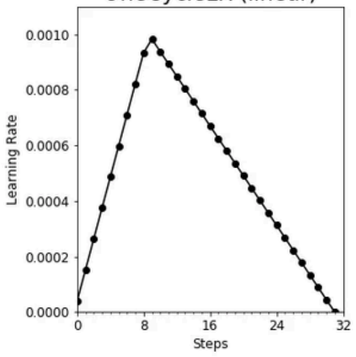
CosineAnnealingWarmRestarts



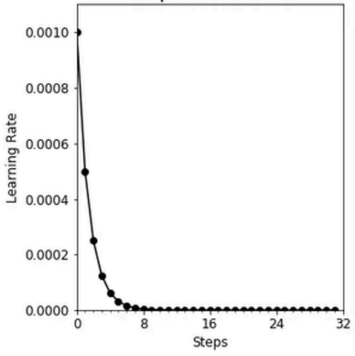
CyclicLR (triangular2)



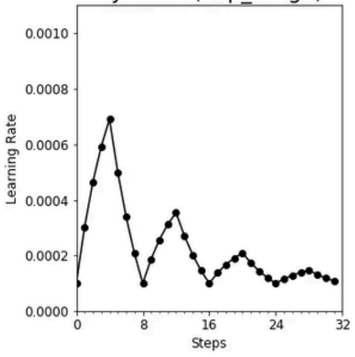
OneCycleLR (linear)



ExponentialLR

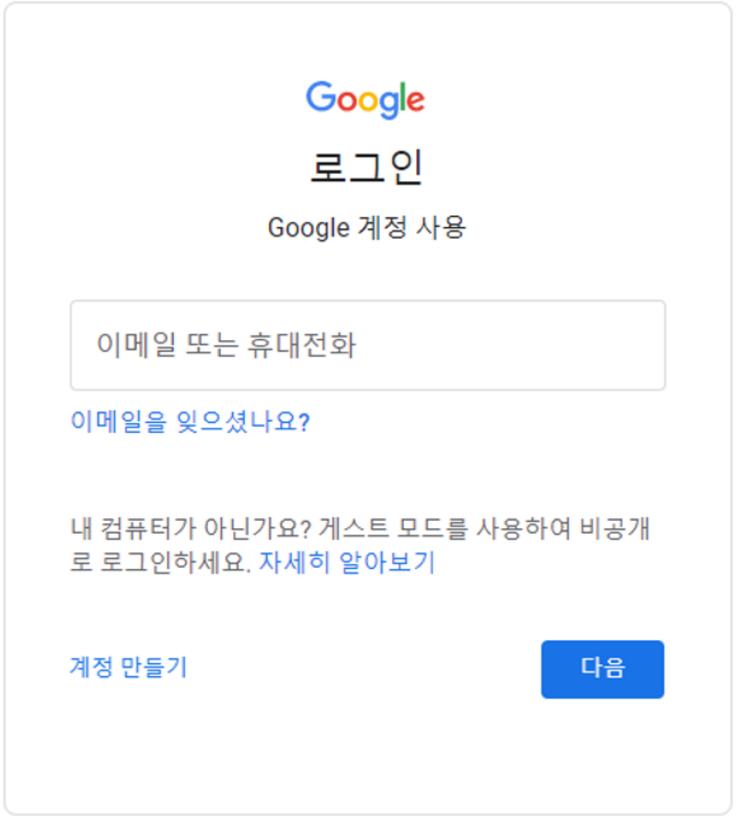


CyclicLR (exp_range)



Colab을 사용하려면 구글 계정이 필요합니다

- 계정이 없다면 가입을 해봅시다



10000장

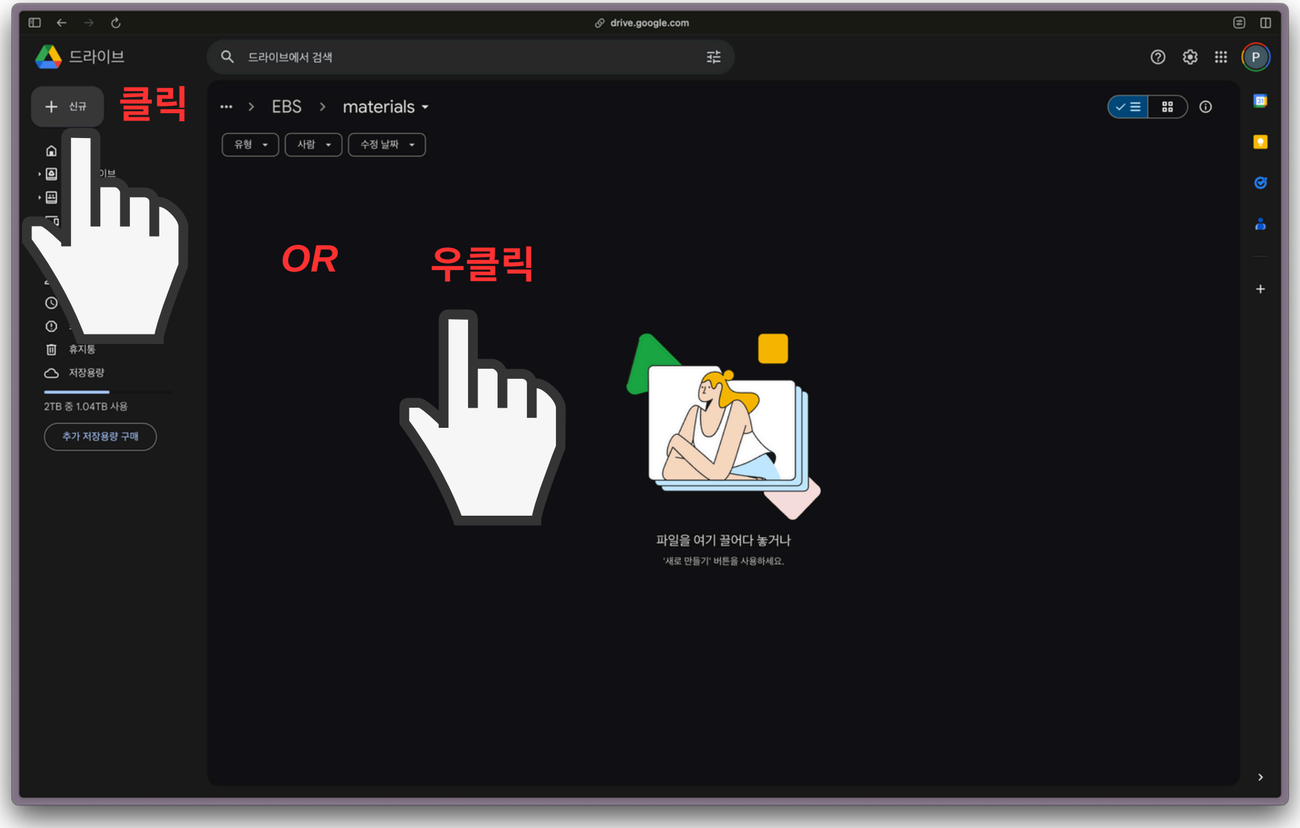
VS

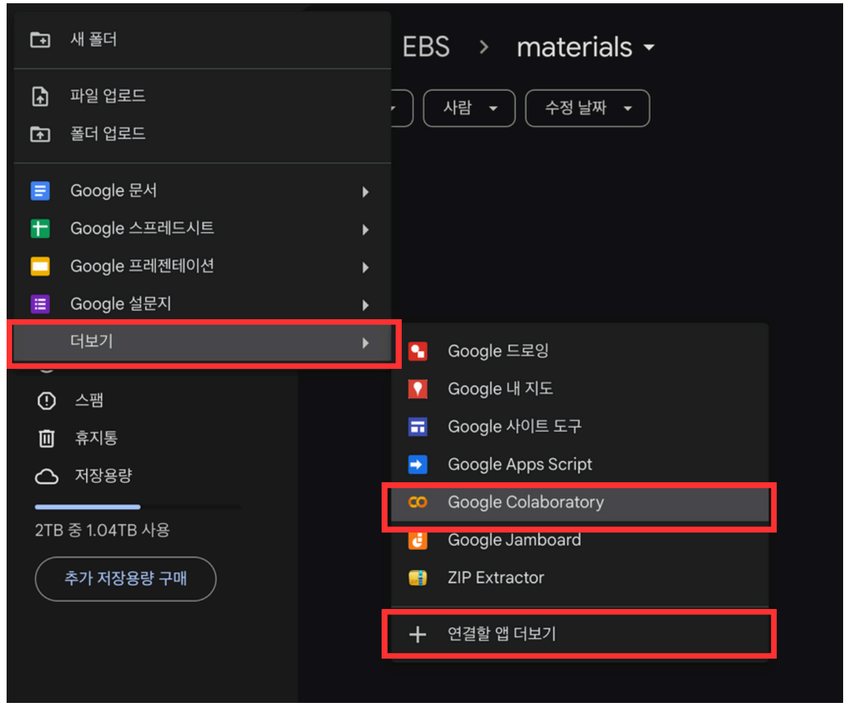
10000장

고양이 VS 개
자동으로 구분하는 인공지능을 만들고 싶다.

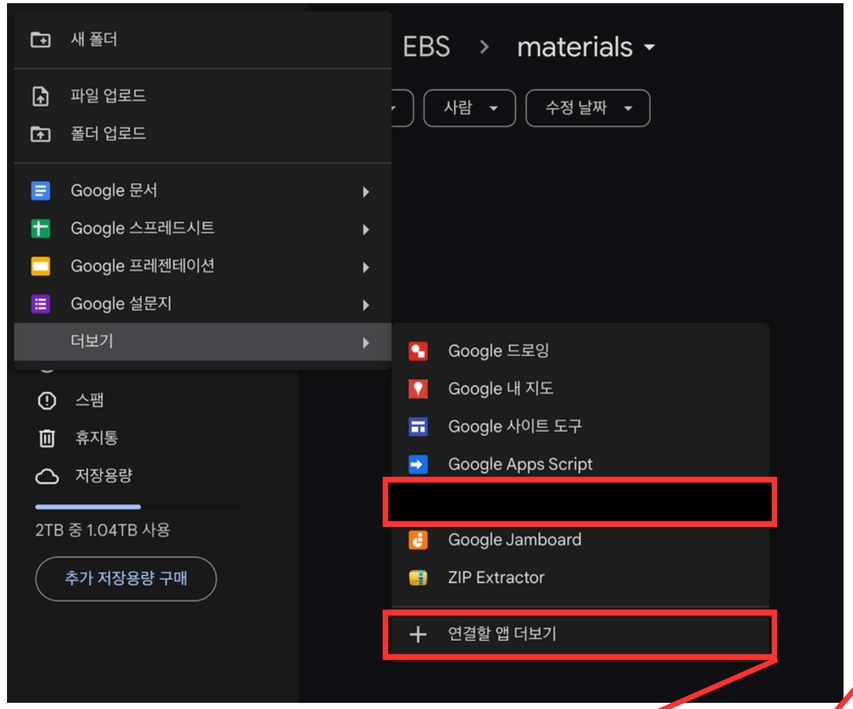
GOOGLE COLABORATORY

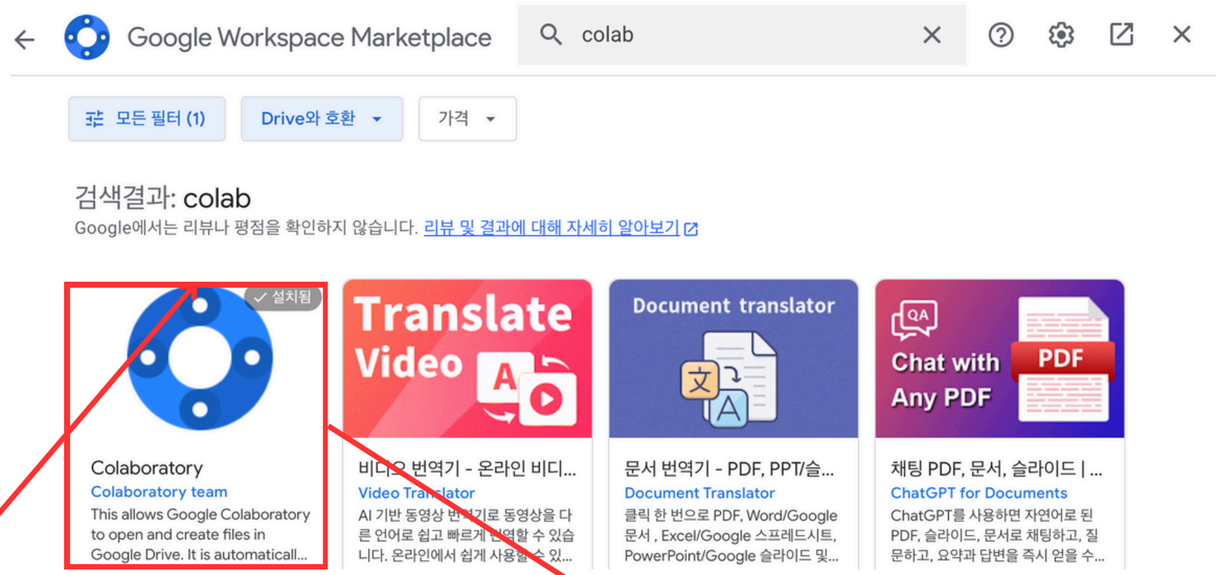
COLAB = 구글이 빌려주는 컴퓨터
DRIVE = 외장하드

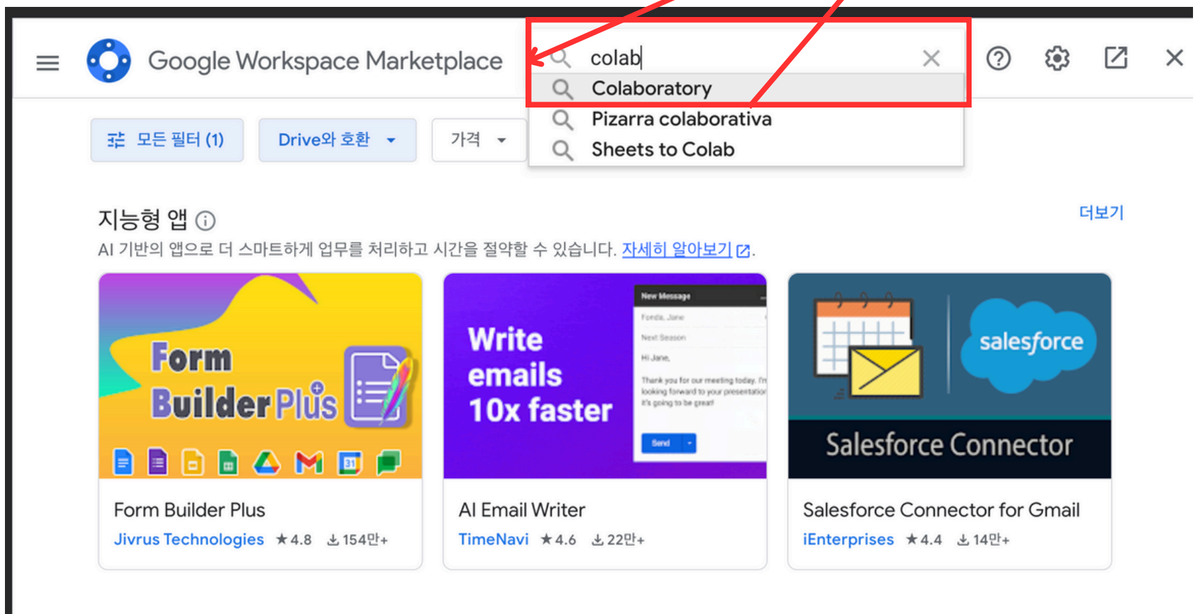


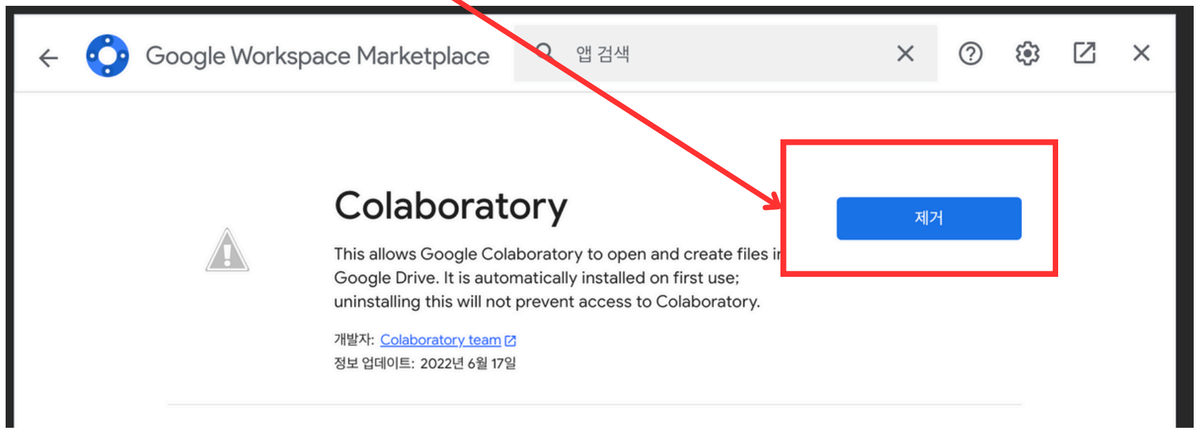


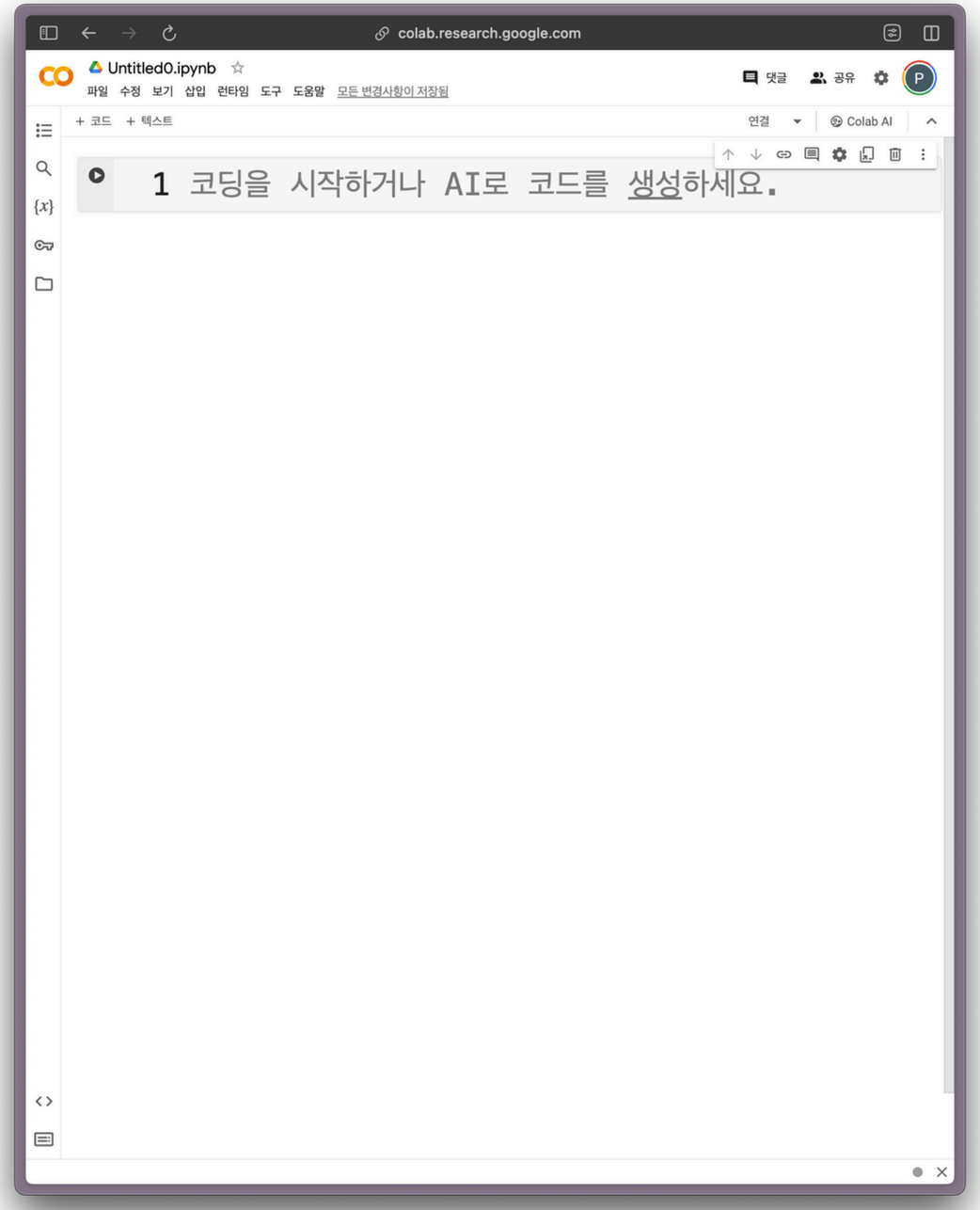
만약 *COLAB*이 없다면? → 연결할 앱 더보기 → *COLAB* 검색 →설치





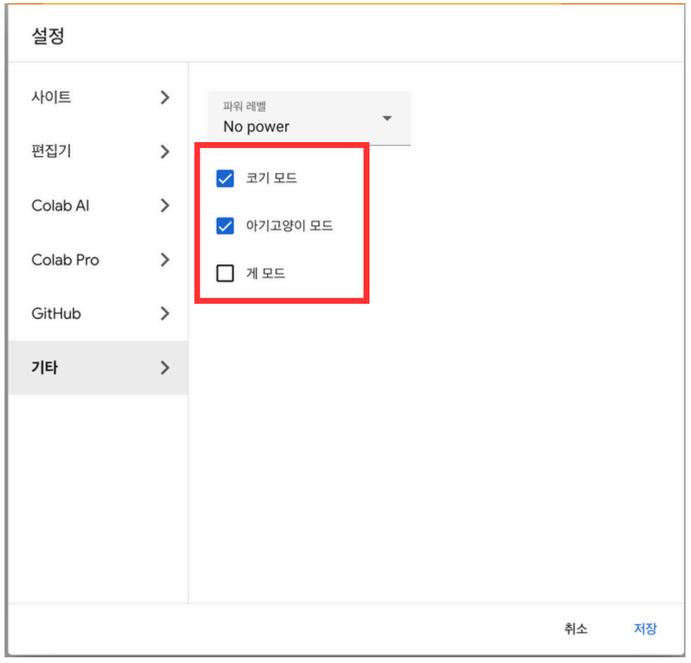


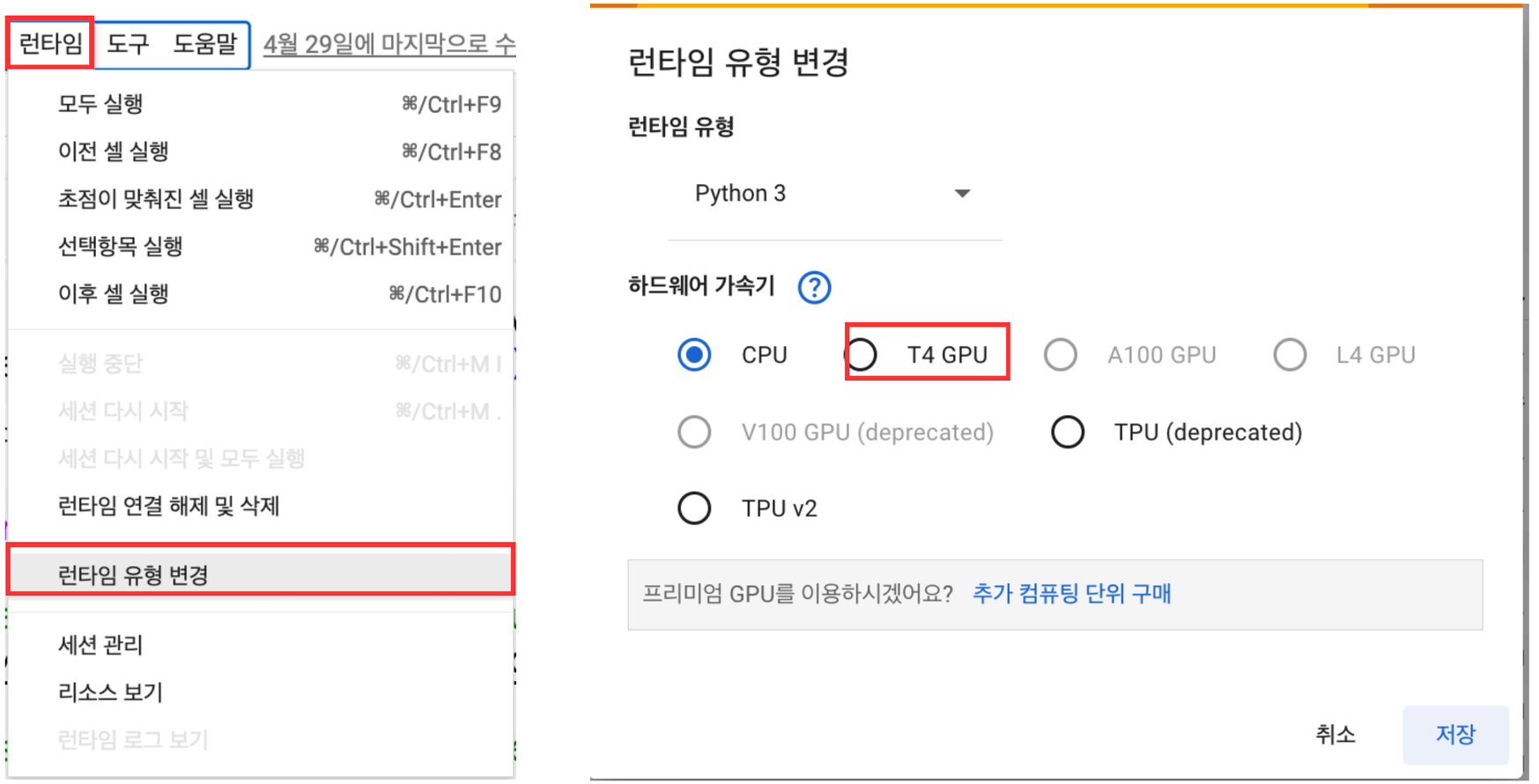



colab.research.google.com
Untitled0.ipynb
파일 수정 보기 삽입 런타임 도구 도움말 모든 변경사항이 저장됨
댓글 공유
+ 코드 + 텍스트
연결 Colab AI
1 코딩을 시작하거나 AI로 코드를 생성하세요.

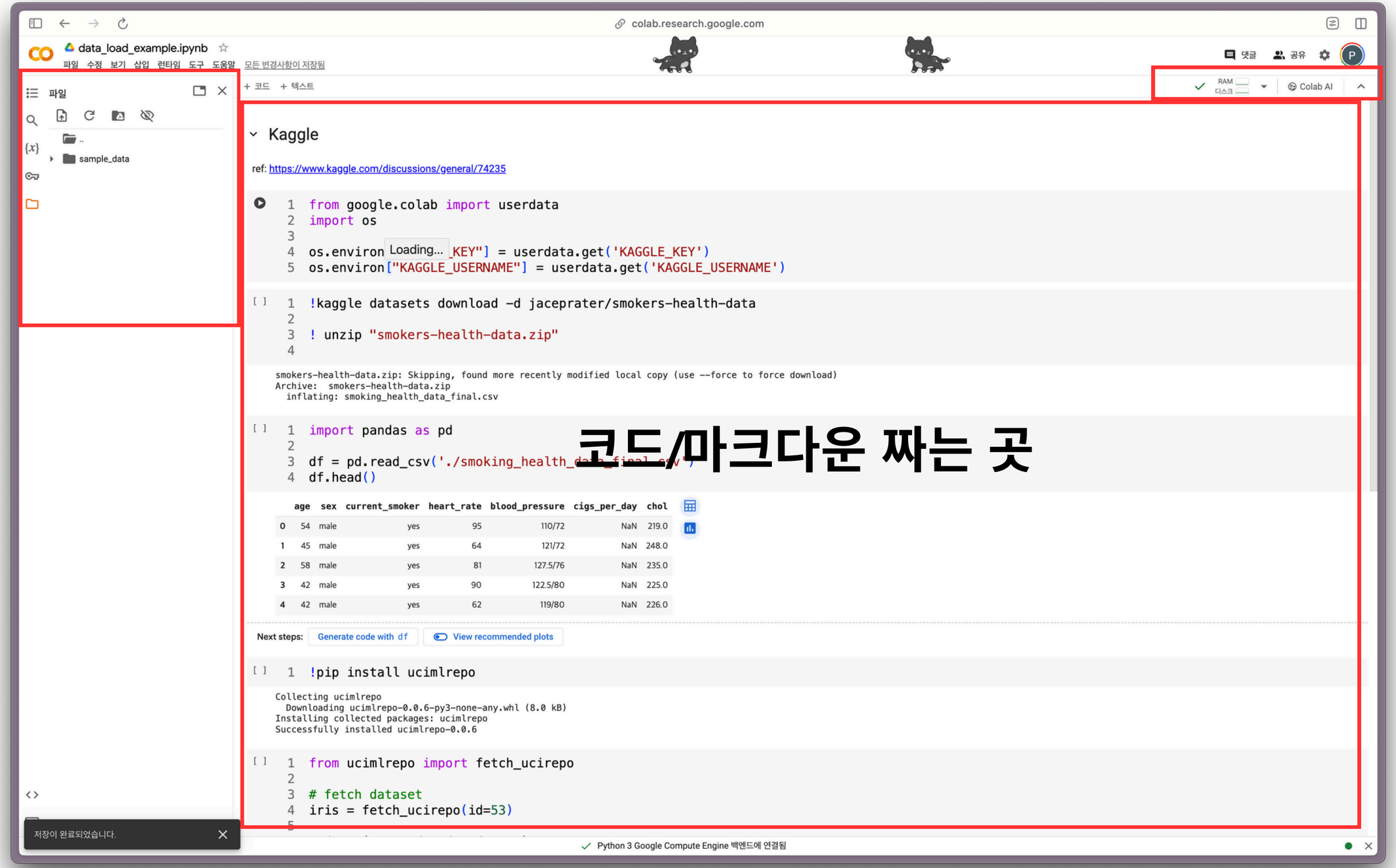
Untitled0.ipynb
파일 수정 보기 삽입 런타임 도구 도움말 모든 변경사항이 저장됨
+ 코드 + 텍스트
명령 팔레트 ⌘/Ctrl+Shift+P
설정
단축키 ⌘/Ctrl+M H
비교 노트북
1 코딩을

설정
사이트
편집기
Colab AI
Colab Pro
GitHub
기타
편집기 키 바인딩
default
글꼴 크기(픽셀)
36
코드 렌더링에 사용되는 글꼴 모음
monospace
들여쓰기 너비(공백 개수)
4
세로 눈금자 열
80
코드 완성 제안을 자동으로 표시
행 번호 표시
들여쓰기 가이드 표시
취소
저장


설정
사이트
편집기
Colab AI
Colab Pro
GitHub
기타
파워 레벨
No power
코기 모드
아기고양이 모드
게 모드
취소
저장

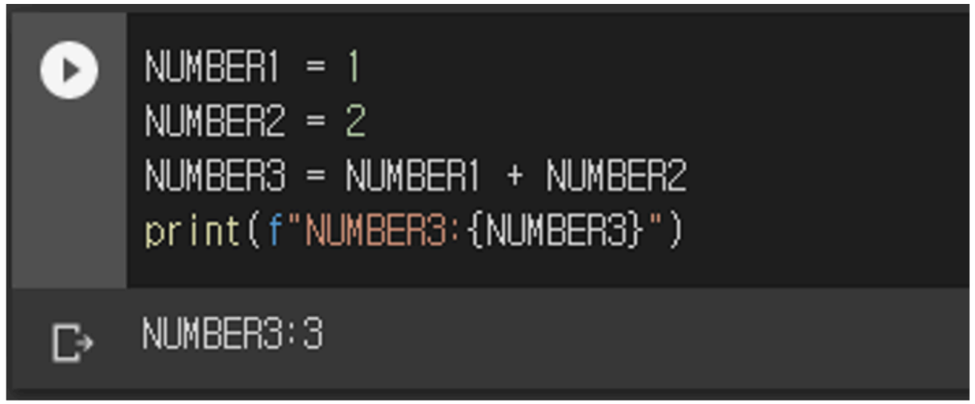


월 5만원 내면 A100도 쓸수있음! (쾌적!)
제가 애용합니다

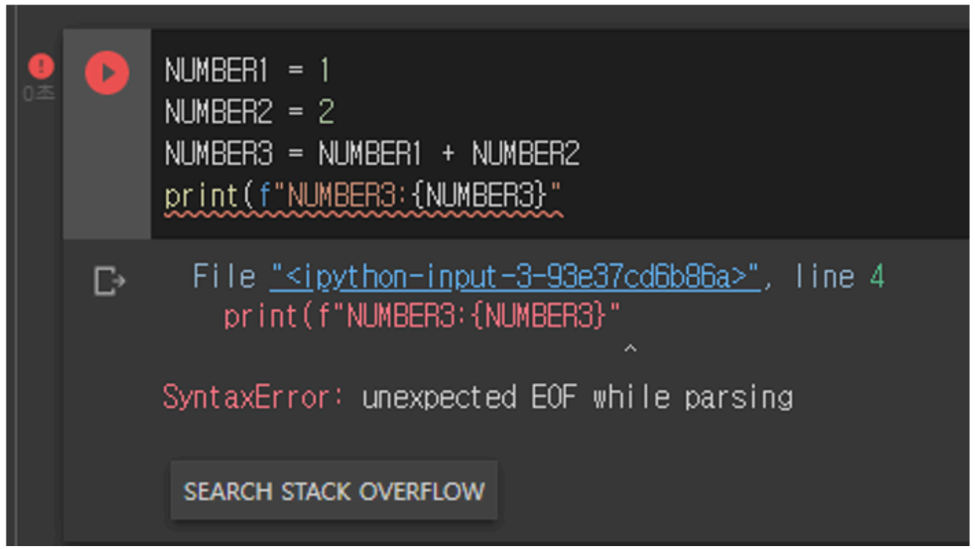


코드/마크다운 짜는 곳

연결

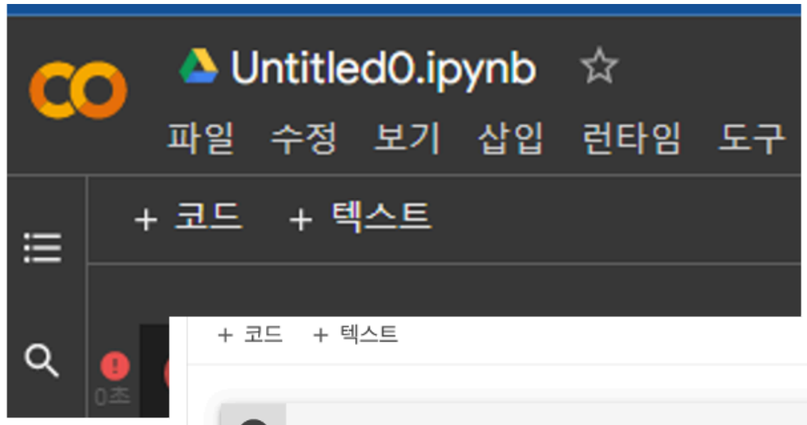


코드를 입력하고 실행 버튼을 눌러 실행시킬 수 있습니다

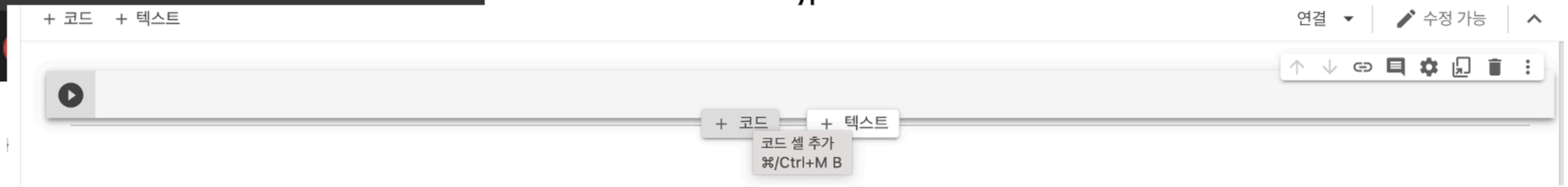


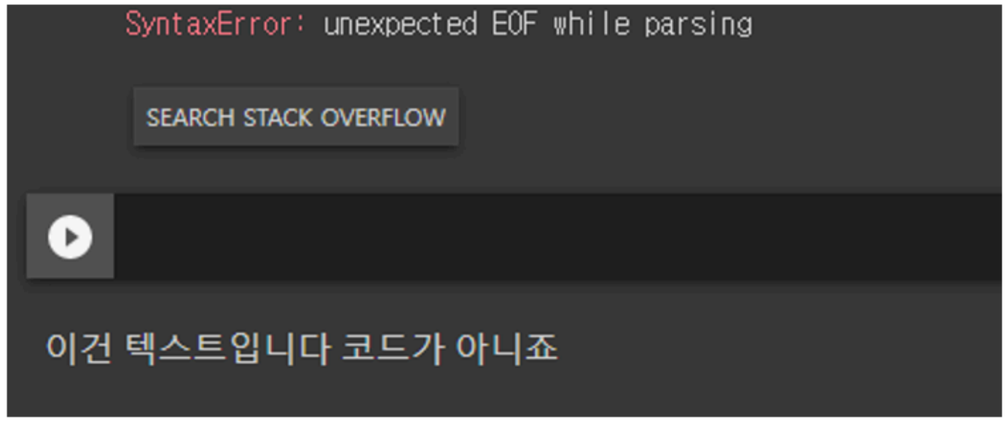
에러가 난 상황입니다!

- 괄호를 닫지 않아서 생긴 에러입니다



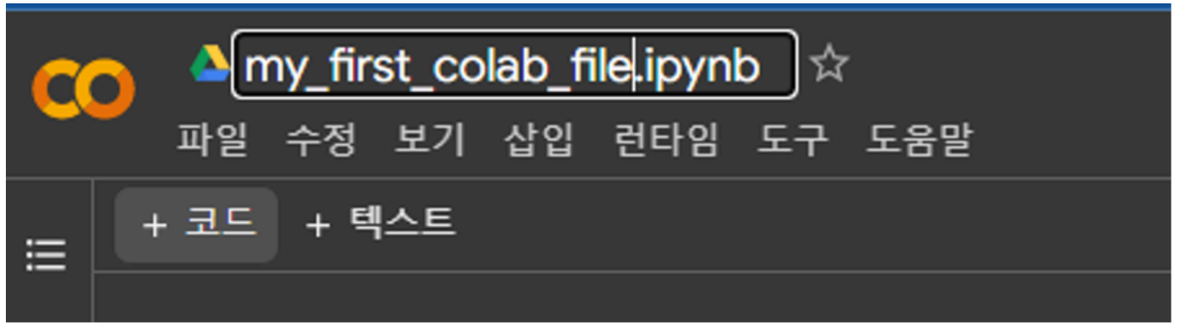
코드나 텍스트를 추가하려면 좌측 상단의 버튼이나
블록 bottom에 마우스를 올려 나오는 버튼 클릭하기



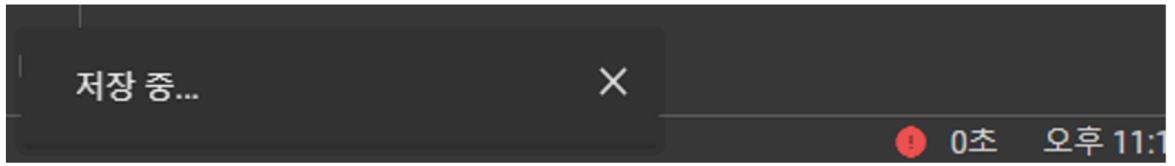


텍스트 블록은 실행이 되지 않으므로
자유롭게 설명을 써놓을 수 있습니다

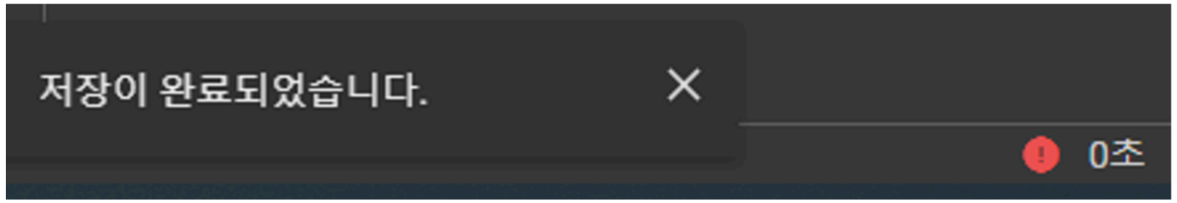
마크다운 문법으로 작성



CTRL + S 키 눌러서 저장
(자동저장이 되긴 하지만, 안전을 위해 자주 저장해줍시다!)

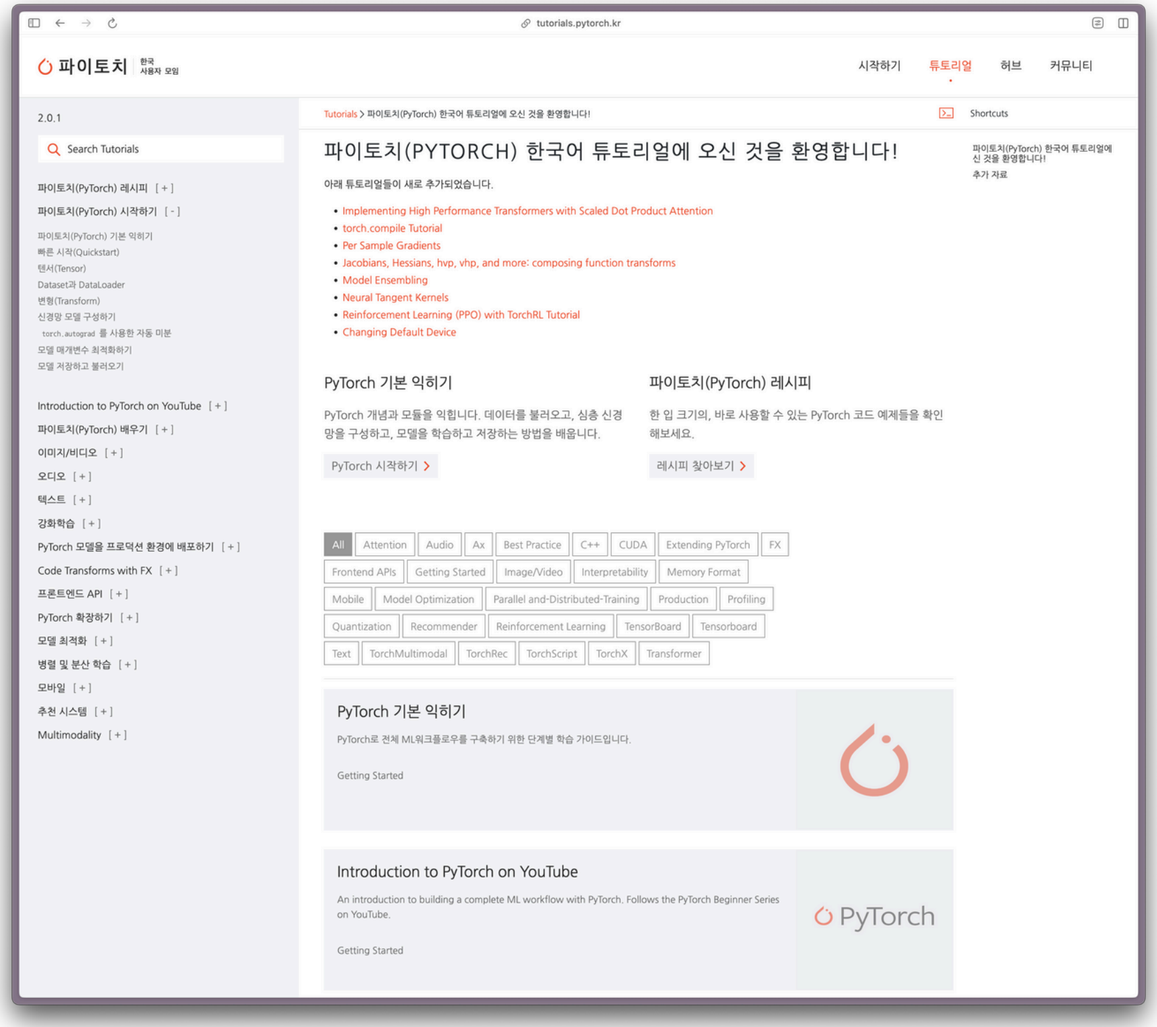


추후에 다른 컴퓨터에서도 구글 계정으로 로그인 하면
드라이브에서 손쉽게 오늘 작성한 코랩 파일에 접근 가능



튜토리얼을 사랑합시다

HTTPS://TUTORIALS.PYTORCH.KR/



라이브러리 IMPORT

0초

```
import torch
from torch import nn
from torch.utils.data import DataLoader
import torchvision
from torchvision import datasets
from torchvision.transforms import ToTensor
```

GPU 연결

```
# Get cpu or gpu device for training.
device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Using {device} device")
```

DATASET 만들기

1초

```
# Download training data from open datasets.
training_data = datasets.FashionMNIST(
    root="data",
    train=True,
    download=True,
    transform=ToTensor(),
)

# Download test data from open datasets.
test_data = datasets.FashionMNIST(
    root="data",
    train=False,
    download=True,
    transform=ToTensor(),
)
```

```
Downloading http://fashion-mnist.s3-website.eu-centr
Downloading http://fashion-mnist.s3-website.eu-centr
100% 26421880/26421
```

DATALOADER 만들기

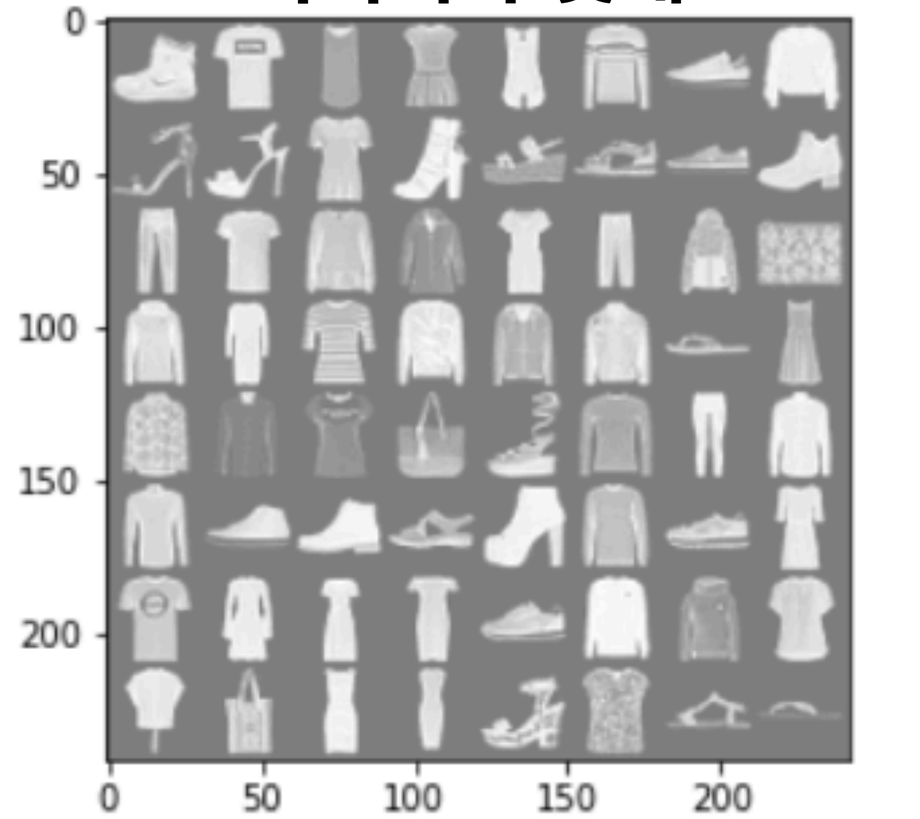
```
batch_size = 64

# Create data loaders.
train_dataloader = DataLoader(training_data, batch_size=batch_size)
test_dataloader = DataLoader(test_data, batch_size=batch_size)

for X, y in test_dataloader:
    print(f"Shape of X [N, C, H, W]: {X.shape}")
    print(f"Shape of y: {y.shape} {y.dtype}")
    break
```

```
Shape of X [N, C, H, W]: torch.Size([64, 1, 28, 28])
Shape of y: torch.Size([64]) torch.int64
```

DATALOADER TEST

```
import matplotlib.pyplot as plt
import numpy as np

# functions to show an image


def imshow(img):
    img = img / 2 + 0.5     # unnormalize
    npimg = img.numpy()
    plt.imshow(np.transpose(npimg, (1, 2, 0)))
    plt.show()


# get some random training images
dataiter = iter(train_dataloader)
images, labels = dataiter.next()

# show images
imshow(torchvision.utils.make_grid(images))
# print labels
print(' '.join(f'{classes[labels[j]]:5s}' for j in range(batch_size)))
```

이미지가 몇개?



Ankle boot T-shirt/top T-shirt/t

MODEL 정의

생성자

동작

```
# Define model
class NeuralNetwork(nn.Module):
    def __init__(self):
        super(NeuralNetwork, self).__init__()
        self.flatten = nn.Flatten()
        self.linear_relu_stack = nn.Sequential(
            nn.Linear(28*28, 512),
            nn.ReLU(),
            nn.Linear(512, 512),
            nn.ReLU(),
            nn.Linear(512, 10)
        )

    def forward(self, x):
        x = self.flatten(x)
        logits = self.linear_relu_stack(x)
        return logits

model = NeuralNetwork().to(device)
print(model)
```

LOSS & OPTIMIZER

0초

```
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=1e-3)
```

학습코드

0초

```python
def train(dataloader, model, loss_fn, optimizer):
    size = len(dataloader.dataset)
    model.train()
    for batch, (X, y) in enumerate(dataloader):
        X, y = X.to(device), y.to(device)

        # Compute prediction error
        pred = model(X)
        loss = loss_fn(pred, y)

        # Backpropagation
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        if batch % 100 == 0:
            loss, current = loss.item(), batch * len(X)
            print(f"loss: {loss:>7f}  [{current:>5d}/{size:>5d}]")
```

테스트 코드

0초

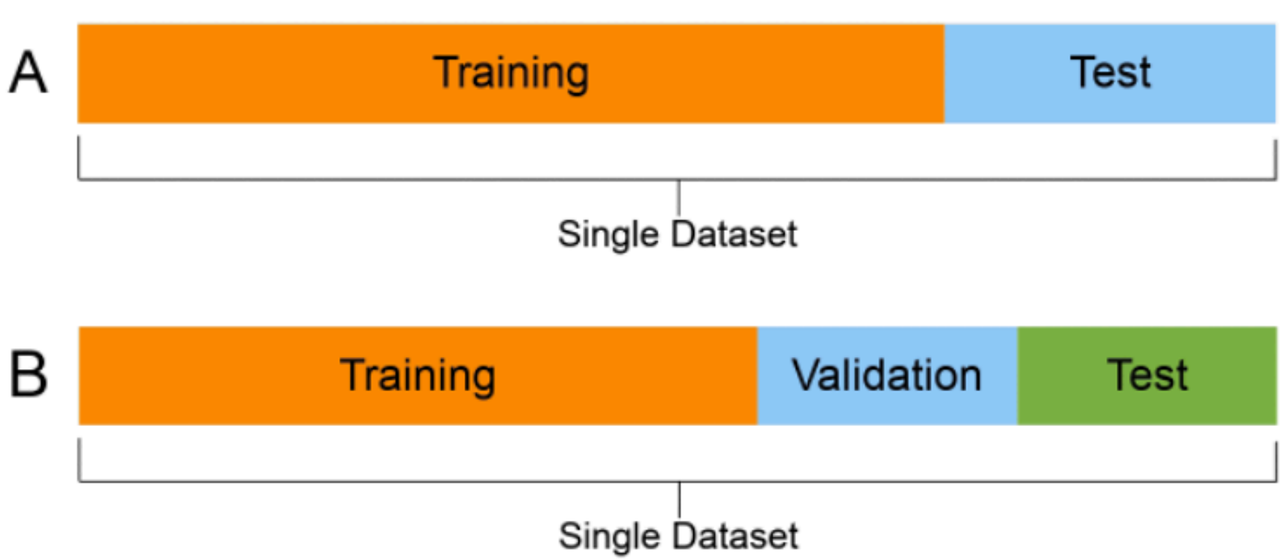
```
[21] def test(dataloader, model, loss_fn):
         size = len(dataloader.dataset)
         num_batches = len(dataloader)
         model.eval()
         test_loss, correct = 0, 0
         with torch.no_grad():
             for X, y in dataloader:
                 X, y = X.to(device), y.to(device)
                 pred = model(X)
                 test_loss += loss_fn(pred, y).item()
                 correct += (pred.argmax(1) == y).type(torch.float).sum().item()
         test_loss /= num_batches
         correct /= size
         print(f"Test Error: \n Accuracy: {(100*correct):>0.1f}%, Avg loss: {test_loss:>8f} \n")
```

학습 실행

```
epochs = 2
for t in range(epochs):
    print(f"Epoch {t+1}\n-------------------------------")
    train(train_dataloader, model, loss_fn, optimizer)
    test(test_dataloader, model, loss_fn)
print("Done!")
```

```
Epoch 2
-------------------------------
loss: 1.431152  [    0/60000]
loss: 1.240798  [  500/60000]
loss: 0.644093  [ 1000/60000]
```

테스트



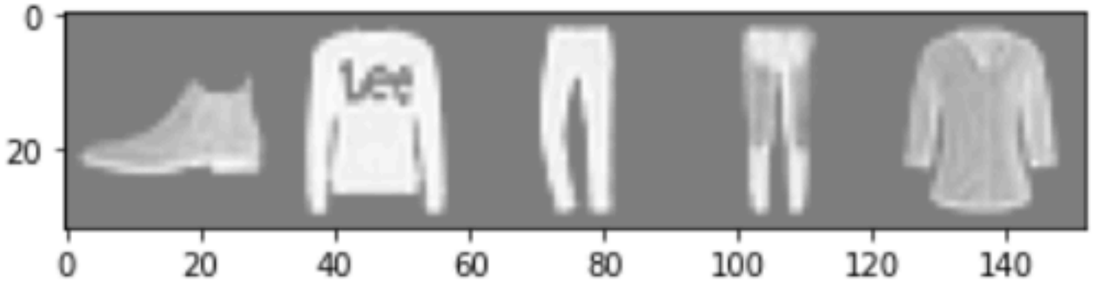

```
loss: 0.540326  [57500/60000]
loss: 0.201658  [58000/60000]
loss: 0.191838  [58500/60000]
loss: 0.466846  [59000/60000]
loss: 0.683563  [59500/60000]
Test Error:
 Accuracy: 79.6%, Avg loss: 0.584817

Done!
```

추론: INFERENCE

테스트용 데이터 준비

0초

```
# get some random training images
dataiter = iter(test_dataloader)
images, labels = dataiter.next()

# show images
imshow(torchvision.utils.make_grid(images))
# print labels
print(' '.join(f'{classes[labels[j]]:5s}' for j in range(batch_size)))
```




```
Ankle boot Pullover Trouser Trouser Shirt
```

0초

```
model.eval()
x, y = test_data[0][0], test_data[0][1]
with torch.no_grad():
    pred = model(x)
    predicted, actual = classes[pred[0].argmax(0)], classes[y]
    print(f'Predicted: "{predicted}", Actual: "{actual}"')
```

```
Predicted: "Ankle boot", Actual: "Ankle boot"
```

0초

```
model.eval()
x, y = test_data[1][0], test_data[1][1]
with torch.no_grad():
    pred = model(x)
    predicted, actual = classes[pred[0].argmax(0)], classes[y]
    print(f'Predicted: "{predicted}", Actual: "{actual}"')
```

```
Predicted: "Pullover", Actual: "Pullover"
```

모델 저장 및 호출

COLAB은 관심을 주지 않으면 끊깁니다.. (ㅜㅜ공짜니까요..)　　저장하세요!

0초

```
torch.save(model.state_dict(), "model.pth")
print("Saved PyTorch Model State to model.pth")
```

```
Saved PyTorch Model State to model.pth
```

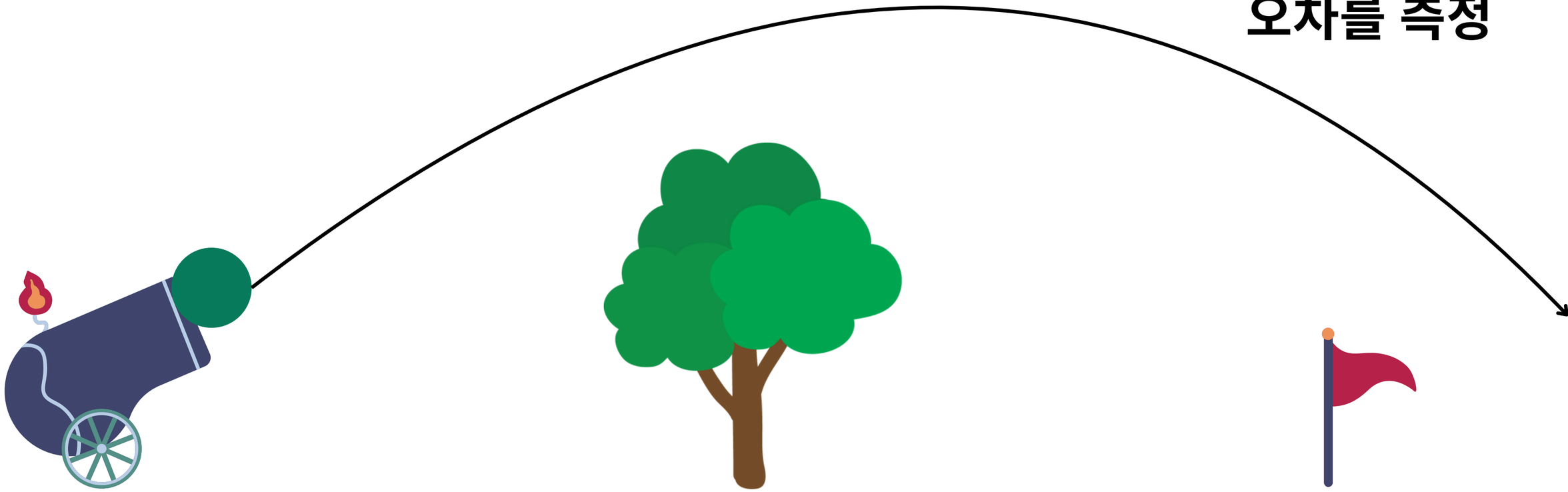
저장한 네트워크를 정의하는 코드 필요

```
model = NeuralNetwork()
model.load_state_dict(torch.load("model.pth"))
```

그러면 LOSS나 OPTIMIZER 같은 것들은 어떻게 저장할까요?

대포를 쏜다 = 현재 네트워크에 학습 데이터를 넣어 실행

오차를 측정



대포의 위치와 각도를 수정한다.

0초

```python
def train(dataloader, model, loss_fn, optimizer):
    size = len(dataloader.dataset)
    model.train()
    for batch, (X, y) in enumerate(dataloader):
        X, y = X.to(device), y.to(device)

        # Compute prediction error
        pred = model(X)
        loss = loss_fn(pred, y)

        # Backpropagation
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        if batch % 100 == 0:
            loss, current = loss.item(), batch * len(X)
            print(f"loss: {loss:>7f}  [{current:>5d}/{size:>5d}]")
```

BATCH?

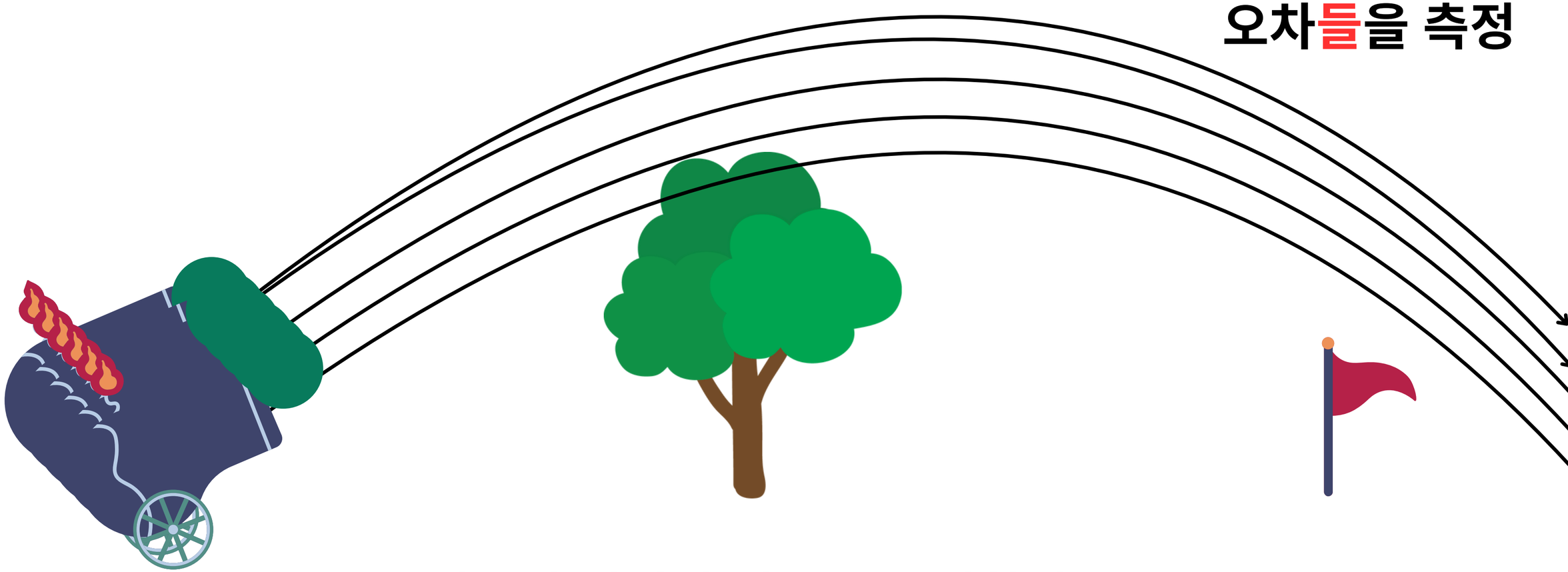
입력X, 목표Y, 실제출력 PRED

학습이 이루어지는 부분

실제로는 여러개를 한번에 넣네요?

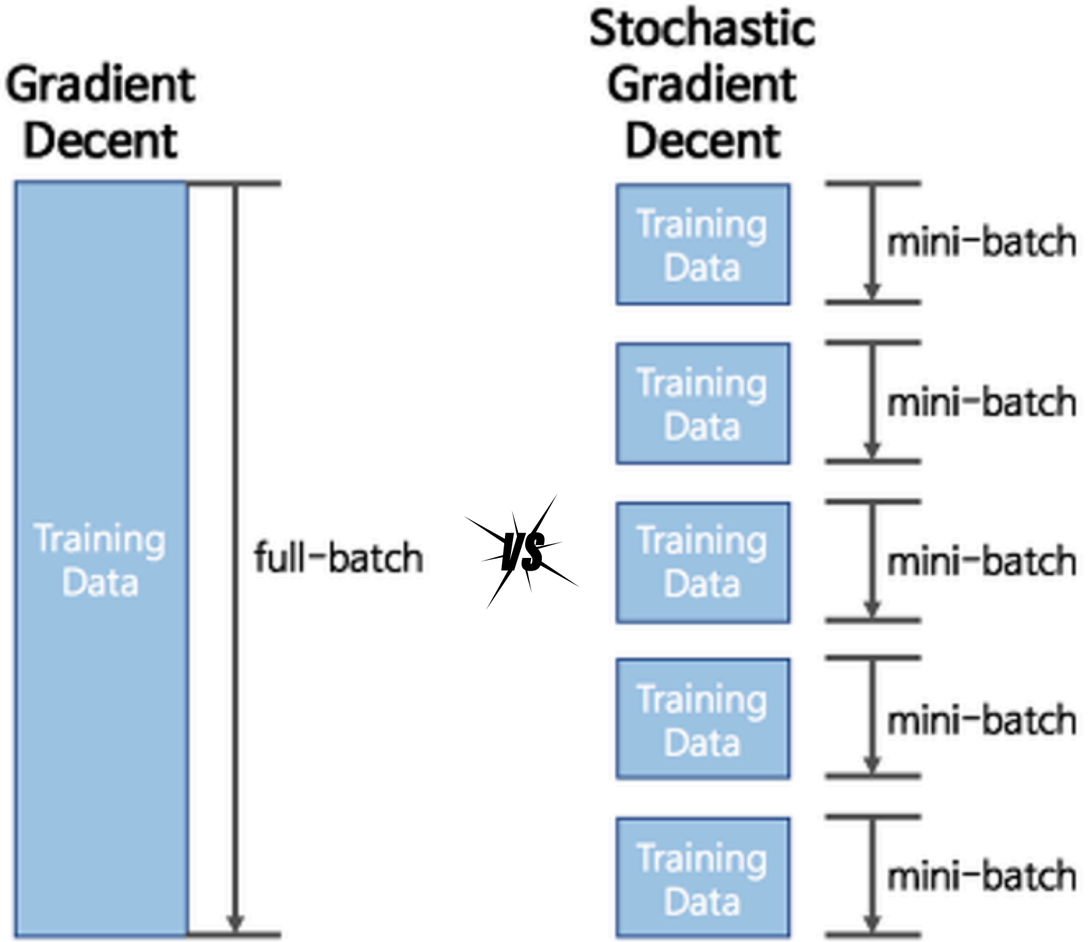
대포를 쏜다 = 현재 네트워크에 학습
데이터들을 넣어 실행

오차들을 측정



대포의 위치와 각도를 수정한다.

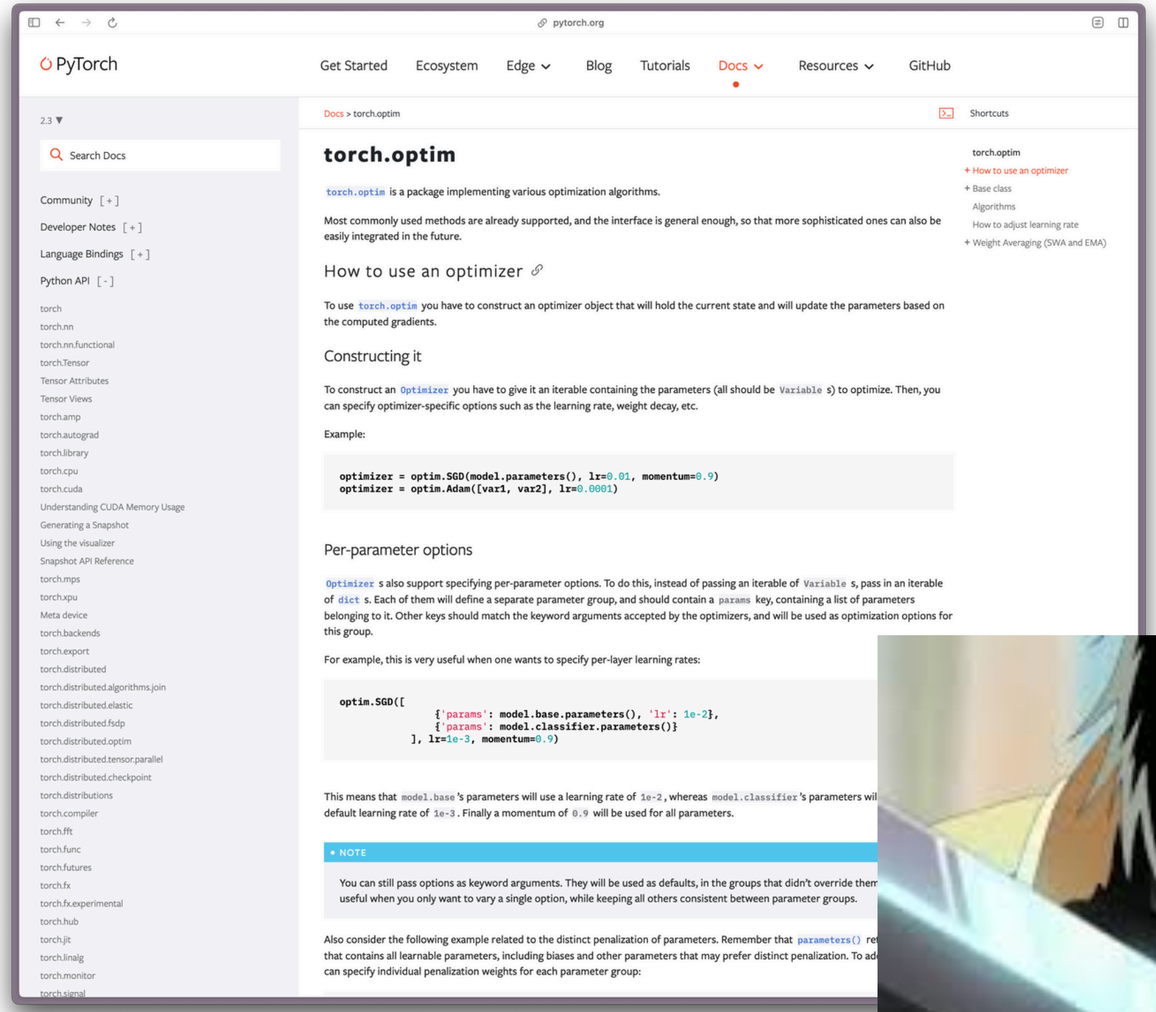
전체를 넣어서
한번에 계산

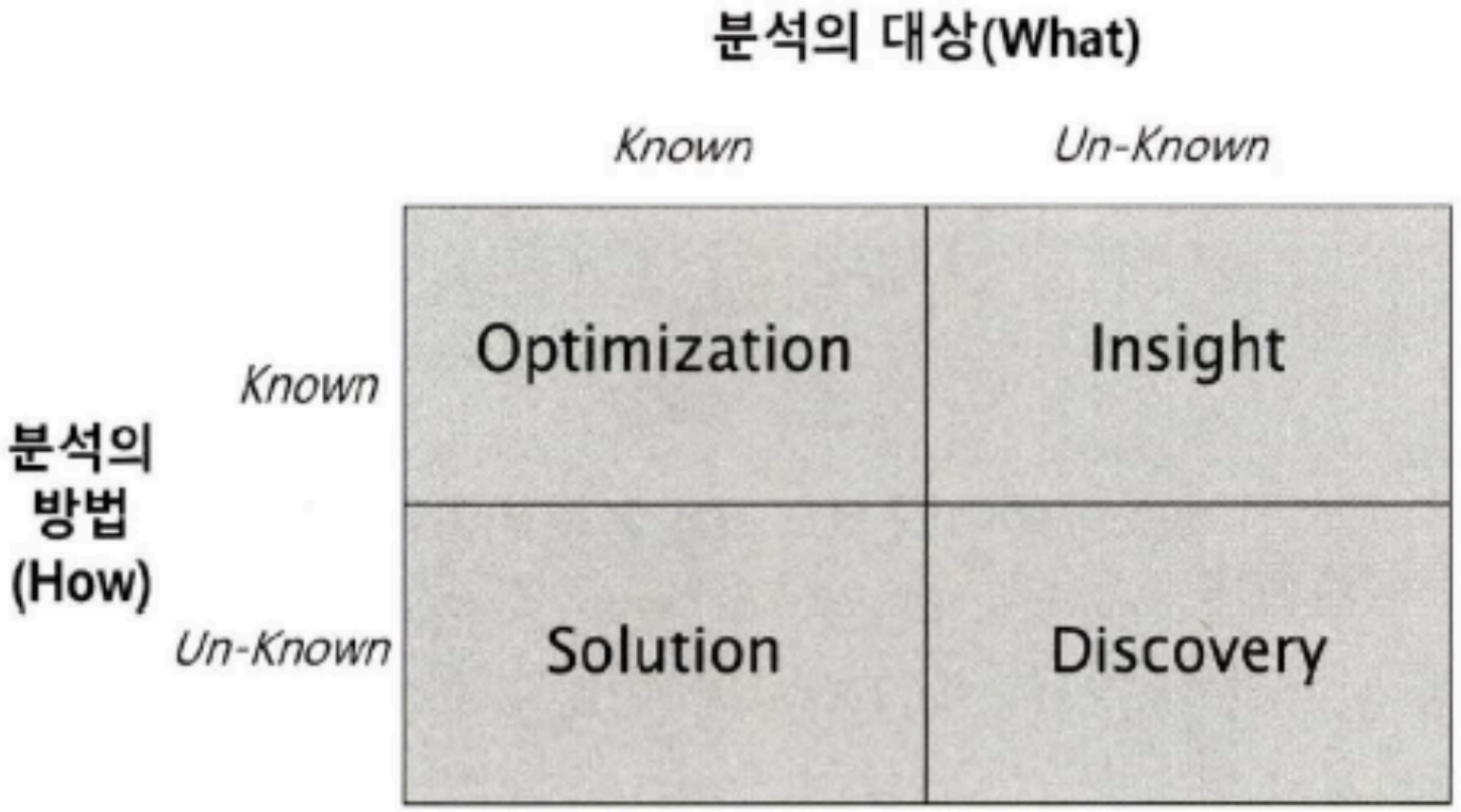


일부 혹은 하나만
넣어서 계산

다양한 연구를 통해 다양한 방법이 제시 됨.

여러분이 필요한 거의 대부분의 기능을 TORCH가 이미 거의 다 만들어 놓았습니다!




분석의 대상(What)
Known
Un-Known
분석의 방법 (How)
Known
Un-Known
Optimization
Insight
Solution
Discovery